专

C 公司业务应用运行平台项目进度管理研究

张俊侨

北京理工大学

C 公司业务应用运行平台项目进度管理研究

姓名：张俊侨

学号：3420240587

学院：管理学院

2026 年 5 月

中图分类号：TQ028.1
UDC 分类号：540
学校代码：10007

★ 特别类型
□ 工程硕博士专项
□ 交叉研究方向
□ 政府项目留学生

C 公司业务应用运行平台项目进度管理研究

作 者 姓 名　张俊侨
二级培养单位　管理学院
指 导 教 师　邱青安副教授
行业合作导师　何傈
答辩委员会主席　颜志军教授
申 请 类 别　工程管理硕士
学 位 领 域　工程管理
学位授予单位　北京理工大学
论文提交日期　2026 年 5 月 28 日

Research on the Project Schedule Management of C Company's Business Application Operation Platform

Candidate Name: Junqiao Zhang

School or Department: School of Management

Faculty Mentor: Assoc. Prof. Qing'an Qiu

Industry Collaboration Mentor: Li He

Chair, Thesis Committee: Prof. Zhijun Yan

Degree Applied: Master of Engineering Management

Major: Engineering Management

Degree by: Beijing Institute of Technology

Submission date of the paper: May 28th，2026

研究成果声明

本人郑重声明：所提交的学位论文是我本人在指导教师的指导下独立完成的研究成果。文中所撰写内容符合以下学术规范（请勾选）：

☑ 论文综述遵循“适当引用”的规范，全部引用的内容不超过 50%。

☑ 论文中的研究数据及结果不存在篡改、剽窃、抄袭、伪造等学术不端行为，并愿意承担因学术不端行为所带来的一切后果和法律责任。

☑ 文中依法引用他人的成果，均已做出明确标注或得到许可。

☑ 论文内容未包含法律意义上已属于他人的任何形式的研究成果，也不包含本人已用于其他学位申请的论文或成果。

☑ 与本人一同工作的合作者对此研究工作所做的任何贡献均已在学位论文中作了明确的说明并表示了谢意。

特此声明。

签　　名：  　　日 期：2026.05.28

关于学位论文使用权的说明

本人完全了解北京理工大学有关保管、使用学位论文的规定，其中包括：

①学校有权保管、并向有关部门送交学位论文的原件与复印件；

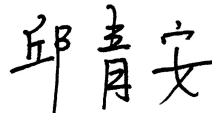
②学校可以采用影印、缩印或其它复制手段复制并保存学位论文；

③学校可允许学位论文被查阅或借阅；

④学校可以学术交流为目的,复制赠送和交换学位论文；

⑤学校可以公布学位论文的全部或部分内容（保密学位论文在解密后遵守此规定）。

签　　名：  　　日期：2026.05.28

导师签名：  　　日期：2026.05.28

致谢

时光荏苒，两年的研究生学习生涯进入尾声，心中感慨万千。这段时间除了在学校学习到的知识，还有同学之间的友情、浓浓的师生情。在此，谨以最朴素的文字，向所有助我完成此程的师长、亲友与同仁致以诚挚谢意。

首先谨向我尊敬的导师邱青安老师致以诚挚的谢意和崇高的敬意。从我进入学校读研深造伊始老师就对我严格要求，并帮助我开展本研究的选题，一直到论文完成，每周例会上都会帮助我整理写作思路，将碎片化的内容凝练成研究框架。每一步的前进都离不开老师的精心指导与谆谆教诲。老师渊博的知识、丰富的经验和严谨的治学态度是我一生学习的榜样。

感谢母校为我提供了良好的学习环境与科研平台，让我在硕士阶段能够系统学习项目管理相关理论知识，积累扎实的学术基础，为论文研究奠定了坚实前提。同时，感谢学院各位授课老师，他们在课堂上传授的专业知识与实践经验，拓宽了我的研究视野，为本文的研究提供了重要的理论支撑。

感谢我的同窗好友与同门伙伴，在硕士学习与论文撰写期间，我们相互交流、彼此支持，分享研究思路、探讨疑难问题，在互助共进中收获了知识与友谊，也让我在科研道路上不再孤单。感谢我的家人，他们的理解、包容与默默支持，是我克服困难、潜心研究的坚强后盾，给予我无限的动力与勇气。

最后，感谢参与本文评审与答辩的各位专家、老师，他们提出的宝贵意见与建议，对本文的完善起到了重要作用。同时，感谢这段硕士旅程中努力的自己，也感谢所有给予我帮助与支持的人。

摘要

随着全球软件行业的迅猛发展及软件项目的数量不断增多、规模不断增大，项目进度管理已经成为软件项目按时交付的关键环节之一。不仅要了解整个软件生命周期中各阶段的任务来协调配合，而且目前行业中还存在诸多不足，其中最突出的就是进度问题，使得很多项目不能在预定时间内完成。C 公司是一家专注于广电行业信息化建设的高科技企业，负责 Z 公司业务应用运行平台开发项目，但由于之前管理项目的经验有限，在项目过程中缺乏沟通协调、进度拖延等情况，使得该项目在完成度及客户满意度上较低，给公司在业界造成了不良的影响。

针对 Z 公司在业务应用运行平台项目的进度管理中存在的问题，本文采用问卷调查的方式，对 C 公司目前的项目管理情况进行调研，并运用鱼骨图对问卷结果进行结构化归因，发现该项目进度延误主要源于四个核心方面：一是工作任务划分过于粗糙；二是工作资源调配不够合理；三是工作任务工期评估标准主观化，缺乏科学的估算方法支撑；四是缺乏有效的沟通机制与全面的进度监督系统，导致信息传递不畅、风险预警不及时。因此，本文制定了全方位的进度优化改进措施：首先，在编制项目进度计划阶段，以 WBS 任务分解为基础，确定各项工作的先后关系和责任主体，采用 PERT 三点估算法进行科学的工期估算，识别初始关键路径及工期缺口；其次，采用关键路径法（CPM）进行进度与流程优化，通过拆分核心瓶颈任务、合理并行执行关键活动，并缩短审批环节与冗余工时，以提升整体效率；接着，利用关键链法分析资源约束、选择合理的资源分配方案、消除安全时间并科学设置缓冲区，提升了进度计划的实操性和抗风险能力；最后从管理制度上制定有效的沟通制度、变更控制制度以及奖惩制度，再辅以缓冲区监控三色图对项目实施全过程进行直观的进度控制，从而保证项目的顺利进行。

研究不仅为 C 公司该项目规避进度风险、保障按期交付提供支持，还能拓展软件项目进度管理理论在企业数字化转型垂直领域的应用场景，为其他同类企业的项目管理实践提供理论参考与实操范例，进而推动整个行业数字化转型项目的高效落地。

关键词：软件研发；进度管理；关键链；项目管理

Abstract

On the one hand, against the background of the flourishing worldwide software industry and the constant increase of both quantity and size of software projects,project schedule management became an essential factor for success in delivering software projects. On one side, it is important to know the key points on the lifecycle of developing software to make advancement in a synchronized way.Meanwhile, it is common for the industry to have some deficiencies and one of them is related to the schedules where most of the project cannot meet its deadline. For instance, company C,which provides digital transformation solution to enterprise took in charge the development project of Z Company’s business application operation platform.But in the past it's had a poor record of project management with inadequate experience and lack of complete communication channel,and delay, which results directly in a poor project delivery rate, loss of customer satisfaction, and reputational damage among others in the marketplace.

Regarding the problems existing in the progress management of the business application running platform project of Z Company, this article adopts the questionnaire survey method to investigate the current project management situation of C Company, and uses the fishbone diagram to structurally attribute the questionnaire results. It is found that the main reasons for the delay of the project progress are four core aspects: first, the task division is too rough; second, the allocation of work resources is not reasonable; third, the assessment standard of task duration is subjective and lacks the support of scientific estimation methods; fourth, there is a lack of effective communication mechanism and a comprehensive progress supervision system, resulting in poor information transmission and untimely risk warning. Therefore, this article has formulated comprehensive progress optimization improvement measures: Firstly, in the stage of preparing the project progress plan, based on the WBS task decomposition, determine the sequence relationship and responsibility subjects of each task, and use the PERT three-point estimation method to conduct scientific duration estimation, identify the initial critical path and duration gap; Secondly, use the critical path method (CPM) for progress and process optimization, by splitting the core bottleneck tasks, reasonably and concurrently executing key activities, and

shortening the approval process and redundant working hours to improve overall efficiency; Then, use the critical chain method to analyze resource constraints, select reasonable resource allocation schemes, eliminate safe time and scientifically set buffers, improving the practicality and risk resistance of the progress plan; Finally, formulate effective communication systems, change control systems and reward and punishment systems from the management system aspect, and supplement with the three-color monitoring chart of the buffer zone to conduct intuitive progress control of the entire project implementation process, thereby ensuring the smooth progress of the project.

This study can not only provide a basis for Company C to eliminate schedule risk and guarantee the timely completion of this project, but also enriches the application cases of software project schedule management theory in the vertical direction of enterprise digital transformation.providing theoretical basis and case reference to the project management work of other similar companies,, so as to encourage effective implementation of digitization programs across the sector.

Key Words: Software Development; Progress Management; Critical Chain; Project Management

目次

1 绪论 1
1.1 研究背景 1
1.2 问题提出 2
1.3 研究意义 3
1.4 研究内容 4
1.5 研究方法与技术路线 5
1.5.1 研究方法 5
1.5.2 技术路线 6
2 相关理论基础与文献综述 8
2.1 相关理论 8
2.1.1 项目管理概述 8
2.1.2 项目进度管理概述 8
2.2 文献综述 9
2.2.1 软件项目管理研究现状 9
2.2.2 软件进度管理研究现状 11
2.2.3 文献评述 12
3 C 公司项目进度管理现状及问题识别 14
3.1 业务应用运行平台项目简介 14
3.1.1 C 公司概况 14
3.1.2 业务应用运行平台项目基本情况介绍 15
3.1.3 组织结构及团队 16
3.2 业务应用运行平台进度管理现状 17
3.2.1 项目工作分解结构 17
3.2.2 项目进度计划 18
3.2.3 项目监控措施 19
3.2.4 项目进度情况 20
3.3 业务应用运行平台进度管理问题及原因 20
3.3.1 问卷发放人员、问题确定 20

3.3.2 问卷调查结果整理与分析 21
3.4 本章小结 24
4 C 公司项目进度管理优化方案 25
4.1 基于计划评审技术（PERT）制定进度计划 25
4.1.1 细化 WBS 工作分解 25
4.1.2 工作顺序制定及职责明确 26
4.1.3 项目活动持续时间估算 29
4.1.4 确定关键路径 31
4.2 基于关键路径法（CPM）的进度优化 36
4.2.1 进度优化 36
4.2.2 流程优化 38
4.2.3 基于 CPM 进度优化效果 43
4.3 基于关键链法的资源优化 48
4.3.1 项目资源分配与资源约束分析 48
4.3.2 按照任务优先级重新排序 51
4.3.3 资源分配方案设计与对比分析 52
4.3.4 消除安全时间 56
4.3.5 设置缓冲区 61
4.4 本章小结 70
5 项目进度保障、监控措施与效果评价 71
5.1 项目进度保障措施 71
5.1.1 建立需求变更机制 71
5.1.2 完善沟通交流机制 73
5.1.3 完善绩效考核、激励与晋升制度 75
5.2 项目进度监控 76
5.2.1 缓冲区监控三色图 76
5.2.2 缓冲区监控状态及分析 77
5.3 项目效果评价 78
5.4 本章小结 81
结论 82

参考文献 84
附录 A 87

1 绪论

1.1 研究背景

近年来，全球软件行业呈现出蓬勃发展的态势。从功能上看，最初的软件可能只是完成简单计算或者文字编辑等功能，而现在开发出来的软件如大型 ERP 软件、复杂 App 等都包含了诸多业务和功能点，包含着大量的程序编码、系统整合、与其他硬件及软件之间的相互调用等等。从工信部披露的数据显示，2025 上半年，软件产品收入 15441 亿元，同比增长 10.6%，占全行业收入的 21.9%，其中基础软件收入 903 亿元、同比增长 13.8%，工业软件产品收入 1445 亿元、同比增长 8.8%[1]，且自 2013 年至今，全球工业软件市场规模已以每年 5%-6.5%速度增长，同时软件项目数量不断增长、规模不断扩大，在金融、医疗、教育、交通等领域广泛应用，已经成为促进各行各业转型升级、发展的重要推动力量。

在软件项目数量和规模不断增长的同时，软件项目管理是确保项目成功交付的关键环节。它需要对整个软件开发生命周期有充分的了解，包括明确愿景、规划任务、召集执行人员、评估工作量、制定进度表、监督工作、收集需求、设计和编写软件，以及测试最终产品[2]。与此同时，软件项目管理暴露出诸多问题，尤其是项目进度管理。许多软件项目无法按时交付，据相关统计，大量的软件项目存在不同程度的进度延误情况。国际项目管理大师詹姆斯·刘易斯说：“美国每年花在软件项目上的钱是 2500 亿美元，但是所有投资的项目，最终实现最初确立目标的只有 27%，另有 50%的项目改变了最初确立的目标，还有 23%的项目因为无法完成而被迫取消了。”

C 公司成立于 2006 年，一直致力于为广电行业提供信息化建设方案，在为客户提供高质量的服务中积累了大量的用户资源以及大量的软件产品。但是在具体的实施阶段，有些软件产品在开发的过程中存在严重的延期情况。据内部项目复盘报告显示，近一年来 C 公司承接的项目中，仅约 64%能按合同约定时间交付，超过 36%的项目存在不同程度的交付延迟，其中中大型项目延迟率更高达 43%，而核心症结正是项目管理工作缺乏一套科学系统的管理方法，导致项目推进受阻。这不仅直接影响到客户的满意度，而且由于项目延期完成而造成项目成本严重超标，如项目人员的加班费、额外的硬件设备租借费用等，给公司带来了严重的经济损失以及商业信誉损失。上述情况均说明了 C 公司存在着严重的软件项目进度管理问题，所以急需对其进行软件项目

进度管理优化的研究，以达到提高项目按期交付率、降低成本、提升企业市场竞争力的目的，这对 C 公司的稳定发展具有重要意义。

1.2 问题提出

本文的研究对象为 C 公司业务应用运行平台研发项目，C 公司是一家依托大型传媒集团背景成立的高科技企业，核心聚焦广电行业信息技术开发与集成领域。当前，C 公司承接了 Z 公司“业务应用运行平台”的研发项目，项目整体规划为 9 个月内完成，交付日期为 2025 年 2 月 28 日前，核心目标是通过新平台研发与部署，实现对 Z 公司原有业务平台的全面替换。

从项目实施流程来看，该研发项目需依次完成需求分析、系统架构设计、核心功能研发、系统集成测试及最终部署上线等关键任务模块；从项目价值定位来看，新平台的落地将助力 Z 公司实现核心业务从线下传统模式向线上数字化模式的迁移，进而达成三大核心目标：一是优化业务流程并提升整体运营效率，二是构建规范化的数据管理体系以保障业务数据安全，三是为 Z 公司后续深化数字化转型进程奠定坚实的技术平台基础。

C 公司为了在预期时间内顺利完成 Z 公司业务应用运行平台项目，决定对公司当前发展状况以及公司管理模式进行分析，发现项目管理工作缺乏一套科学的管理方法，这正是导致诸多项目进度延迟交付的症结所在。经过初步研究后得知，在现阶段中 C 公司在项目进度管理上存在以下几个方面的不足：

首先，任务分解过于粗糙，没有将项目目标分解到具体的可以完成的工作节点上，导致团队成员对工作内容和优先级不清晰，出现重复工作或者遗漏的情况。从历次项目复盘和公司统计发现，在任务分解粗糙的前提下，会出现约 10%的重复工作，这无疑浪费了大量的人力与时间成本。同时在遗漏任务方面，平均会遗漏 2~3 项系统间的接口任务，而这些接口是系统之间正常连通和工作的关键部分。除此之外，由于任务分解不够明确，各任务节点的启动条件以及交付时间都没能得到清晰界定。经过对所有任务的梳理与分析，发现其中近 70%的任务都缺乏明确的进度把控基准，项目团队很难准确判断这些任务是否在按计划推进、何时应该完成等，从而为后续进度滞后埋下了极大的隐患，使得项目后续陷入一种进度失控的风险状态。

其次，任务评估体系存在缺陷，即不能正确地估计一个任务的难度及完成该任务所需要的工时和人力资源。据统计，C 公司近半年的项目中，任务工时预估偏差率平

均达 35%，其中新增需求的工时预估偏差尤为严重：如某客户提出的“人员信息同步”需求，小组成员依据过去的经验主观判断只需 5 个工作日就能完成，而在实际实施过程中由于涉及的数据接口来源于不同的系统以及跨部门之间协作等原因，耗时长达 13 个工作日，偏差率高达 160%。这种偏差的产生主要源于两方面：一是缺乏历史数据作为支撑，在进行新项目评估的过程中有近 70%是依靠项目经理自己的主观判断而决定的，并没有类似项目的工时数、资源耗费等可以参考。造成近四分之一的工作由于估算不充分而终止；二是估算过程过于乐观，例如客户在有了新的创意后提出一个新需求，项目组没有对需求进行适当的评估就轻易接受，结果使得原来计划的工作量增加，进度拖期。

再次，资源分配不合理，表现为资源在某些节点上配置过剩，造成资源空置和浪费的同时，又在关键节点上出现资源短缺现象。特别是在重要的任务节点上缺乏合理的人力资源配置，在关键岗位员工离职或者休假离开项目团队的情况下，会造成当下工作无人负责，比如某次历史项目中的一个重要节点由于一名高级工程师离职后，此工作项陷入停滞状态，后来临时抽调的一位初级工程师及一位中级工程师连续加班两周才补回进度，期间团队平均周加班时长较正常水平增加 50%，导致员工满意度下降，甚至需要牺牲部分质量来换取项目进度以至于需求方不满意导致返工。资源的不合理分配和调度，不仅影响了项目的进度，也降低了团队的工作效率和士气。

最后，缺乏有效的项目团队沟通及监督制度，项目管理者与需求部门之间没有进行及时的沟通交流，导致需求变更时无法及时通知到开发人员。这会导致各环节工作脱节，出现返工现象，严重影响项目进度。此外，项目管理者还不能对项目整体实施情况进行有效的跟进。超过 60%的项目只能依靠一周一次的例会和周报来了解进度，很难及时发现执行偏离的情况，若监控不到位，等发现项目进度滞后时已难以挽回。

1.3 研究意义

随着全球数字化转型进程不断深入，传统企业正由“信息化基础建设”迈向“数字化价值重构”，业务上云已成为传统企业的必选路径。传统企业将线下零散的业务流程、数据资产上云后，打破了时间和空间限制，提高了资源利用率，进一步提高了全面信息化管理的规范性和自动化程度。其中，业务应用运行平台是将企业的业务活动和数字技术融合在一起的基础媒介，在很大程度上影响着企业数字化进程的快慢以及实施效果的好坏，是传统型企业业务线上化的重要基础性平台。本文通过对复杂软

件项目进度及业务适配协同的研究，寻求解决的方法和途径，在形成项目进度管理体系的同时，还能拓展软件项目进度管理理论在企业数字化转型平台这一垂直领域的应用，为同类项目提供理论与实践参考。

从应用视角来看，本研究具备广泛的应用价值。一方面，对于正在推进数字化转型的企业而言，业务应用运行平台是转型的核心支撑，若平台研发因进度管理不当出现延期交付、功能与需求脱节等问题，不仅会阻碍转型进程，还可能增加成本投入、错失市场竞争机遇，而本研究提出的进度管理优化思路，能为这类企业规避平台建设中的进度风险提供方向；另一方面，对于承接同类“企业数字化转型支撑平台”研发的软件服务企业或内部研发团队，本研究提炼的进度管理框架具有可复制性与可借鉴性，可从任务拆解、资源分配、风险预警、变更管控等方面提供参考，帮助更多企业提升复杂软件项目的进度管控效率，减少因进度问题导致的损失，最终推动整个行业数字化转型项目的高效落地。

本文以 C 公司负责的 Z 公司业务应用运行平台开发项目为例子，旨在解决其当前面临的进度管理难题。该项目建设中包含需求调研、系统设计、系统开发、系统测试、系统实施等阶段，并且还需要结合企业实际业务，若项目进度管理不当，极易出现延期交付，功能与需求脱节等风险。这些风险会使得企业信息化建设受阻，还可能增加项目成本投入。因此本文将在充分考虑项目当前进度管理中存在的不足的基础上，并借鉴其他相关项目优秀实践以及相应的理论成果来提出适合该项目实际的切实可行的进度管理对策，具体包括细化任务拆解标准、完善资源分配机制、建立需求变更管控流程、搭建实时进度监控体系等，助力 C 公司有效规避进度风险，保障项目按预期交付，并为 C 公司后续承接同类大型软件项目提供通用性的进度管控方法，提高公司参与政府信息化建设项目的核心竞争力。

1.4 研究内容

本文以 C 公司项目为例，分析该项目进度管理中存在的问题及成因。通过对该项目的深入研究发现项目执行过程中存在任务分解粗糙、评估体系存在缺陷、资源分配以及沟通机制不合理等原因导致无法正常交付等问题，结合项目进度管理理论与方法，优化方案并进行评估。

本文将用六个章节对所研究的内容进行详细分析和描述，具体为：

第一章：绪论。阐述论文研究背景、研究目标以及研究意义。分析研究该目标的方法以及技术路线。

第二章：相关理论与文献综述。简要描述项目进度管理的相关理论，对 WBS 分解，关键路径、关键链等项目进度管理中主要概念进行阐述。介绍国内外软件项目管理研究现状和进度管理研究现状。

第三章：项目进度管理现状及问题识别。通过介绍 C 公司的基本情况，项目实施管理现状，对该公司现存的问题做出简要介绍。同时为精准定位问题，采用问卷调查法对 C 公司项目核心成员开展定向调研。在此基础上通过鱼骨图（因果图）分析法对已识别的进度管理问题进行逐层拆解，追溯问题产生的根本原因，为后续提出针对性的优化策略奠定分析基础。

第四章：项目进度管理优化方案。本章运用多种项目管理工具，结合 C 公司业务应用运行平台项目的实际情况，系统性设计具有针对性的进度优化方案。具体而言，方案将计划评审技术（PERT）、关键路径法、甘特图、关键链法及缓冲区设计等工具与方法融入业务应用运行平台项目的进度计划制定全过程，为项目高效推进提供技术支撑。

第五章：项目进度保障、监控措施与效果评价。本章探讨如何对项目进度进行保障，如建立完善的沟通交流机制，建立需求变更机制以及建立绩效激励机制等。同时通过缓冲区监控三色图来动态监控进度计划进展情况与缓冲区的消耗情况的关系。

第六章：结论。总结业务应用运行平台项目进度管理研究过程，得出本研究所提出的进度管理方案和方案保障措施能够优化 C 公司项目进度的结论，同时提出本项目管理和执行的过程中可以优化内容，为 C 公司提升进度管理水平，提高项目交付率指明方向。

1.5 研究方法与技术路线

1.5.1 研究方法

本文研究采取理论研究与实际项目情况相结合、定性研究与定量研究相结合的综合研究方法，结合科学的项目进度管理理论，对 C 公司业务应用运行平台建设过程中的进度管理相关问题进行分析和探索，并提出优化方案。

（1）文献研究法

文献研究法是一种通过系统搜集、梳理、分析与研究主题相关的现有文献，进而为当前研究提供理论支撑、逻辑基础或论证依据的研究方法。文献研究法所要解决的主要是如何在浩如烟海的文献资料中选取适用于课题的资料，并对这些资料做出恰当的分析，归纳出有关问题[3]。本文首先搜集和整理项目进度管理相关的文献资料，了解和掌握目前项目进度管理研究的现状及不足等。然后在对项目进度管理相关文献的学习和研究基础之上，结合 C 公司的实际情况，查找出 C 公司项目进度管理面临的问题，进而总结出对业务应用运行平台进行进度优化的具体方案。

（2）调查研究法

调查研究法是一种重要的科学研究方法，它是一种通过有目的、有计划地收集相关资料和信息，以充分了解某种现象或问题的方法。他的优点是在真实环境中调查要研究的问题，而不是在实验室等人为条件下做研究[4]。本文在对 C 公司及其项目基本环境具备一定了解的基础之上，通过问卷调查的方式与 C 公司项目中不同角色、不同职级的人员进行充分沟通，获取项目进度管理相关的信息和数据。然后对获取的信息和数据进行整理和汇总，作为发现项目问题、查找问题原因、并制定项目进度管理优化方案的事实依据。

（3）案例分析法

案例分析法是兼具实践性与客观性的实证研究方法，能为研究者提供真实素材，助力其深入了解实际情况并提炼有价值的结论。该方法可针对具体事件深度剖析，帮助梳理复杂现象、开拓研究视角，其分析结果对案例具有较强指导意义。在运用该方法时，需秉持全面研究视角，并确保案例信息记录的完整准确[5]。本文以案例分析法为核心，基于问卷调查的一手信息，结合鱼骨图这一因果分析工具，对 C 公司业务应用运行平台项目展开梳理与分析。一方面识别出该公司项目进度管理制度存在的问题及成因，另一方面可以明确项目进度延误的根本原因，为后续制定改进方案提供坚实依据。

1.5.2 技术路线

根据上文中的研究内容以及研究方法，制定本研究的技术路线如图 1.1 所示：

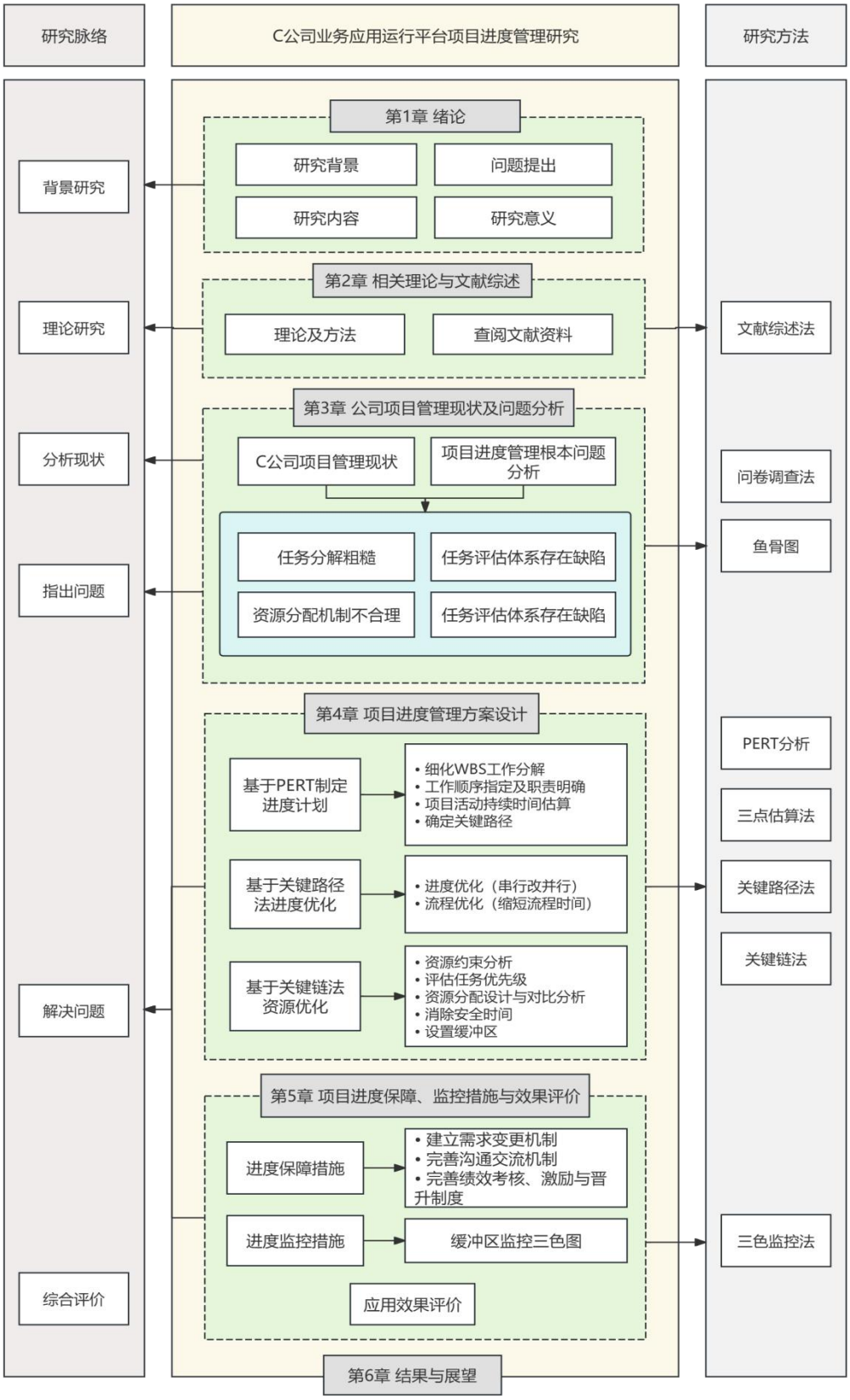

研究脉络
C公司业务应用运行平台项目进度管理研究
研究方法
第1章 绪论
研究背景
问题提出
研究内容
研究意义
背景研究
第2章 相关理论与文献综述
理论及方法
查阅文献资料
理论研究
文献综述法
第3章 公司项目管理现状及问题分析
C公司项目管理现状
项目进度管理根本问题分析
分析现状
问卷调查法
任务分解粗糙
任务评估体系存在缺陷
资源分配机制不合理
任务评估体系存在缺陷
指出问题
鱼骨图
第4章 项目进度管理方案设计
基于PERT制定进度计划
• 细化WBS工作分解
• 工作顺序指定及职责明确
• 项目活动持续时间估算
• 确定关键路径
PERT分析
三点估算法
基于关键路径法进度优化
• 进度优化（串行改并行）
• 流程优化（缩短流程时间）
关键路径法
基于关键链法资源优化
• 资源约束分析
• 评估任务优先级
• 资源分配设计与对比分析
• 消除安全时间
• 设置缓冲区
关键链法
解决问题
第5章 项目进度保障、监控措施与效果评价
进度保障措施
• 建立需求变更机制
• 完善沟通交流机制
• 完善绩效考核、激励与晋升制度
进度监控措施
缓冲区监控三色图
三色监控法
综合评价
应用效果评价
第6章 结果与展望

图 1.1 技术路线图

2 相关理论基础与文献综述

2.1 相关理论

2.1.1 项目管理概述

如今，公众对于项目管理的内涵及其知识体系已形成较为一致的看法，典型代表包括国际项目管理协会（IPMA）和美国项目管理协会（PMI）的研究成果。PMBOK第六版中也对项目管理做出了定义，它认为：“项目管理是运用各种相关的知识、技能、方法与工具，为满足或超越项目有关各方对项目的要求与期望，所开展的各种计划、组织、领导和控制等方面的活动[6]。”项目管理过程包括了启动、规划、执行、监控、收尾五个过程组，这些过程组在项目管理中并非独立存在的阶段，而是紧密相关，相互重叠于整个项目管理活动中。通过有序地执行这些过程，项目管理能够确保项目从启动到收尾的每一个环节都得到妥善处理，从而提高项目成功的可能性。此外，PMBOK 中还定义了十大知识领域：整合、范围、时间、成本、质量、人力资源、沟通、风险、采购和项目利益相关者。项目管理知识领域与过程组对应关系如表 2.1。

表 2.1 项目管理知识领域与过程组对应关系表

	过程组				
知识领域	启动过程组	计划过程组	执行过程组	监控过程组	收尾过程组
整合	√	√	√	√	√
范围		√		√	
时间		√		√	
成本		√		√	
质量		√	√	√	
人力资源		√	√		
沟通		√	√	√	
风险		√		√	
采购		√	√	√	√
利益相关者	√	√	√	√	

2.1.2 项目进度管理概述

项目进度管理是指在项目实施过程中，对各阶段的进展程度和项目最终完成的期限所进行的管理[7]。其目的是保证项目能在满足其时间约束的前提下实现其总体目标

[8]。一个项目进度控制的关键点在于三个方面，分别是软件项目开展的进度、成本、质量，这三个指标综合起来才能够反映出项目管理的整体质量[9]。这三个指标的目标既是对立的，又是统一的。项目管理者需要解决好三者的矛盾，既要进度快，又要投资省、质量好[10]。软件项目管理实行进度控制能够从综合角度对实操过程进行控制，及时发现并改正错误，是非常有效的保证软件质量的方法，减少了不必要的资源浪费，合理调整，确保设计符合要求，同时，提高了项目的运行效率[11]。

在项目进度计划制定阶段，项目管理者需要制定出一个切实可行的、科学的进度计划，然后再按照计划逐步实施。这些信息资料包括项目背景、项目实施条件、项目实施单位，人员数量和技术水平、项目实施各个阶段的定额规定等[8]。项目管理者需全面权衡项目范畴、资源约束、活动间的依赖关系，借助多种科学工具与算法开展进度规划。其中，工作分解结构（WBS）是项目计划编制的基础方法，通过逐层拆解复杂业务模块，将项目工作细化为可执行、可管控的最小工作单元，明确任务边界与工作内容，为工期估算、任务排序、资源分配提供清晰框架。关键路径法（CPM）依托网络逻辑图梳理活动依赖关系，计算各工序的总时差与自由时差，识别制约项目总工期的关键活动与关键路径，为工期压缩、任务优化提供判断依据。计划评审技术（PERT）属于概率性工期估算方法，通过乐观时间、最可能时间、悲观时间三点估算方式，降低人为主观估算带来的偏差，使工期测算更加科学、客观。关键链法（CCPM）则在关键路径基础上综合考虑人力、设备等资源约束，消除资源冲突，并整合分散安全时间设置集中缓冲区，有效抵御项目执行过程中的不确定风险[12]。

在项目进度计划控制阶段，项目管理者需持续跟踪监控实施进度，采集实际数据并与计划进行对比分析，识别偏差根源及解决方案，制定调整措施并修订原计划后执行。该过程通过“监测-分析-纠偏-再监测”的闭环机制循环推进，直至项目竣工交付[13-14]。在监控过程中，可结合缓冲监控、三色预警、绩效考核等手段动态把控项目执行状态，及时修正工期偏差，保障优化后的进度方案稳定落地。

2.2 文献综述

2.2.1 软件项目管理研究现状

在 20 世纪 60 年代，计算机硬件性能弱，软件开发规模小、复杂度低，在这种情况下，快速编写代码以及对程序进行调试修改是主流做法。所以当时比较流行的一种软件开发模式叫做“编码与修补”(Code and Fix) 。当然，并不是所有的软件项目都

是小规模、低复杂度的，当“编码与修补”模式应用到相对大型的软件项目上，问题便逐渐显现。除了极少数的成功案例之外，大部分的大规模软件开发都是失败的。这些失败案例暴露出该模式在需求变更管理、系统架构设计、团队协作效率等方面的局限性，为了解决大型软件项目的软件危机，北大西洋公约组织（NATO）提出了“软件工程”这一术语，以改进软件开发设计过程[15]。这一时期，软件项目管理开始受到重视。

20 世纪 70 年代～90 年代，一个重要的变化是软件作为独立产品出现了，不再依托于硬件设备。尤其在 20 世纪 80 年代，个人电脑的出现使得软件的复杂度和规模都显著增加，功能需求也在不断变化，对软件项目管理的要求也更高，值得一提的是 Brooks 在《没有银弹》一文中总结了软件开发的四大本质难题，即复杂性、一致性、可变性以及不可见性[16]被业界广泛认同。这一阶段有两种开发方法获得了重视，其一是形式化方法[17]，是一种基于数学的技术，用于确保计算机系统满足其需求。通过使用数学模型和逻辑推理，形式化方法能够精确描述系统的行为、属性和规范，从而有助于发现设计和实现中的错误，提高软件的可靠性和安全性。其二是 Royce 提出的瀑布生命模型[18]。其核心内容是从先设计再开发、设计文档化、“Build it twice”（二次构建）、规划和监控测试以及引入客户这样一个软件开发过程的改进路径。同时增加过程元素会提升开发成功率，但也会增加时间和人力成本，因此需权衡取舍。

20 世纪 90 年代中期至今，软件的规模继续扩大，随着互联网和移动互联网的普及，用户数量也出现指数级增长，大量的软件过程在这个阶段涌现出来。其中，具有迭代式开发特征的一系列开发方法逐渐得到软件行业的广泛应用，迭代式开发将产品开发周期划分为几个小周期，即多个迭代（Iteration），每次迭代均包括需求分析、设计、编程和测试的完整过程[19]。这种软件开发过程得到了业界的空前重视，进而演化出了一种新的体系，在 2001 年 2 月，17 位世界轻量级方法学家共同发布敏捷宣言，宣告敏捷开发方法正式诞生[20]。

敏捷体系按“价值观、原则、方法、实践、工具”这 5 个层次进行解构，是行业内一种较为公认的划分方式，其中最受关注的敏捷项目管理方法包括 Scrum[21]、XP [22]（极限编程）和看板管理[23]，具体如下：Scrum 由 Jeff Sutherland 和 Ken Schwartz 提出，是一种迭代式增量软件开发过程，也是帮助团队更好协作的框架，其基于经验过程控制理论，具有透明、检视和适应三大支柱，通过透明的工件和过程，团队可实时检视工作进展与成果，并根据实际情况及时调整计划和做法，以适应不断变化的需求

和环境；XP 是一种软件工程方法学，同样是敏捷软件开发中富有成效的方法之一，它强调通过快速迭代、客户参与、团队合作等方式提高软件质量和对需求变化的响应能力，但由于 XP 过程框架中实践较少，逐渐被 Scrum 取代；看板管理最早由 Taiichi Ohno 为日本丰田汽车公司开发，初衷是在生产的每个阶段以最佳方式控制和管理工作与库存，后来 David J. Anderson 等将这一概念应用于 IT 及软件开发领域，通过可视化方式帮助团队管理工作流程、提升效率，并识别和解决工作中的瓶颈。

2019 年 Patrick Debois 在“DevOpsDays”会议中提出“DevOps”的概念[24-25]。从起源看，DevOps 是敏捷开发的延续，它更关注文化与工具的融合，为的是打破开发团队和运维团队之间的隔阂，推动二者紧密协作，以期快速响应变化和交付价值。刘博涵等人提出随着 DevOps 的逐渐成熟,员工的计划外工作将减少，这有利于软件过程的可管理性和易预测性.这样的组织,员工将有更高的敬业度和满意度,这有利于组织的可持续性发展[26]。

2.2.2 软件进度管理研究现状

甘特图技术是甘特（Gantt）最早提出并应用于项目进度管理[27]。它以日历的形式将项目的主要内容罗列出来，能够较为清晰地反映出项目活动各时间段的进度信息，为今后的项目管理工作树立了格式标准[28]。与其他技术相比，甘特图技术更为清晰直观、简单便捷、易于绘制。但这一技术难以表达各项目活动间的逻辑关系，同时进行定量分析及定性计算，反映不出任务拖延或资源调配的问题。

美国杜邦公司于 1956 年提出了关键路径法（CriticalPath Method，CPM）[27]，其工作原理为数字分析。在实际处理过程中，先确定项目中各项活动的先后顺序以及估算项目总工期。它基于网络图的构建，通过分析各个任务之间的逻辑依赖关系，识别出决定项目最短完成时间的一系列关键任务所组成的路径，也就是关键路径，这些任务的任何延误都会直接导致整个项目工期的延长。CPM 一般采用单一时间估算方法，侧重于计算活动的最早开始时间、最早结束时间、最迟开始时间、最迟结束时间等时间参数，通过这些时间参数的准确计算，来清晰界定每个活动在时间维度上的合理安排范围[29]。这种分析更适用于活动工期相对确定、工作环境较为稳定、有过往类似项目经验可参考的项目。

计划评审技术（PERT）主要是一种用于项目管理的工具和技术，旨在帮助规划、监控和控制复杂项目的进度和资源分配[30]。和 CPM 一样，都是根据网络计划图为基

础来呈现项目中各个活动之间的先后顺序以及相互依赖关系，以及找出项目中心关键路径展开分析。PERT 和 CPM 的区别在于其工期估算方式不同，PERT 利用的是活动持续时间的三种时间（乐观时间、最可能时间和悲观时间）加权值[31]来推算项目在某一特定时间内完成的概率。这种分析方法更适用于各节点很难准确知晓具体时间的项目，需要借助 PERT 的概率分析特点来应对工期的不确定性[32]。

关键链管理（CCPM）提出于 1997 年[33]，其核心思想是通过识别项目中的约束资源（瓶颈），并围绕这些资源来安排项目进度，而非传统的任务依赖关系[34]。关键链法就是在项目的执行过程中，将每个节点分散的安全时间集中提取出来，放置在关键链的末尾或者关键任务衔接处等位置形成项目缓冲（Project Buffer）[35]。关键链技术是继计划评审技术（PERT）和关键路径法（CPM）之后出现的一种全新的项目管理理论和方法，它在许多方面与传统的各种项目管理理论相比都具有较大的优越性。大量的研究表明，CCPM 在提高工期完成率、缩短工期方面效果显著[36]。黄振宇、张文雅基于关键链的敏捷软件开发项目进度管理提出了几点改进意见[37]。其中提到要将分散管理模式改为集中管理模式，做到管理统一。这样有利于项目成本的节约，项目资源的合理调配，项目风险的有效控制，保证项目的正常运行并顺利完成。在管理过程中要合理调配项目资源并积极解决约束因素。最后在管理过程中要重视人的作用，激发项目人员的积极性和创造性。

2.2.3 文献评述

国内外学者已经从不同角度、不同情境下对软件项目的进度管理进行了大量研究工作，并取得了一定成果，但在实际工作中还存在着诸多亟待解决的问题：一方面，软件项目要与企业的业务紧密结合在一起，而企业业务又是复杂的、变化的，所以如何准确地确定影响软件项目进度的各种因素依然是一个难题；另一方面，以上研究大多针对一种管理技术进行改进，并不适用于同时具有确定性和不确定性的混合型软件项目的管理。此外，通过文献检索可见，先进的管理理念、工具方法及信息技术多源于建筑工程管理领域的创新，在软件项目开发领域的应用与拓展则相对滞后。

因此本文基于最新的研究成果，从项目进度管理发展动向、软件项目的特点以及 C 公司软件实施开发进度管理情况出发，以 C 公司业务应用运行平台研发项目为例进行分析，提出进度优化方案及资源优化方案，并探索在实际工作中如何灵活地进行进

度管理。更好地应对未来不确定性和动态变化，也为同类软件项目管理实践提供有益借鉴及启发。

3 C 公司项目进度管理现状及问题识别

本章节结合 C 公司的实际情况，通过问卷调查的方式对 C 公司项目中不同角色、不同职级的人员进行采样收集，获取项目进度管理相关的信息和数据，并借助鱼骨图工具对其问题产生的原因进行深入剖析，为 C 公司业务应用运行平台项目的进度管理研究找准问题切入点，确保后续章节研究的有效性和实用性。

3.1 业务应用运行平台项目简介

3.1.1 C 公司概况

C 公司成立于 2006 年，是具有大型媒体集团背景的高新技术企业，以广电行业的 IT 开发和系统集成为主攻方向，在业内有着举足轻重的地位。主要业务涉及电视、广播、有线网络、新闻出版、互联网、融合媒体等行业，为国内多个省级以上主流级广播电视单位提供核心 IT 解决方案，服务能力和技术水平得到业内高度认可。

C 公司专注于为广电行业客户打造全流程的生产与运营支撑体系，核心围绕业务数字化、信息化、智能化转型需求，提供一体化解决方案及专业化产品服务。具体业务涵盖媒体技术规划咨询、全媒体融合生产、IP 流总控调度、生产管理与媒体运营、媒体大数据智慧应用、监控运维平台搭建等多元化服务内容，同时布局移动化应用、云计算及大数据技术领域的平台开发与集成运维服务，形成全方位的技术与服务矩阵，助力客户实现业务升级与效率提升。公司自开展业务以来一直重视人才培养，目前有员工 500 多人，90％以上具有大学本科学历，拥有一批精通软件工程理论与实践、计算机软件开发的复合型高级人才和具有多年电视媒体业务经验的项目咨询专家。公司依靠雄厚的技术实力和人才优势，努力营造“精诚协作、拼搏进取、完美服务”的公司文化氛围。

C 公司基于自身商业模式及产品特点设立了以总经理为最高管理人员的组织架构，基于自身承接的项目特点设立以项目开发部和产品研发部为核心，联动咨询规划部、技术服务部协同开展项目全流程工作的项目架构。销售部负责市场调研、客户关系维护与产品销售工作，达成产品和服务的销售目标。行政部门负责公司行政、财务、人力资源等后勤类工作的规范管理。C 公司的组织架构如图 3.1 所示。

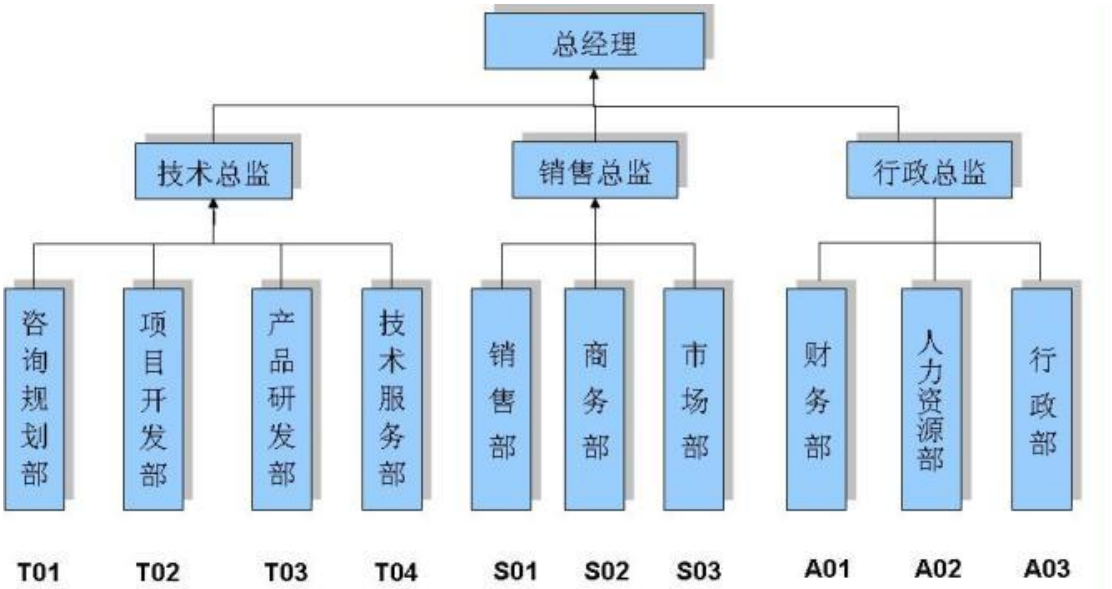


图 3.1 C 公司组织架构图

3.1.2 业务应用运行平台项目基本情况介绍

（1）项目背景

随着数字经济发展以及智慧广电建设进程不断加快，Z 公司作为广电领域相关机构，对 IT 服务的一体化、移动化的诉求日益迫切。伴随其业务规模持续扩大，服务场景向融媒体延伸，原各系统相对割裂的 OA、项目管理等系统已经不能满足当前的发展需求。各系统之间数据不互通造成“信息孤岛”现象，各部门间协作困难，在移动办公环境下无法及时掌握相关业务数据以及重要资源，给客户带来技术问题处理慢、解决问题时间长等问题，客户满意率比同业标杆企业差了 18%，由系统碎片化造成的经营困扰愈发突出。为顺应行业转型趋势、破解自身发展瓶颈，同时巩固市场地位，Z 公司决定启动一体化业务应用运行平台建设项目，并将该项目委托给深耕广电领域 IT 服务的 C 公司承接实施，业务应用运行平台建设项目由此正式启动。

（2）项目介绍

C 公司业务应用运行平台项目是一个服务于广电行业业务的综合性移动应用支撑系统建设项目，将“聚合业务资源、助力高效运营”作为其主要目标，致力于解决 C 公司原有多分散业务系统所带来的孤岛效应，实现集“开发-运营-服务”的移动业务

综合支撑体系。本次项目启动于 2024 年 5 月 20 日，整体项目的计划工期将近 9 个月，在 2025 年 2 月 28 日内完成资产的交付，整体项目投入预算共 800 万元左右，从需求调研至系统的实施及上线运行到最后的移交运维。

本项目核心目标聚焦三方面：构建统一移动业务入口以实现内外部服务移动端全覆盖、建立标准化数据交互机制以消除信息孤岛、形成规范化开发运维流程以降低成本并提升平台扩展性与稳定性。围绕这些目标，结合广电行业特性及 C 公司服务经验，平台明确了四大核心功能模块：

移动办公模块：支持移动端任务分配、流程审批、消息通知、日程管理、文档协同编辑等操作，员工可随时随地处理工作事务，无需依赖 PC 端，大幅提升办公灵活性与效率。

业务管理模块：将广电企业客户服务全生命周期的功能进行集成，如立项、过程控制、合同起草及管理、计费管理、客户报修、工作订单处理等等，在不同阶段信息可以共享，并能对客户需求到最终解决形成一个完整的管理链条。

数据可视化模块：以报表或图表的形式展示业务经营状况、工程实施状态、客户满意程度、资源占用率等关键信息，并能够进行交叉分析以及下钻操作，辅助领导层掌握实际经营状况并及时做出相应决策。

运维支撑模块：具备故障上报、工单处理、系统实时监控、性能预警、日志分析、定期巡检等功能，可及时发现并解决平台及关联业务系统的运行问题，保障系统 7×24 小时稳定可靠运行，降低运维风险与成本。

3.1.3 组织结构及团队

项目建设初期应建立项目管理机制，运用项目管理模式及组织机构完成对项目的规范化建设，并确保系统的正常运行和最终的项目交付成果。结合目前常见的项目组织模式，在本项目中确定使用扁平化项目型组织架构：以项目负责人为核心统筹全流程工作，项目组核心管理层由公司各相关职能部门主管构成；同时基于实际需求细化分工，下设产品、技术、测试、运维四个专项团队，项目的组织架构如图 3.2 所示。

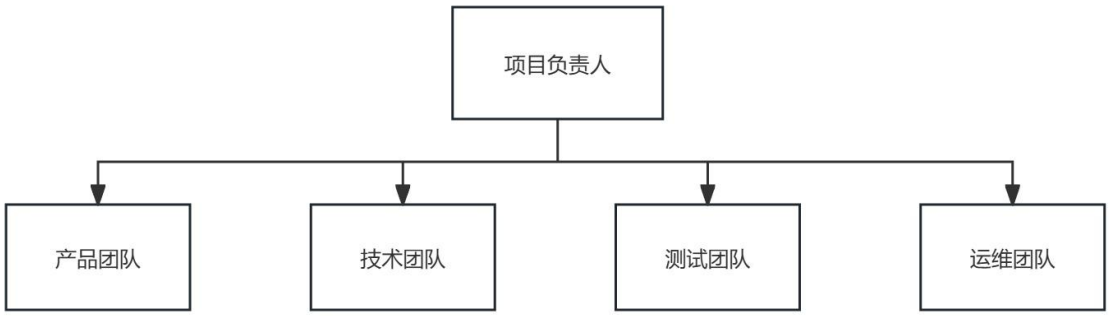


图 3.2 项目组织架构图

项目负责人作为项目总协调者，负责制定项目整体目标与进度计划，统筹跨团队资源协调、需求优先级判定、风险管控及项目交付质量的最终把控，向公司技术/业务部门同步项目进展。

产品团队承担项目的需求管理工作。包括对接公司内部业务部门及外部客户需求，输出需求文档、产品原型与功能规划；跟进需求落地进度，协调各团队对齐功能实现方向，同时负责产品上线后的迭代规划。

技术团队聚焦平台的技术实现。基于产品需求完成系统架构设计、代码开发、接口联调等工作；负责核心功能模块（如移动办公、数据可视化）的技术选型与功能开发，保障平台的性能、安全性与可扩展性。

测试团队负责全流程质量管控。制定测试计划与用例，开展功能测试、兼容性测试、性能测试等。跟进 Bug 修复进度，输出测试报告，确保平台功能符合需求标准，降低上线后故障风险。

运维团队覆盖项目的部署与运维环节。负责平台的环境搭建、版本部署，上线后承担系统监控、服务器维护、故障应急响应等工作，保障平台稳定运行，同时协助处理用户的运维类问题反馈。

3.2 业务应用运行平台进度管理现状

3.2.1 项目工作分解结构

在项目开始启动之初，为了更好的对项目进行管理和控制，项目管理部门制定了本次项目的工作分解结构。优化前项目分解结构如图 3.3 所示，整个项目被划分为了四个阶段，但并未进行更详细的任务分解，部分工作内容存在缺项漏项。

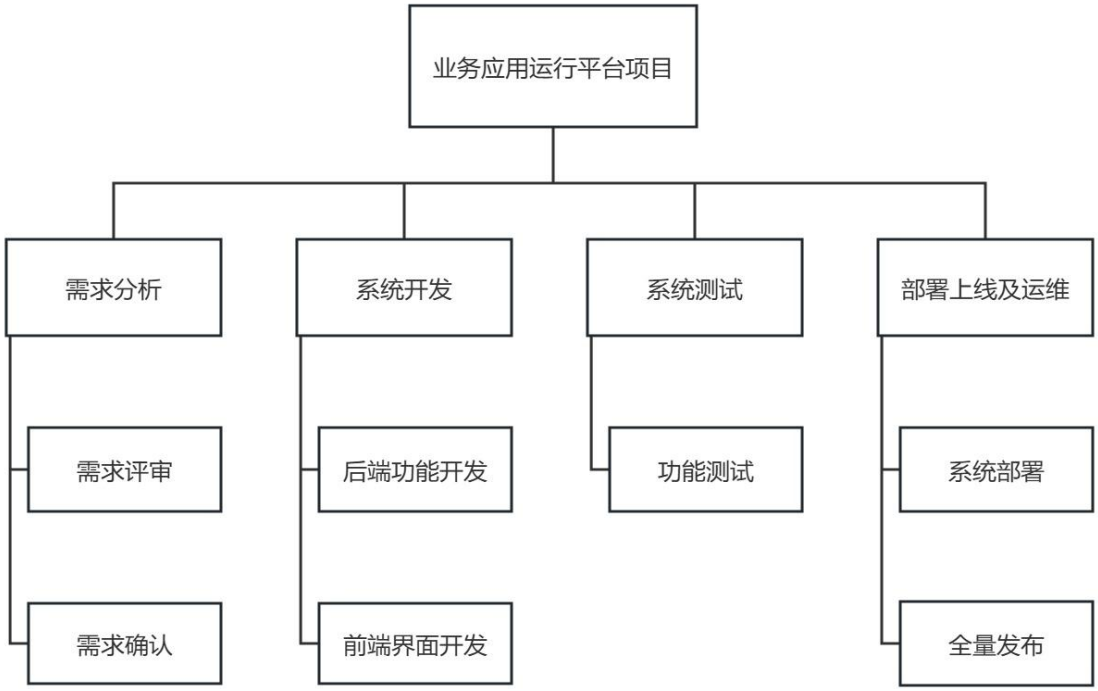


图 3.3 优化前项目分解结构

3.2.2 项目进度计划

C 公司计划在 9 个月内完成业务应用运行平台的建设工作。项目管理部门与项目参与人员共同协商后，制定了项目里程碑计划，里程碑计划如表 3.1 所示。项目中的每个里程碑节点的完成时间更多依赖于项目人员的主观判断，而非基于 WBS 工作分解的量化任务拆解、历史项目数据沉淀的客观测算及关键路径法的科学推演。

表 3.1 业务应用运行平台项目里程碑计划

序号	里程碑节点	初步时间预估	工作内容
1	项目启动	2024/05/20	完成项目立项、团队组建及启动会，明确项目目标与整体计划
2	需求分析	2024/06/14	完成内外部需求调研、需求文档评审与确认，输出最终需求规格说明书
3	架构设计	2024/07/19	完成系统架构、功能模块及数据库设计，输出设计文档并通过评审
4	系统开发	2024/11/15	完成平台四大核心模块的代码开发、接口联调及内部功能初验
5	系统测试	2024/12/31	完成功能、性能、兼容性测试，修复所有高优先级 Bug 并输出测试报告
6	上线部署	2025/01/24	完成生产环境测试并部署系统，完成数据迁移与备份，确保系统稳定运行
7	系统验收	2025/02/21	组织客户侧验收测试，完成用户培训与操作手册交付，确认项目达标交付

3.2.3 项目监控措施

（1）建立分级汇报机制

项目采用周报与月报结合的信息采集与汇总模式。项目各团队成员每周、每月需梳理个人工作进度，向上一级负责人提交阶段性工作成果与进度说明。信息经逐级汇总后形成项目整体进度报告，构建自下而上的闭环汇报体系，确保项目管理层实时掌握全流程进展。同时，项目执行期间每月至少组织 2 次跨团队沟通会议，针对进度偏差、资源协调等问题开展集中研讨，快速制定调整方案。

（2）强化里程碑与关键检查点管控

以表 3.1 中的里程碑计划为核心，建立关键点管控机制，明确各节点交付物标准。需求分析阶段需输出经评审通过的需求规格说明书，系统设计阶段需完成架构设计文档及数据库设计方案并通过验收，系统开发阶段需实现核心功能模块并完成内部联调等。如果在达到计划节点时仍未完成相关工作，则要求及时上报，根据项目工作情况进行调整。

3.2.4 项目进度情况

C 公司业务应用运行平台项目于 2024 年 5 月 20 日开始实施，除了项目启动节点按计划如期完成外，在后续各节点均未按计划推进，需求分析节点原定于 2024 年 6 月 14 日完成，实际因相关工作延迟，该项工作直至 2024 年 6 月 18 日才完成。而系统设计、系统开发、系统测试等后续关键节点均未完成，同时根据前期的需求分析阶段延迟的情况以及项目的实施情况，可以判断出所有的节点都会存在延迟的问题。整体来看，项目的进展情况远远落后于原定的计划，若不及时干预，项目整体将面临延期交付的重大风险。为保障项目按预期达成目标，就需要对项目进度管理工作提出有效的改进措施，对已确定的进度控制措施、资源供应方案、风险对策等内容进行完善。

表 3.2 业务应用运行平台项目实际进度表

序号	里程碑节点	计划完成时间	实际完成时间	项目完成情况	项目进度情况
1	项目启动	2024/05/20	2024/5/20	完成	正常
2	需求分析	2024/06/14	2024/6/18	完成	延期
3	架构设计	2024/07/19	-	未完成	预计延期
4	系统开发	2024/11/15	-	未完成	预计延期
5	系统测试	2024/12/31	-	未完成	预计延期
6	上线部署	2025/01/24	-	未完成	预计延期
7	系统验收	2025/02/21	-	未完成	预计延期

3.3 业务应用运行平台进度管理问题及原因

鉴于 C 公司以往的项目均存在一定程度的延期现象以及目前业务应用运行平台项目的实施情况严重滞后的问题，为了更好地了解此项目在实施过程中存在的进度方面的主要问题及原因，本人对该项目建设过程中的相关人员进行了调查研究工作，以便有针对性地进行改进措施的研究。

3.3.1 问卷发放人员、问题确定

（1）问卷发放人员确定

本次调查对象主要针对 C 公司业务应用运行平台项目的核心参与人员，包括项目管理、产品设计、技术开发、测试验证、运维支撑等岗位，并兼顾不同职级及工作年限员工的占比情况，以确保样本的代表性与数据有效性。岗位级别方面，各岗位均涵盖初级、中级、高级三个层级，其中技术开发作为项目关键环节，该岗位三个级别的员工均参与问卷填写，从而得到不同水平的专业意见。司龄分布方面，13 人司龄不足 3 年、28 人司龄为 3-6 年、6 人司龄为 6-10 年、3 人司龄超过 10 年，该分布与公司整体人员结构基本一致，可以较为真实地反映出公司的现状情况。此次调查共发放问卷 50 份。填表时间为期 7 天，共收回有效问卷 46 份，有效回收率达 92%。样本不仅覆盖项目关键岗位、各职级及不同司龄层次，结构均衡性突出，且优先面向业务应用运行平台项目直接参与人员发放，在聚焦 C 公司项目进度管理现状分析的同时，优先保障了本项目关键问题识别，样本响应质量与数据可靠性均得到充分保障，能够满足本次特定项目进度管理问题识别的研究需求。

表 3.3 问卷发放对象统计

岗位	项目管理	产品设计	技术开发	测试验证	运维支撑	总计
人数	5	8	18	7	8	46

（2）问卷问题确定

问卷题项围绕 C 公司业务应用运行平台项目的进度管理现状与影响因素构建，整体分为三大模块。第一模块为基本信息，包含项目团队、项目角色、参与阶段 3 个题项，用于明确样本特征；第二模块为项目进度管理问题，围绕任务分解、任务评估、资源分配、沟通监控四大关键维度设计 8 个题项，聚焦各维度下具体执行问题；第三模块为开放性建议，包含 2 道开放题，收集其他关键影响因素及优化建议。所有题项表述采用客观中立语言，采用李克特五点记分法（1=完全不符合，5=完全符合）或多选、开放作答形式，确保被调查者能够准确理解并提供真实反馈，具体访谈问题详见附录 A《C 公司项目实施进度响应因素问卷调查表》。

3.3.2 问卷调查结果整理与分析

根据问卷调查表的结果进行汇总，项目进度影响的因素调查结果如表 3.4 所示：

表 3.4《C 公司项目实施进度响应因素问卷调查表》部分问题统计结果

序号	核心原因	问题描述	完全不符合	较不符合	一般	较符合	完全符合
1	任务分解粗糙	项目工作分解未覆盖关键细节，存在遗漏或模糊的任务项	2	5	13	17	9
2	任务评估缺陷	项目任务的时间预估依赖主观经验，缺乏历史数据或行业标准支撑	1	4	8	23	10
3	资源分配不合理	项目人力配置与任务复杂度、工作量不匹配	3	9	12	15	7
4	资源分配不合理	关键技术资源集中在单一环节，导致部分阶段资源闲置、部分阶段资源短缺	2	7	12	17	8
5	资源分配不合理	硬件、软件等技术资源未提前预置，出现资源缺口时延误工作推进	5	7	12	14	8
6	沟通机制不健全	跨团队协作时缺乏高效对接机制，需求变更、问题反馈不及时	2	6	8	22	8
7	监控体系缺位	未建立明确的进度偏差预警机制，出现问题后被动应对	2	6	8	21	9
8	衍生影响因素	用户需求频繁变更，开发任务反复修改	7	6	12	14	7

从调查结果可以推断 C 公司项目实施进度响应因素主要有以下四点。制作鱼骨图如图 3.4 所示。

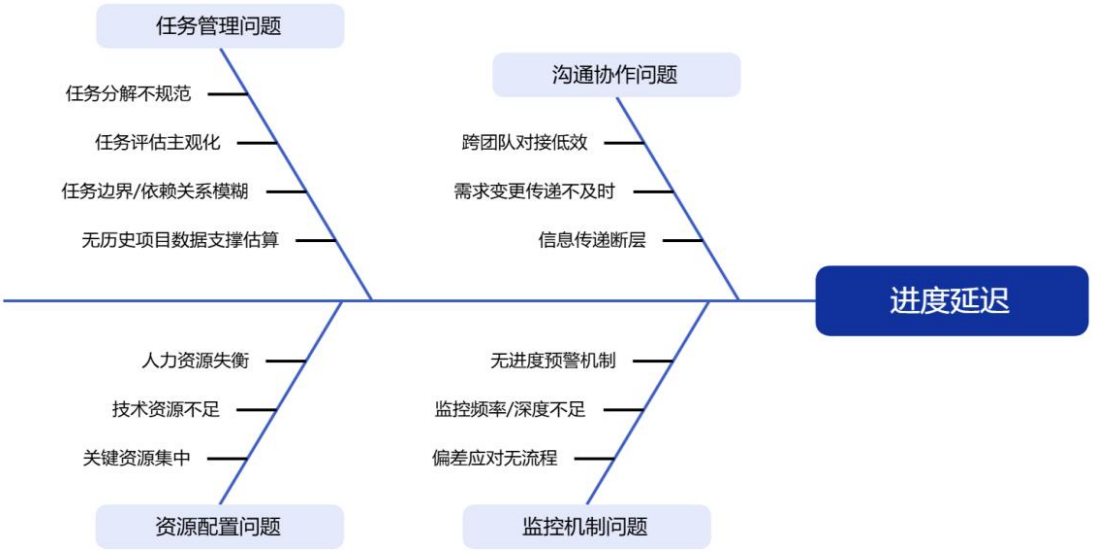


图 3.4 C 公司进度管理延迟问题鱼骨图

（1）任务分解过于粗糙

调查结果显示，接近六成参与人员反馈项目 WBS 工作分解存在明显短板，主要表现为工作项遗漏、任务边界模糊。由于分解过程未全面覆盖项目全流程工作，部分任务的前置依赖与后置衔接环节未纳入原始计划，导致执行阶段频繁出现补漏式工作调整。例如部分测试子任务未提前拆解，待开发环节完成后才临时补充测试规划，直接打乱既定进度节奏，成为项目进度滞后的源头性因素。任务评估体系存在缺陷

（2）任务评估体系存在缺陷

时间预估主观化是进度规划偏差的核心原因。肯定态度样本占比 71.74%，否定态度样本仅 5 份，占比 10.87%，为四大原因中肯定占比最高的维度。数据反映，项目任务周期预估严重依赖个人经验判断，往往会从自身角度出发，倾向于高估所需时间，从而导致时间估算过于保守的问题。为了确保任务能够按时完成，他们会选择在估算时间时预留较多的缓冲时间。

（3）资源分配机制不合理

资源分配机制的系统性缺陷是制约任务顺畅推进的关键瓶颈，主要体现在三大方面：一是人力配置失衡，部分阶段人力与任务复杂度、工作量不匹配，如开发阶段核心技术人员短缺，而测试前期人力闲置；二是技术资源分布不均，关键技术资源集中于单一环节，导致设计、运维等阶段因缺乏技术支撑而效率低下；三是资源预置不足，硬件、软件等技术资源未提前规划部署，出现资源缺口时需临时协调，直接延误工作

推进。调查结果显示，超半数参与人员认为项目存在上述各类资源分配不合理问题，成为拖累整体进度的重要因素。

（4）项目团队沟通机制不健全与监控体系缺位

进度风险管控存在的两个主要缺陷是沟通不畅以及缺少监控体系，从而造成问题不能及时被处理，偏差不断增大。从结果来看，两类问题的肯定态度样本都超过了 30 份，认可程度比较高。沟通方面，各团队之间没有一个统一有效的方式进行交流，在一些关键信息上容易出现漏传的情况，比如测试组发现了 Bug 后通过邮件方式反映给其他人员后，开发团队不常浏览而延误修复，需求变动只用即时通讯软件通知可能漏看；监测方面，没有设置进度偏差提醒，只是按里程碑来检查项目进度情况，在一定范围内出现的小范围偏差不容易察觉到，等到截止日期临近才发现，已经来不及采取有效措施了，只能用后续工作的赶工来补偿，从而增大了总工期的拖延概率。

3.4 本章小结

本章围绕 C 公司业务应用运行平台项目的进度管理展开分析，为后续优化方案制定奠定坚实基础。首先，明确项目背景、核心目标与组织架构，该项目作为 Z 公司数字化转型的关键支撑，需在 2025 年 2 月底完成项目交付，但当前已出现需求分析阶段延期，后续节点面临普遍滞后风险。其次，梳理了项目现有进度管理现状，包括粗糙的 WBS 工作分解和依赖主观判断的里程碑计划。之后通过面向 50 名项目人员的问卷调查（回收有效问卷 46 份），结合数据统计与深度分析，定位 C 公司项目进度管理的四大核心问题：一是任务分解粗糙，存在工作项遗漏与边界模糊；二是任务评估体系存在缺陷，过度依赖主观经验，缺乏历史数据支撑；三是资源分配机制不合理，表现为人力配置失衡、技术资源分布不均及资源预置不足，形成任务推进的关键瓶颈；四是沟通机制不健全与监控体系缺位，跨团队信息传递存在断层，进度预警缺失导致偏差难以及时纠正。本章通过梳理现状与问题诊断，明确了后续进度管理优化的核心方向，为第四章设计针对性、可落地的优化方案提供了直接依据。

4 C 公司项目进度管理优化方案

第三章通过向 C 公司业务应用运行平台项目核心成员发放问卷调查，并结合鱼骨图分析法，对项目进行问题识别与原因分析，最终明确了项目进度管理的四大核心症结：任务分解颗粒度粗糙、任务评估体系缺乏科学性、资源分配机制存在系统性缺陷、沟通协作机制不健全与进度监控体系缺位。在此基础上，本章针对项目的进度管理现状，综合运用计划评审技术（PERT）、关键路径法（CPM）、甘特图、关键链法及缓冲区设计等项目管理工具，对该项目进行项目进度管理改进研究，以达到进度计划科学编制、资源分配均衡、风险管理实时的目的。

4.1 基于计划评审技术（PERT）制定进度计划

4.1.1 细化 WBS 工作分解

根据第 3 章调查分析得出的结果来看，原项目的 WBS 分解过于粗略，并且没有涵盖所有工作过程，因此在实施过程中常有工作任务被漏掉或者界限不明等问题发生，已经不能满足当前项目进度管理工作的要求。因此，应对项目进行 WBS 的再次分解，形成更为详细、完整的工作包分解准则。结合项目实际场景与全生命周期管理要求，项目组将整体项目拆解为项目启动、架构设计、系统开发、系统测试、上线实施及项目验收六个核心工作包，每个工作包划分出相应的活动和可交付成果，确保覆盖从项目启动到项目交付的全流程关键环节，各工作模块分解情况见图 4.1 所示。

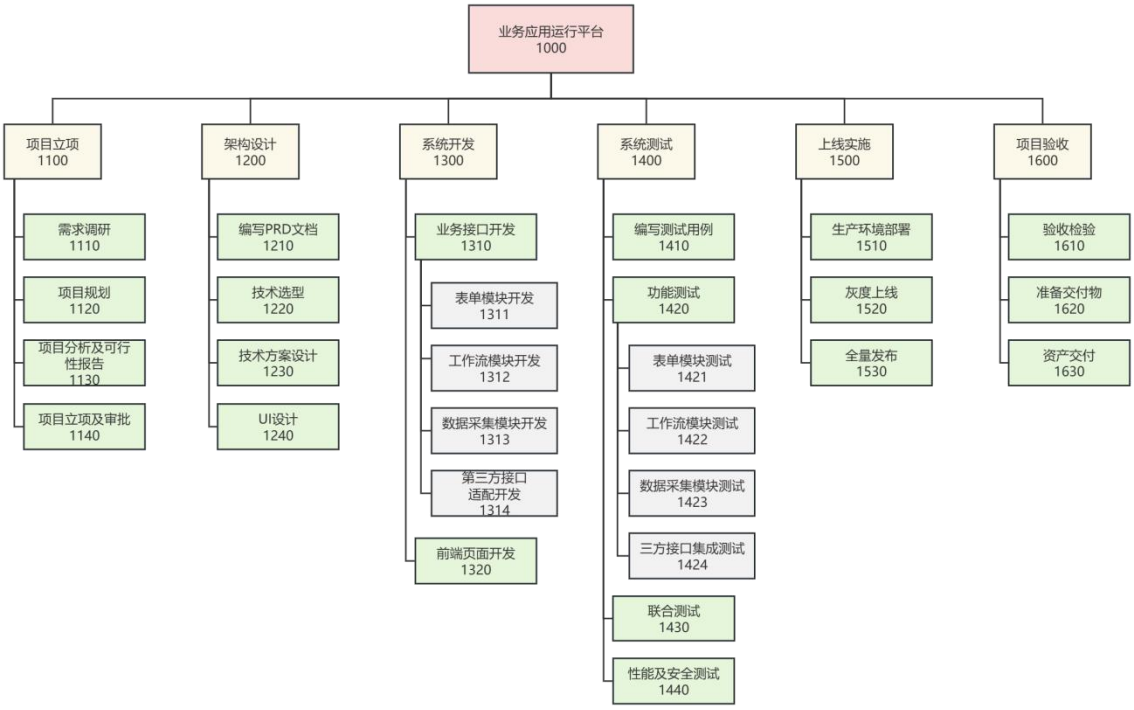


图 4.1 优化后业务应用运行平台项目 WBS 工作分解结构

4.1.2 工作顺序制定及职责明确

（1）工作顺序制定

在完成业务应用运行平台项目 WBS 分解后，还应合理安排每个工作包的任务，这样不仅可以确定工作的先后顺序，而且可以清楚地了解各个任务之间的关系，它是制定项目进度计划的基础。项目活动排序是识别并记录项目各项活动逻辑关联的关键过程，其核心目标是在项目既定约束条件（如资源、时间）下，通过定义合理的工作执行顺序，最大化提升项目推进效率[38]。

在完成项目 WBS 分解的基础上，结合业务应用运行平台项目的实际执行场景与技术特性，通过与项目技术专家、核心执行成员的深度沟通，系统梳理了各项工作的前置依赖与后置关联，最终确定了科学可行的项目活动逻辑顺序，具体排序结果如表 4.1 所示。

（2）工作职责明确

除了对工作进行分解优化，还需明确 WBS 各工作项的执行角色与分工，以此避免任务边界模糊、项目成员职责不清晰的问题。因此，在考虑了项目不同阶段的任务后，对项目执行者进行准确分配，如表 4.1 所示。在项目立项阶段，由产品负责人主

导需求调研（1110），项目负责人统筹项目规划（1120）与项目立项及审批（1140），技术负责人协同完成项目分析及可行性报告（1130）；进入架构设计环节，产品负责人负责输出 PRD 文档（1210），技术负责人牵头推进技术选型（1220）与技术方案设计（1230），UI 设计师独立完成 UI 设计（1240）；系统开发阶段，后端工程师承担业务接口开发（1310）包含多个业务模块开发（1311-1314），前端工程师负责前端页面开发（1320）；系统测试环节（1400），测试工程师负责编写测试用例（1410）、多个开发模块的功能测试（1421-1424）以及联合测试（1430），性能及安全测试（1440）交由专业测试团队执行；上线实施阶段，运维工程师统筹生产环境部署（1510）、灰度上线（1520）与全量发布（1530），产品负责人协同参与灰度验证工作；项目验收阶段，甲方项目负责人主导验收检验（1610），产品负责人与项目负责人协同完成交付物准备（1620），最终由项目负责人落实资产交付（1630）。通过明确的角色分工，各岗位成员的职责边界得以厘清，为项目各环节的有序推进提供保障。

表 4.1 业务应用运行平台项目活动排序表

WBS 编码	**活动名称**	**负责人**	**紧前活动编码**
1110	需求调研	需求分析师、产品负责人	-
1120	项目规划	项目负责人	1110
1130	项目分析及可行性报告	项目负责人、技术负责人	1120
1140	项目立项及审批	项目负责人	1130
1210	编写 PRD 文档	需求分析师、产品负责人	1140
1220	技术选型	技术负责人	1210
1230	技术方案设计	技术负责人	1220
1240	UI 设计	UI 设计师	1210
1311	表单模块开发	后端开发工程师	1230
1312	工作流模块开发	后端开发工程师	1230
1313	数据采集模块开发	后端开发工程师	1230

表 4.1（续） 业务应用运行平台项目活动排序表

WBS 编码	活动名称	负责人	紧前活动编码
1314	第三方接口适配开发	后端开发工程师	1230
1320	前端页面开发	前端开发工程师	1230、1240
1410	编写测试用例	测试工程师	1311、1312、1313、1314、1320
1421	表单模块测试	测试工程师	1410
1422	工作流模块测试	测试工程师	1410
1423	数据采集模块测试	测试工程师	1410
1424	三方接口集成测试	测试工程师	1410
1430	联合测试	测试工程师	1320、1421、1422、1423、1424
1440	性能及安全测试	安全工程师、测试工程师	1430
1510	生产环境部署	运维工程师	1440
1520	灰度上线	运维工程师	1510
1530	全量发布	运维工程师	1520
1610	验收检验	项目负责人	1530
1620	准备交付物	技术负责人、产品经理	1530
1630	资产交付	项目负责人	1610、1620

为了更直观的体现项目中各个活动的逻辑关系，根据表 4.1 的紧前活动编码制作了单代号网络图，如图 4.2 所示。

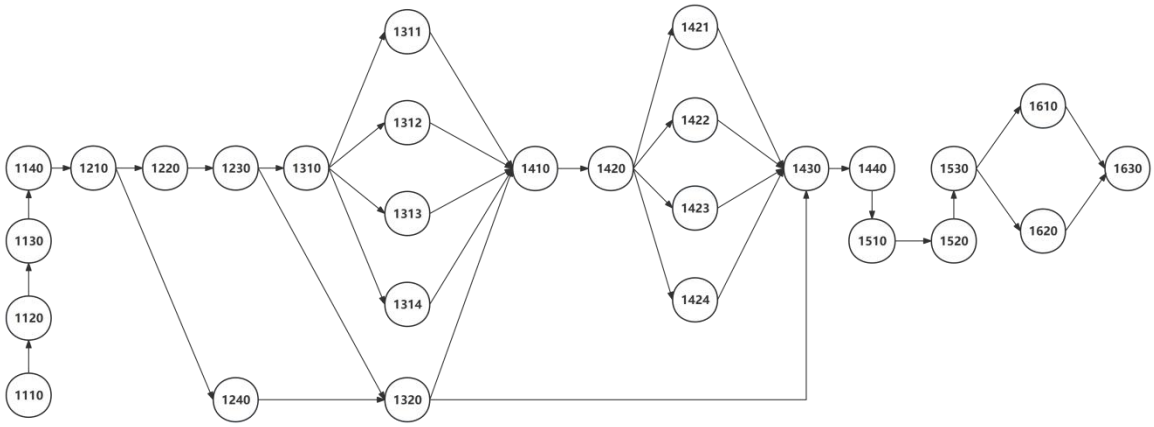


图 4.2 业务应用运行平台单代号网络图

4.1.3 项目活动持续时间估算

对 WBS 分解结果的工作内容进行持续时间估算，是软件开发项目进度管理的核心环节，其科学性与准确性直接决定项目计划编制的合理性，进而影响项目实施的顺利推进与进度控制的有效性[39]。由于项目工程师在项目经验及技能水平上存在个体差异，仅依赖主观判断易导致工期估算出现系统性偏差，因此需建立标准化估算流程以规范评估行为，最大限度降低主观因素干扰。

基于此，本文结合 C 公司业务应用运行平台项目的技术特性，参考历史项目经验，通过多方协同评估和 PERT 三点估算量化的方法开展工期估算。首先由项目负责人牵头，召集产品经理、技术负责人、前后端开发工程师、运维人员、测试工程师等核心团队成员，结合以往类似项目的实践经验与历史数据，通过头脑风暴梳理项目难点、特点及资源需求，综合考量任务规模与复杂程度，形成初步估算。随后，项目负责人对初步结果进行校验调整，并结合领域专家意见进一步修正。最终，引入计划评审技术（PERT）中的三点估算法，明确三项关键时间参数，包含最乐观时间（T_o）、最可能时间（T_m）、最悲观时间（T_p）。

最乐观时间是指在资源充分、流程顺畅、无任何意外延误等最理想条件下，完成该活动的最短可能持续时间；最可能时间是指在资源配置合理、工作流程正常、无特殊干扰等常规工作条件下，完成该活动最可能持续时间，是对活动时长最具代表性的估计值；最悲观时间是指在资源短缺、流程受阻、意外风险发生等最不利条件下，完成该活动的最长可能持续时间。根据三点估算法公式（4.1）计算出项目中各个活动的

工期，根据公式（4.2）和公式（4.3）计算各个活动方差和标准差。根据三点估算法计算出的每个阶段项目活动时间及方差如表 4.2 所示[40]。

$$T_e=\frac{T_o+4T_m+T_p}{6} \tag{4.1}$$

$$\sigma^2=\left(\frac{T_p-T_o}{6}\right)^2 \tag{4.2}$$

$$\sigma=\frac{T_p-T_o}{6} \tag{4.3}$$

表 4.2 业务应用运行平台项目活动持续时间表

WBS 编码	任务名称	T_o	T_m	T_p	T_e	σ^2	σ
1110	需求调研	-	-	-	-	-	-
1120	项目规划	3	5	8	5	0.69	0.83
1130	项目分析及可行性报告	5	8	10	8	0.69	0.83
1140	项目立项及审批	5	10	15	10	2.78	1.67
1210	编写 PRD 文档	10	18	25	18	6.25	2.50
1220	技术选型	3	6	10	6	1.36	1.17
1230	技术方案设计	8	10	12	10	0.44	0.67
1240	UI 设计	8	15	20	15	4.00	2.00
1311	表单模块开发	19	26	33	26	5.44	2.33
1312	工作流模块开发	33	42	50	42	8.03	2.83
1313	数据采集模块开发	24	32	39	32	6.25	2.50
1314	第三方接口适配开发	28	36	45	36	8.03	2.83
1320	前端页面开发	28	35	41	35	4.69	2.17

表 4.2（续）业务应用运行平台项目活动持续时间表

WBS 编码	任务名称	T_o	T_m	T_p	T_e	σ^2	σ
1410	编写测试用例	13	18	22	18	1.50	2.25
1421	表单模块测试	4	8	11	8	1.36	1.17
1422	工作流模块测试	6	10	15	10	2.25	1.50
1423	数据采集模块测试	5	9	12	9	1.36	1.17
1424	三方接口集成测试	4	8	15	9	3.36	1.83
1430	联合测试	4	9	12	9	1.78	1.33
1440	性能及安全测试	3	5	8	5	0.69	0.83
1510	生产环境部署	4	9	12	9	1.78	1.33
1520	灰度上线	5	10	15	10	2.78	1.67
1530	全量发布	3	5	10	6	1.36	1.17
1610	验收检验	10	15	20	15	2.78	1.67
1620	准备交付物	5	10	15	10	2.78	1.67
1630	资产交付	3	5	10	6	1.36	1.17

4.1.4 确定关键路径

关键路径是指从项目开始到结束、由一系列相互依赖的活动所构成的持续时间最长的路径[41]。它决定了项目完成所需的最少时间。如果关键路径中的某个活动延期，则整个项目的完工时间就会相应延长；如果能够缩短关键路径中某些活动的历时，则可以缩短项目的总工期。

关键路径最显著的特征是其上所有活动的自由浮动时间（Free Float, FF）和总浮动时间（Total Float, TF）均为零，即没有缓冲的时间。所以关键路径中的活动就是制

约整个项目进程的关键活动。项目负责人可以通过找出关键路径，对重点项目所需资源予以保证，加强对该项目实施的控制，以更好地把握整个项目的进展状况，为项目进度改进创造条件。

在本项目中，根据表 4.1 对项目工作进行排序和表 4.2 通过三点估算法计算出的工作时长，我们可以通过工具计算出项目中每个活动的自由浮动时间和总浮动时间。如表 4.3 所示。

表 4.3 业务应用运行平台项目活动自由时差和总时差信息表

WBS 编码	任务名称	活动持续时间	自由浮动时间	总时差
1110	需求调研	1	0	0
1120	项目规划	5	0	0
1130	项目分析及可行性报告	8	0	0
1140	项目立项及审批	10	0	0
1210	编写 PRD 文档	18	0	0
1220	技术选型	6	0	0
1230	技术方案设计	10	0	0
1240	UI 设计	15	1	8
1311	表单模块开发	26	16	16
1312	工作流模块开发	42	0	0
1313	数据采集模块开发	32	10	10
1314	第三方接口适配开发	36	6	6
1320	前端页面开发	35	7	7
1410	编写测试用例	4	0	0
1421	表单模块测试	8	2	2
1422	工作流模块测试	10	0	0

表 4.3（续） 业务应用运行平台项目活动自由时差和总时差信息表

WBS 编码	任务名称	活动持续时间	自由浮动时间	总时差
1423	数据采集模块测试	9	1	3
1424	三方接口集成测试	9	1	1
1430	联合测试	9	0	0
1440	性能及安全测试	5	0	0
1510	生产环境部署	9	0	0
1520	灰度上线	10	0	0
1530	全量发布	6	0	0
1610	验收检验	15	0	0
1620	准备交付物	10	5	5
1630	资产交付	6	0	0

根据表中所列各任务的自由浮动时间（FF）和总浮动时间（TF），可以识别出本项目的关键路径。从数据可见，以下任务的总浮动时间均为 0，分别是 1110（需求调研）、1120（项目规划）、1130（项目分析及可行性报告）、1140（项目立项及审批）、1210（编写 PRD 文档）、1220（技术选型）、1230（技术方案设计）、1312（工作流模块开发）、1410（编写测试用例）、1422（工作流模块测试）、1430（联合测试）、1440（性能及安全测试）、1510（生产环境部署）、1520（灰度上线）、1530（全量发布）、1610（验收检验）、1630（资产交付）。而任务 1240（UI 设计）、1311（表单模块开发）、1313（数据采集模块开发）、1314（三方接口适配开发）、1320（前端页面开发）、1421（表单模块测试）、1423（数据采集模块测试）、1424（三方接口集成测试）和 1620（准备交付物）的总时差均不为 0，说明它们位于非关键路径上，存在一定的调度弹性，不会直接影响项目总工期。因此，将关键路径上各任务的持续时间累加，项目总工期为 207 个工作日，为更直观地展示关键路径，已在图 4.3 中对其进行高亮标注。

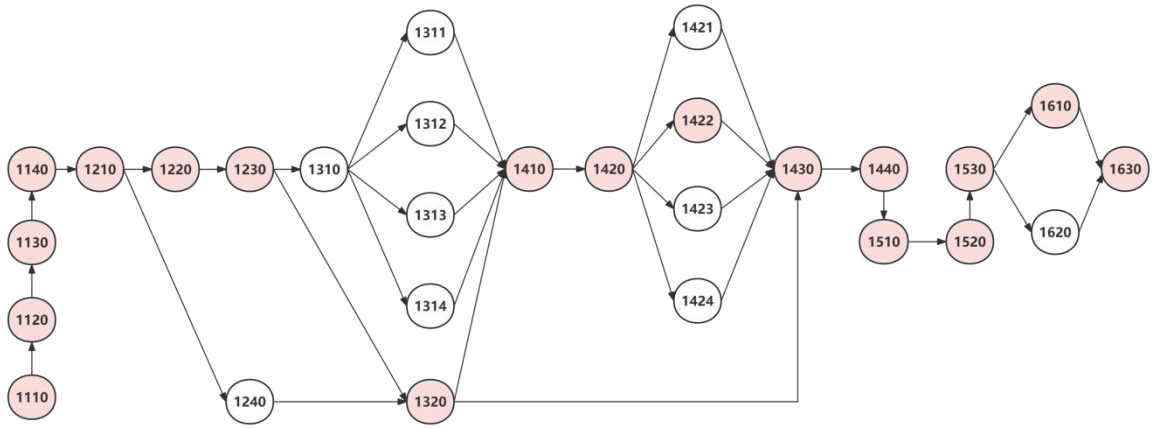
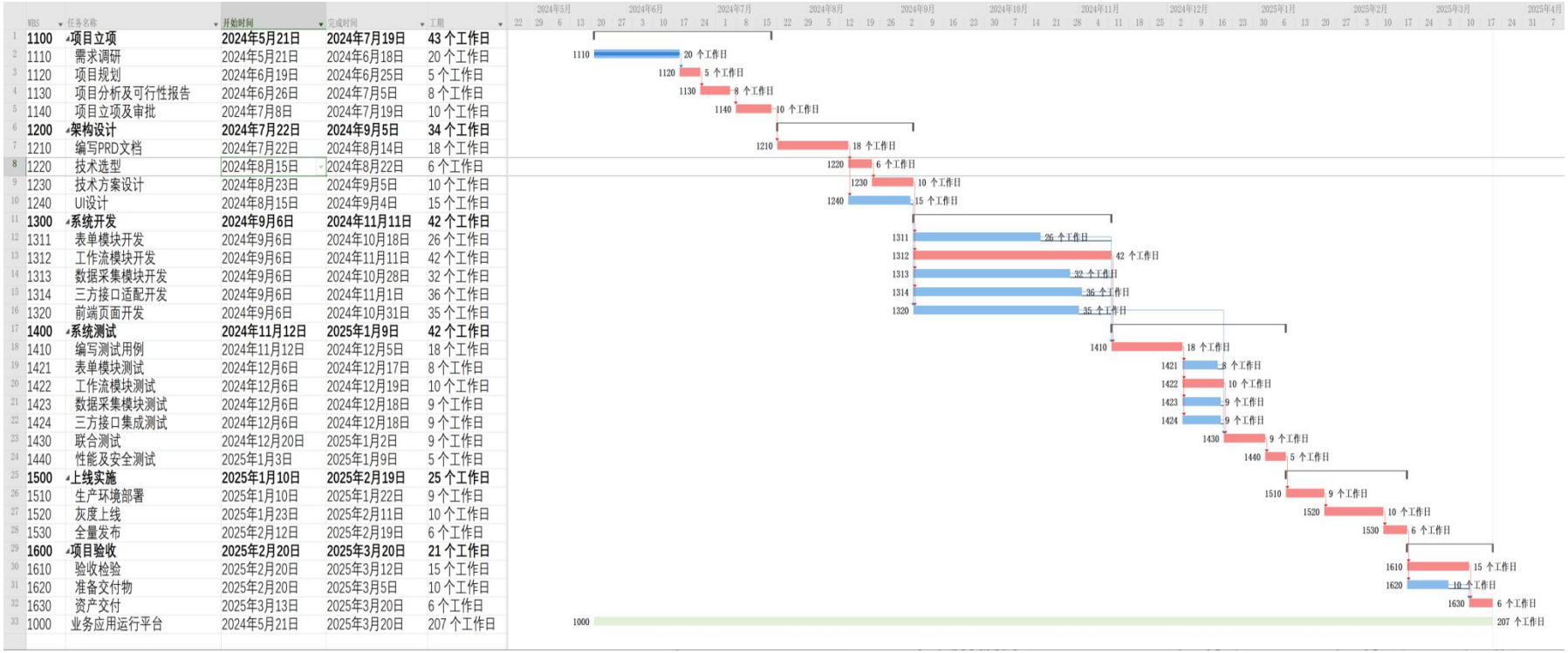


图 4.3 业务应用运行平台关键路径图

同时，为了让项目管理人员更精准的把控项目计划及项目进度，便于对项目进行精细化管理，在本项目中使用甘特图对项目整体进度计划进行展示。作为项目进度展示的基础图形工具，甘特图能以直观图示清晰呈现项目活动的先后顺序与持续时长，方便管理人员快速掌握进度安排。以下为通过 Microsoft Project 软件制作的项目甘特图（见图 4. 4）。



WBS	任务名称	开始时间	完成时间	工期
1100	**项目立项**	**2024年5月21日**	**2024年7月19日**	**43 个工作日**
1110	需求调研	2024年5月21日	2024年6月18日	20 个工作日
1120	项目规划	2024年6月19日	2024年6月25日	5 个工作日
1130	项目分析及可行性报告	2024年6月26日	2024年7月5日	8 个工作日
1140	项目立项及审批	2024年7月8日	2024年7月19日	10 个工作日
1200	**架构设计**	**2024年7月22日**	**2024年9月5日**	**34 个工作日**
1210	编写PRD文档	2024年7月22日	2024年8月14日	18 个工作日
1220	技术选型	2024年8月15日	2024年8月22日	6 个工作日
1230	技术方案设计	2024年8月23日	2024年9月5日	10 个工作日
1240	UI设计	2024年8月15日	2024年9月4日	15 个工作日
1300	**系统开发**	**2024年9月6日**	**2024年11月11日**	**42 个工作日**
1311	表单模块开发	2024年9月6日	2024年10月18日	26 个工作日
1312	工作流模块开发	2024年9月6日	2024年11月11日	42 个工作日
1313	数据采集模块开发	2024年9月6日	2024年10月28日	32 个工作日
1314	三方接口适配开发	2024年9月6日	2024年11月1日	36 个工作日
1320	前端页面开发	2024年9月6日	2024年10月31日	35 个工作日
1400	**系统测试**	**2024年11月12日**	**2025年1月9日**	**42 个工作日**
1410	编写测试用例	2024年11月12日	2024年12月5日	18 个工作日
1421	表单模块测试	2024年12月6日	2024年12月17日	8 个工作日
1422	工作流模块测试	2024年12月6日	2024年12月19日	10 个工作日
1423	数据采集模块测试	2024年12月6日	2024年12月18日	9 个工作日
1424	三方接口集成测试	2024年12月6日	2024年12月18日	9 个工作日
1430	联合测试	2024年12月20日	2025年1月2日	9 个工作日
1440	性能及安全测试	2025年1月3日	2025年1月9日	5 个工作日
1500	**上线实施**	**2025年1月10日**	**2025年2月19日**	**25 个工作日**
1510	生产环境部署	2025年1月10日	2025年1月22日	9 个工作日
1520	灰度上线	2025年1月23日	2025年2月11日	10 个工作日
1530	全量发布	2025年2月12日	2025年2月19日	6 个工作日
1600	**项目验收**	**2025年2月20日**	**2025年3月20日**	**21 个工作日**
1610	验收检验	2025年2月20日	2025年3月12日	15 个工作日
1620	准备交付物	2025年2月20日	2025年3月5日	10 个工作日
1630	资产交付	2025年3月13日	2025年3月20日	6 个工作日
1000	业务应用运行平台	2024年5月21日	2025年3月20日	207 个工作日

图 4.4 业务应用运行平台项目项目计划进度甘特图

4.2 基于关键路径法（CPM）的进度优化

关键路径法作为项目进度管理的关键技术，该方法通过对关键路径上活动的工期或资源进行优化，从而缩短项目的工期并控制进度风险。CPM 的核心是关注项目进度瓶颈，并对其进行有针对性地优化以达到控制项目进度的目的[42]。

结合 4.1 节 PERT 分析结果，本项目的关键路径已明确为：1110（需求调研）→1120（项目规划）→1130（项目分析及可行性报告）→1140（项目立项及审批）→1210（编写 PRD 文档）→1220（技术选型）→1230（技术方案设计）→1312（工作流模块开发）→1410（编写测试用例）→1422（工作流模块测试）→1430（联合测试）→1440（性能及安全测试）→1510（生产环境部署）→1520（灰度上线）→1530（全量发布）→1610（验收检验）→1630（资产交付）。总工期为 207 个工作日，超出甲方要求的 195 个工作日的交付时间，因此需要通过关键路径法压缩项目总工期，保障项目按时交付。

从项目实施角度来看，由于第 3 章已经找到了任务划分粗略、资源不均衡等问题，并且这些问题集中在关键路径上耗时大的活动上，使得关键路径没有压缩余地。为此，有必要引入关键路径法进行分析。本节聚焦关键路径的效率瓶颈，一方面通过关键路径上活动拆分和串并行调整的方法缩减关键路径完成时间，另一方面对关键路径上的冗余流程进行精简，使得关键活动的工期更精准、资源配置更合理、流程更高效，同时解决了第三章识别的任务分解粗糙、资源分配不合理、沟通机制不健全等核心问题，为后续关键链缓冲设置与进度监控奠定了基础。

4.2.1 进度优化

结合第三章识别的任务分解粗糙、资源分配不合理等核心问题，这些问题在关键路径中的业务接口开发和测试环节尤为突出。原有工作中测试相关工作采取串行的方式进行处理，存在着明显的资源分配不合理的问题，使得关键路径上没有足够的可压缩空间以达到对项目工期的约束。因此，在这一部分中，通过对活动进行拆分及串并行调整，将原本集中的测试工作拆分为与开发模块一一对应的平行工作，并对其中的业务接口开发工作和测试工作进行整合以达到同步的目的。

项目负责人通过关键路径分析识别出核心瓶颈，关键路径中的 1410（编写测试用例）活动采用集中式推进模式，需等待所有系统开发子模块（1311 表单模块、1312

工作流模块等）及前端页面开发全部完成后才能统一启动，这种做法使得系统开发与测试部分的关键路径是串联式的，进而导致整个测试阶段等待时间太长，使得关键路线可缩短的空间有限。基于此，本次优化从任务拆解、逻辑重构及流程连接三个维度展开，详细说明如下：

第一步需要拆分 1410（编写测试用例）为模块级子任务。依据 WBS 精细化分解原则，对应业务开发模块将 1410（编写测试用例）活动拆分为 4 个专项子任务，分别为 1411（表单模块编写测试用例）、1412（工作流模块编写测试用例）、1413（数据采集模块编写测试用例）、1414（三方接口编写测试用例）。每个子任务工期根据对应开发模块的复杂度设定，既贴合 WBS 的细化颗粒度，也为后续业务测试并行提供基础。

第二步需要调整业务开发与测试用例的逻辑关系。打破原流程中全量开发完成后再统一编写测试用例的串行逻辑，建立单个模块开发完成即启动对应模块测试用例编写的并行关联模式，例如 1311（表单模块开发）完成后，直接启动 1411（表单模块测试用例编写），无需等待其他开发模块收尾，有效规避了全量开发完成后的集中等待冗余，提升了任务推进效率。

最后，关联测试用例与模块测试串行逻辑。建立单个模块测试用例编写完成后对应模块测试启动的精准关联。以表单模块为例，1411（表单模块测试用例编写）完成后，立即启动 1421（表单模块测试），其他模块遵循相同逻辑推进。该调整替代了原先测试用例编写完成后再统一启动模块测试的模式，最终形成同一功能模块内业务接口开发、测试用例编写、模块功能测试串行推进，不同模块间并行作业的优化流程，进一步压缩了测试环节的总耗时。为了更加直观的呈现优化后各任务间的关系，重新绘制了进度优化后的单代号网络图（见图 4.5）。

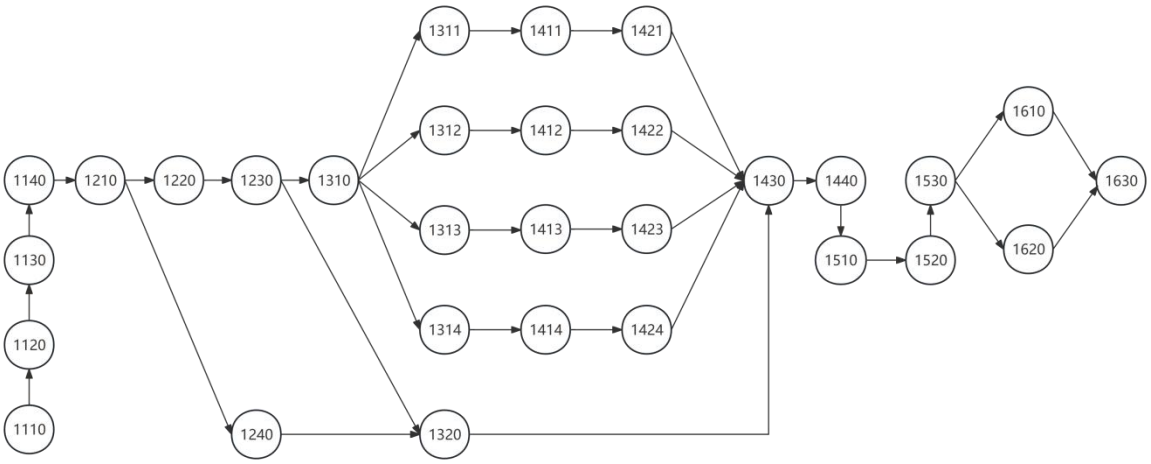


图 4.5 优化后业务应用运行平台单代号网络图

本次活动主要通过任务拆分以及串并行逻辑更改进行项目优化，在此改进下，本项目开发和测试在关键路径上的串行问题得到了解决，并且也使得测试的任务资源可以与开发的任务相互匹配，从而进一步地缩短了核心阶段的工期。经重新整理任务间逻辑关系、梳理任务名称及工期等内容后统计发现，从系统开发到系统测试的整体周期由原来的 85 天缩减为 74 天，大大增强了关键路线的机动性和可操作性，成为整个项目在给定时间限制内完成的关键所在。

4.2.2 流程优化

在 C 公司历史项目中，导致工期拖延的因素除了任务串行外还有因跨部门，跨组织沟通链路长，内部流程层级多而出现的反复评审、反复返工现象。结合第三章通过问卷调查与鱼骨图分析识别的沟通机制不健全的问题，本文节选取了关键路径中的两个任务，以编写 PRD 文档与验收检验为优化点，通过简化内部审批和提前协调甲方参与任务为优化思路，在严格按照甲方要求的基础上，重组审批流程，降低沟通成本以及需求不匹配带来的重复工作风险，缩短关键路径时间。

（1）编写 PRD 文档流程优化

PRD 文档是需求调研到开发阶段中间的重要环节，旨在把之前的需求调研结果抽象出来，以明确的技术实现方式、可测试性及甲方认可的标准呈现，直接决定后续技术选型（1220）、UI 设计（1240）、系统开发（1300）等环节的执行方向，对整个项目的开发需求有着至关重要的影响。其重要性体现在 PRD 文档的交付质量与推进效率直接影响项目整体进度，若此项工作出现延误，或因审核复杂导致流转过慢，会直

接导致后续所有关键节点的推迟，造成任务链式反应。而 PRD 文档的不完善或者含糊不清也可能导致后期开发结果偏离甲方需求，返工成本增加，甚至会影响关键路线的安全性，对于满足项目的工期约束有着很大的风险。

该任务原执行流程为内部产品初稿→产品部评审→技术部预审→甲方业务评审→修改→甲方终验（具体流程如图 4.6），全程涉及多次评审以及跨部门信息传递，冗余流程导致工期占用 18 个工作日。结合 C 公司历史项目实践数据，此类多环节串行评审极易引发两类问题：一是缺乏沟通，内部产品部与技术部门的评审意见未完整同步甲方，导致甲方评审时出现需求偏差；二是需求对齐不及时，核心需求点没有提前进行确认，造成后面反复修改，直接影响到关键路径周期。

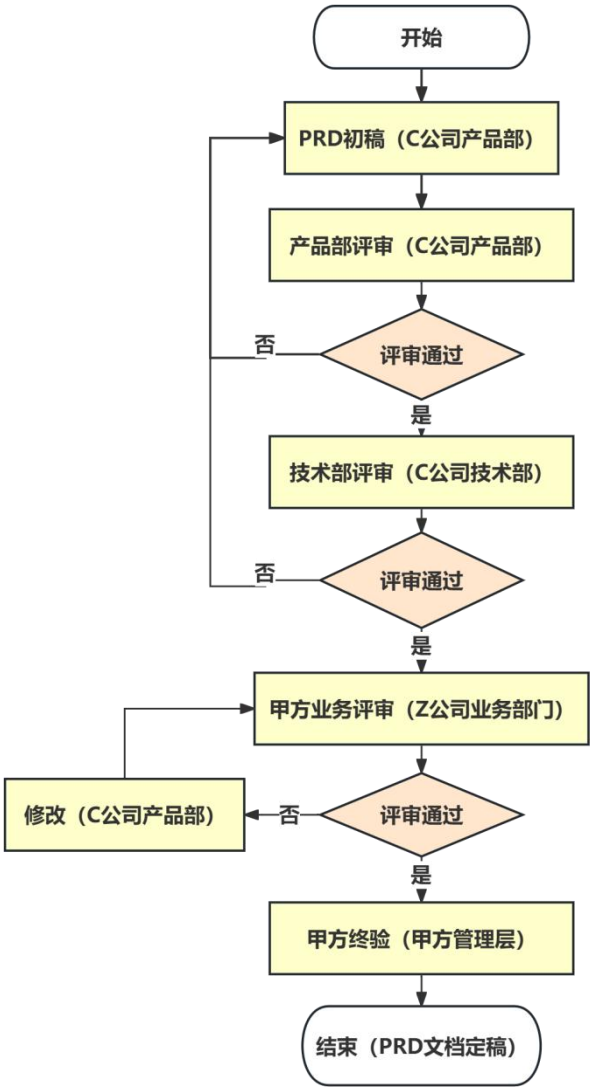


图 4.6 优化前编写 PRD 文档审批流程图

针对上述痛点，优化方案主要分为两点，首先对内部流程合并，精简内部评审环节，将原独立的产品部评审与技术部预审合并为内部联合评审，由产品经理、技术负责人同步参与，一次性输出需求可行性和技术落地性双维度意见，减少内部信息传递耗时；其次是提前协调甲方参与任务，邀请甲方业务对接人直接参与内部联合评审，现场对齐字段定义、审批流程逻辑等核心需求细节，避免后期评审时出现需求偏差。优化后流程简化为内部产品初稿→甲乙双方联合评审（甲方业务对接人）→修改→甲方终验（具体流程如图 4.7），既保留甲方最终审批权限，又通过前期协同减少返工风险。

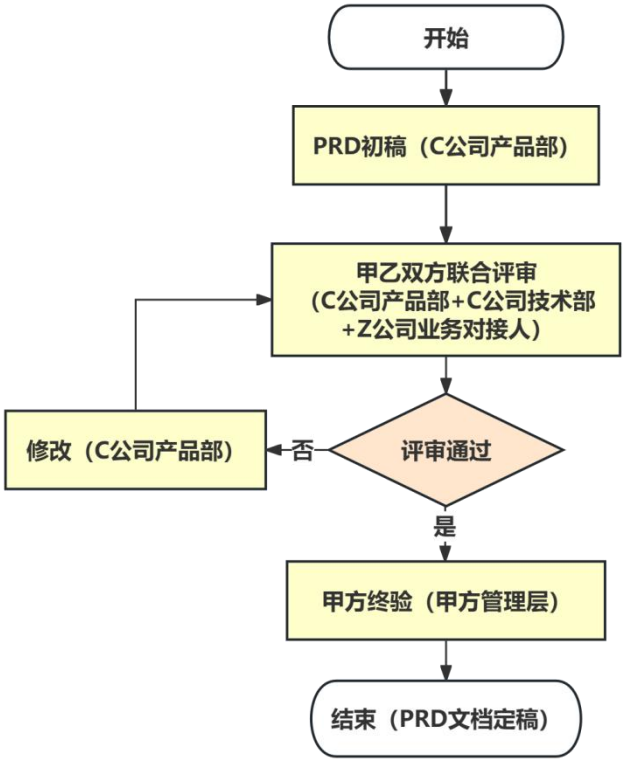


图 4.7 优化后编写 PRD 文档流程图

（2）验收检验流程优化

验收检验工作是在项目结束阶段进行的质量管理活动，主要目的是确认所提供的产品功能及产品质量是否满足用户方的需求以及合同中规定的要求，保证向用户提供满足要求的产品，并作为资产移交（1630）的前提条件之一，实施过程要及时记录验收结果情况，作为准备可交付成果（1620）中的验收报告等相关文件的基础信息。

这项任务的原执行流程参照 C 公司历史项目规范，采用内部验收→测试报告提交→甲方对接人审核→甲方业务部门复核→甲方技术部门确认→验收通过的串行模式（如图 4.8），流程链路包含 6 个环节。其中甲方 3 个部门的独立审核，导致反馈周期比较长。在不改变甲方内部审批规范与合规要求的前提下，流程冗余主要体现在两方面：一是内部验收环节分阶段推进，功能验证、性能核验、缺陷整改确认各自独立开展，存在文档重复提交、信息重复核验的浪费；二是甲方多部门审核串行，对接人、业务部门、技术部门依次审核，不仅单环节反馈耗时久，还容易出现审核意见不一致的情况。

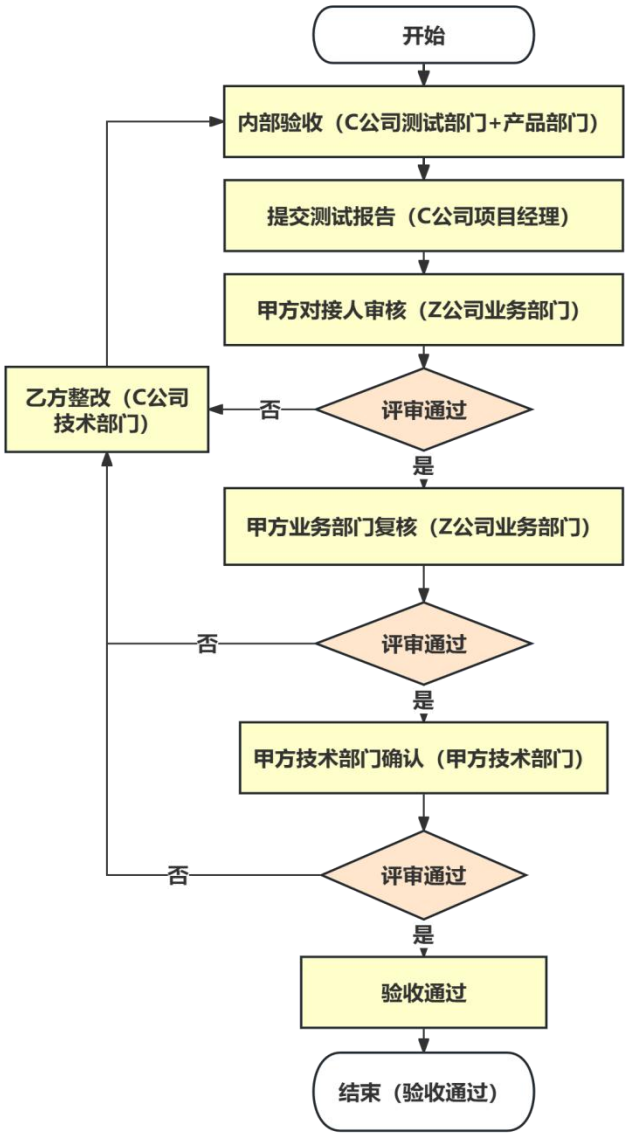


图 4.8 优化前验收检验流程图

为了解决上述情况，我们从内审机制以及与甲方的合作方式进行了改进：首先，将内部审核集中到一起，即项目负责带领测试人员、开发工程师及产品需求分析员对项目进行内部审查，并在此过程中完成功能测试、性能测试、bug 修复这三个任务，把多个版本报告合并成一个版本报告。包括测试用例、bug 修复表单、关键功能检查项，一并提交给甲方，而不是像以前那样按模块逐步提交；二是在合作方式上进行优化，项目负责人在交付前一周会先与甲方沟通确定相关职能部门及技术部门的关键验收点，例如功能上线标准、性能要求等，预先统一验收意见；同时协调甲方对接人组织业务关键人员和技术部负责人参加验收评审会，到现场进行功能确认、沟通分歧点，并达成验收共识，以免后期再由多个单位分别开展审核而延长周期。优化后流程简化为内部联合验收→甲乙双方联合评审→甲方技术部门评审修改→验收通过（具体流程如图 4.9）

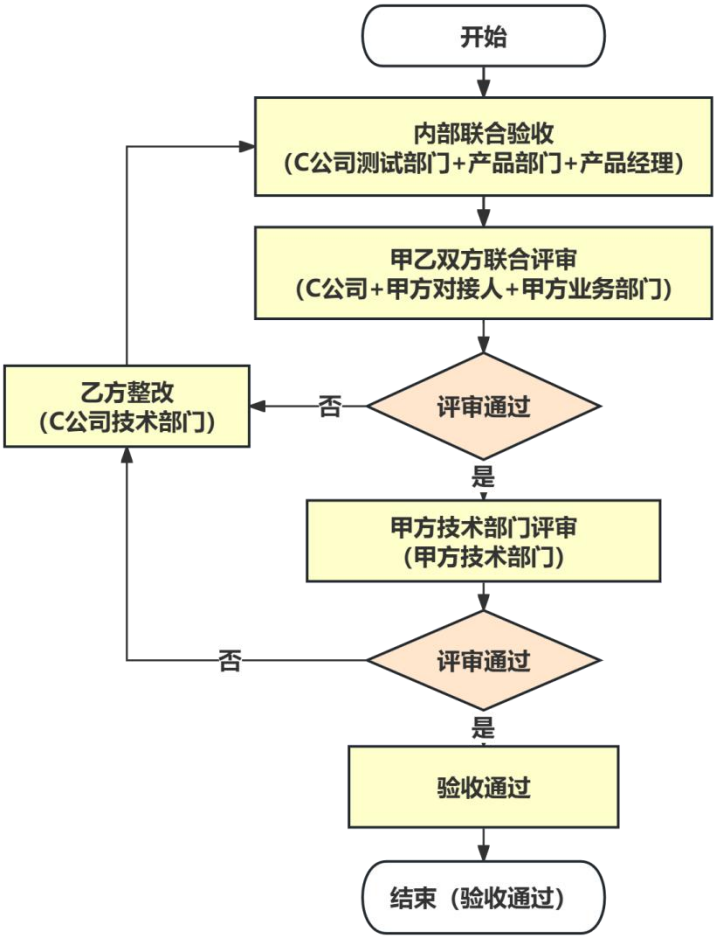


图 4.9 优化后验收检验流程图

结合关键活动流程的冗余精简优化，编写 PRD 文档（1210）与验收检验（1610）两项关键任务的工期效率与跨方沟通质量均实现显著提升，有效缓解了项目前期存在的沟通不畅问题，同时达成了压缩关键路径工期的核心目标。优化前后的具体效果对比详见表 4.4。

表 4.4 项目流程优化效果比对

WBS 编码	任务名称	原工期	优化后工期	原评审次数	优化后评审次数
1210	编写 PRD 文档	18	15	4 次	2 次
1610	验收检验	15	12	4 次	3 次

由表 4.7 数据可知，编写 PRD 文档的工期从 18 个工作日缩短至 15 个工作日，评审次数从 4 次精简为 2 次，核心得益于内部评审环节合并与甲方前置协同，减少了跨部门传递与返工耗时；验收检验的工期从 15 个工作日压缩至 12 个工作日，评审次数从 4 次减少为 3 次，主要通过内部联合验收、甲方多角色联合评审模式，解决了原流程中串行审核导致的反馈周期冗长问题。

4.2.3 基于 CPM 进度优化效果

为系统验证关键路径法（CPM）在项目进度优化中的应用成效，优化后首先对项目内各任务间的逻辑关系进行重新梳理与明确，重点确认各任务的紧前紧后关联，同时由于进度优化中把编写测试用例拆分成 4 个子任务，所以需要重新根据三点估算法计算 4 个子模块编写测试用例时间，再以关键路径法为核心，将活动自由时差、总时差作为核心计算依据，重新推导项目关键路径。具体操作中，先将调整后的任务名称、工期及逻辑关系等信息录入 Microsoft Project 2016 软件，更新活动拆分与流程优化后的相关数据，进而重新计算各活动的自由时差与总时差。依据总时差为 0 的活动构成关键路径的判定规则，本次优化通过单项任务工期压缩与串并行关系调整，不仅改变了优化前的关键路径，更实现了关键路径的工期压缩，确保项目可按进度计划按时交付。最终，通过软件自动运算功能生成 CPM 优化后的业务应用运行平台项目活动自由时差与总时差信息表（详见表 4.5），并在图 4.10 中对优化后关键路径对应的任务活动进行了重新高亮标注。

表 4.5 CPM 优化后的业务应用运行平台项目活动自由时差与总时差信息表

WBS 编码	任务名称	活动持续时间	紧前任务	自由时差	总时差	σ^2	σ
1110	需求调研	20	-	0	0		
1120	项目规划	5	1110	0	0	0.69	0.83
1130	项目分析及可行性报告	8	1120	0	0	0.69	0.83
1140	项目立项及审批	10	1130	0	0	2.78	1.67
1210	编写 PRD 文档	15	1140	0	0	6.25	2.50
1220	技术选型	6	1210	0	0	1.36	1.17
1230	技术方案设计	10	1220	0	0	0.44	0.67
1240	UI 设计	15	1210	1	24	4.00	2.00
1311	表单模块开发	26	1230	0	20	5.44	2.33
1312	工作流模块开发	42	1230	0	0	8.03	2.83
1313	数据采集模块开发	32	1230	0	14	6.25	2.50
1314	第三方接口适配开发	36	1230	0	9	8.03	2.83
1320	前端页面开发	35	1230、1240	23	23	4.69	2.17
1411	表单模块编写测试用例	4	1311	0	20	0.11	0.33
1412	工作流模块编写测试用例	6	1312	0	0	0.44	0.67
1413	数据采集模块编写测试用例	3	1313	0	14	0.25	0.50
1414	三方接口编写测试用例	5	1314	0	9	0.44	0.67
1421	表单模块测试	8	1411	20	20	1.36	1.17
1422	工作流模块测试	10	1412	0	0	2.25	1.50

表 4.5（续） CPM 优化后的业务应用运行平台项目活动自由时差与总时差信息表

WBS 编码	任务名称	活动持续时间	紧前任务	自由时差	总时差	σ^2	σ
1423	数据采集模块测试	9	1413	14	14	1.36	1.17
1424	三方接口集成测试	9	1414	9	9	3.36	1.83
1430	联合测试	9	1320、1421、1422、1423、1424	0	0	1.78	1.33
1440	性能及安全测试	5	1425	0	0	0.69	0.83
1510	生产环境部署	9	1430	0	0	1.78	1.33
1520	灰度上线	10	1510	0	0	2.78	1.67
1530	全量发布	6	1520	0	0	1.36	1.17
1610	验收检验	12	1530	0	0	2.78	1.67
1620	准备交付物	10	1530	2	2	2.78	1.67
1630	资产交付	6	1610、1620	0	0	1.36	1.17

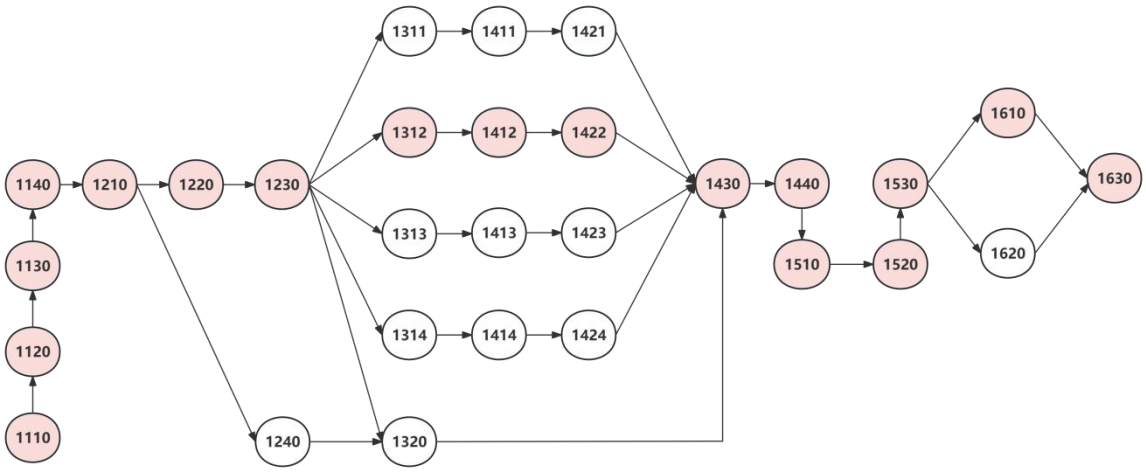
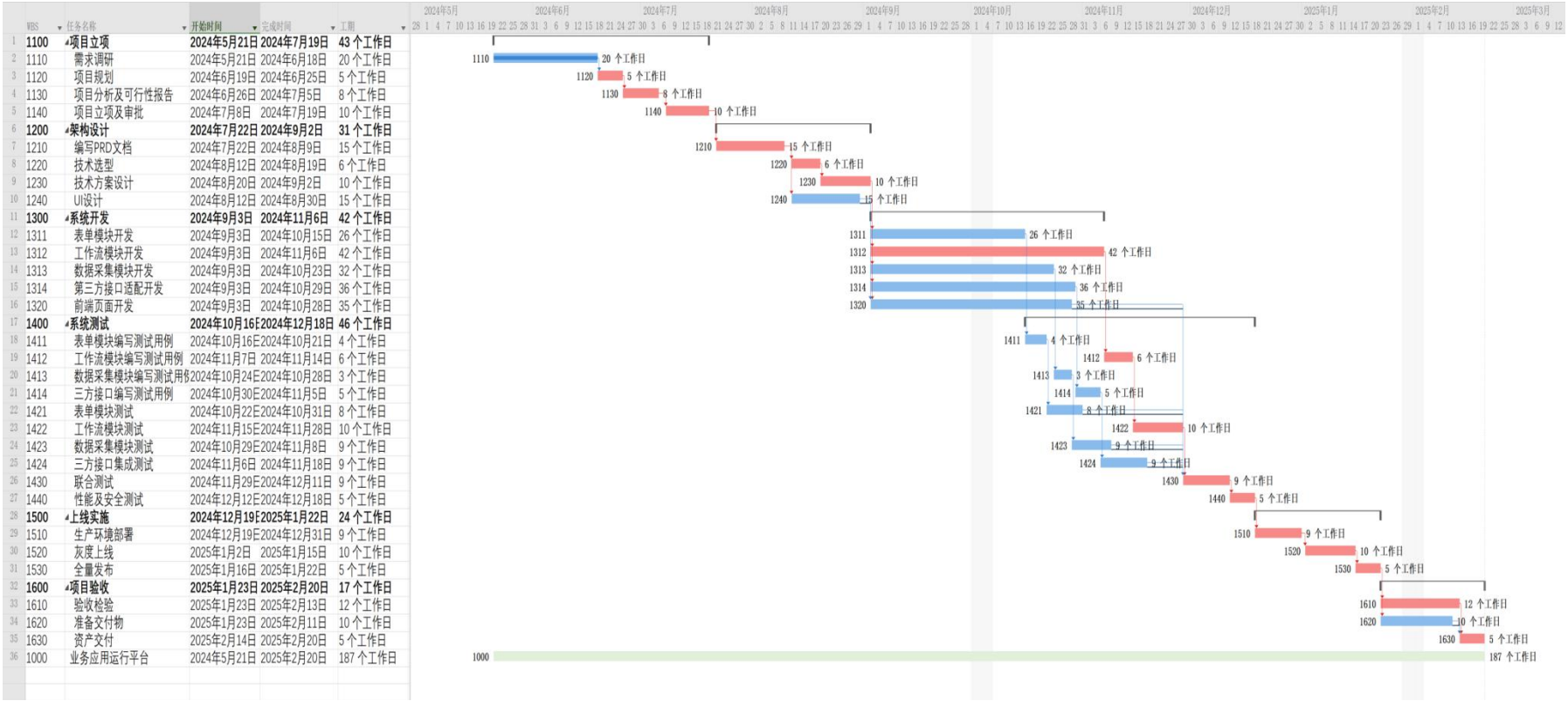


图 4.10 基于 CPM 优化后业务应用运行平台关键路径图

业务应用运行平台项目经过关键路径法（CPM）优化后关键路径调整为 1110（需求调研）→1120（项目规划）→1130（项目分析及可行性报告）→1140（项目立项及审批）→1210（编写 PRD 文档）→1220（技术选型）→1230（技术方案设计）→1312（工作流模块开发）→1412（工作流模块编写测试用例）→1422（工作流模块测试）→1430（联合测试）→1440（性能及安全测试）→1510（生产环境部署）→1520（灰度上线）→1530（全量发布）→1610（验收检验）→1630（资产交付）。项目整体进度计划甘特图如图 4.11 所示。通过在 CPM 框架下采用采取活动分解、串并联调整以及流程优化相结合的方法，项目总工期从原 207 个工作日压缩至 187 个工作日，给项目纠偏预留了缓冲期。



WBS	任务名称	开始时间	完成时间	工期
1100	**项目立项**	**2024年5月21日**	**2024年7月19日**	**43 个工作日**
1110	需求调研	2024年5月21日	2024年6月18日	20 个工作日
1120	项目规划	2024年6月19日	2024年6月25日	5 个工作日
1130	项目分析及可行性报告	2024年6月26日	2024年7月5日	8 个工作日
1140	项目立项及审批	2024年7月8日	2024年7月19日	10 个工作日
1200	**架构设计**	**2024年7月22日**	**2024年9月2日**	**31 个工作日**
1210	编写PRD文档	2024年7月22日	2024年8月9日	15 个工作日
1220	技术选型	2024年8月12日	2024年8月19日	6 个工作日
1230	技术方案设计	2024年8月20日	2024年9月2日	10 个工作日
1240	UI设计	2024年8月12日	2024年8月30日	15 个工作日
1300	**系统开发**	**2024年9月3日**	**2024年11月6日**	**42 个工作日**
1311	表单模块开发	2024年9月3日	2024年10月15日	26 个工作日
1312	工作流模块开发	2024年9月3日	2024年11月6日	42 个工作日
1313	数据采集模块开发	2024年9月3日	2024年10月23日	32 个工作日
1314	第三方接口适配开发	2024年9月3日	2024年10月29日	36 个工作日
1320	前端页面开发	2024年9月3日	2024年10月28日	35 个工作日
1400	**系统测试**	**2024年10月16日**	**2024年12月18日**	**46 个工作日**
1411	表单模块编写测试用例	2024年10月16日	2024年10月21日	4 个工作日
1412	工作流模块编写测试用例	2024年11月7日	2024年11月14日	6 个工作日
1413	数据采集模块编写测试用例	2024年10月24日	2024年10月28日	3 个工作日
1414	三方接口编写测试用例	2024年10月30日	2024年11月5日	5 个工作日
1421	表单模块测试	2024年10月22日	2024年10月31日	8 个工作日
1422	工作流模块测试	2024年11月15日	2024年11月28日	10 个工作日
1423	数据采集模块测试	2024年10月29日	2024年11月8日	9 个工作日
1424	三方接口集成测试	2024年11月6日	2024年11月18日	9 个工作日
1430	联合测试	2024年11月29日	2024年12月11日	9 个工作日
1440	性能及安全测试	2024年12月12日	2024年12月18日	5 个工作日
1500	**上线实施**	**2024年12月19日**	**2025年1月22日**	**24 个工作日**
1510	生产环境部署	2024年12月19日	2024年12月31日	9 个工作日
1520	灰度上线	2025年1月2日	2025年1月15日	10 个工作日
1530	全量发布	2025年1月16日	2025年1月22日	5 个工作日
1600	**项目验收**	**2025年1月23日**	**2025年2月20日**	**17 个工作日**
1610	验收检验	2025年1月23日	2025年2月13日	12 个工作日
1620	准备交付物	2025年1月23日	2025年2月11日	10 个工作日
1630	资产交付	2025年2月14日	2025年2月20日	5 个工作日
1000	业务应用运行平台	2024年5月21日	2025年2月20日	187 个工作日

图 4.11 基于 CPM 优化后业务应用运行平台项目计划进度甘特图

4.3 基于关键链法的资源优化

前文通过关键路径法（CPM）完成开发和测试环节的串并行逻辑调整与工期压缩，有效解决了任务分解粗糙、关键路径过长的问题，但该方法存在显著局限性，而关键链法（CCM）在其基础上补充考量了多重核心问题，为后续资源优化提供了更贴合项目实际的理论支撑。具体而言，在关键路径法的基础上，关键链法还特别强调以下三点问题：第一是实际存在的资源限制条件，关键路径法只考虑了工作间的先后制约以及时间上的制约，而关键链法则会考虑到一个项目中的主要资源是否合理分配，防止出现资源超负荷或者资源竞争等问题进而造成进度安排不成立的情况发生[43]。第二是人为因素引发的工期浪费，针对项目团队普遍存在的拖延现象，通过重构安全时间规避隐性工期损耗；三是对进度风险进行整体防御，采取有效的缓冲方法将关键路径上的活动和非关键路径的风险分开从而增强项目的进度鲁棒性[44]。基于此，为弥补关键路径法的不足，本节引入关键链法开展针对性的资源优化工作。

4.3.1 项目资源分配与资源约束分析

项目实施阶段的资源分配，需要根据资源特点、质量以及是否有利于项目目标的实现来安排资源的任务和工作。在本业务应用运行平台项目中，资源配置主要是人员配置，并按照项目工作分解结构层次进行划分，对应覆盖项目立项、架构设计、系统开发、系统测试、上线实施、项目验收六大核心工作包。项目团队共 22 人，成员包括项目负责人、产品经理、需求分析师、技术负责人、UI 设计师、后端开发工程师、前端开发工程师、测试工程师、安全工程师及运维工程师。人力资源的合理配置直接决定项目进度、质量与成本目标的达成，是保障项目顺利推进的核心支撑。具体人力资源配置详情参见表 4.6。

表 4.6　项目人力资源表

部门	资源名称	资源数量	资源编码
项目开发部	项目负责人	1	P1
项目开发部	UI 设计师	2	P2
项目开发部	后端开发工程师	5	P3
项目开发部	前端开发工程师	3	P4
咨询规划部	产品经理	1	C1
咨询规划部	需求分析师	2	C2
技术服务部	技术负责人	1	T1
技术服务部	测试工程师	2	T2
技术服务部	安全工程师	2	T3
技术服务部	运维工程师	3	T4

资源需求分析以 4.1 节构建的 WBS 任务清单为基础，结合 4.2 节关键路径优化后的进度计划，在根据 C 公司项目组实际资源储备情况，明确各层级任务的核心资源。从资源属性来看，本项目核心资源可划分为三类：其一为技术决策类资源，以技术负责人为核心，主要承担技术方案审核、核心模块开发指导、技术风险预判等关键决策环节；其二为开发实施类资源，即核心开发人员，重点负责 1311 表单模块、1312 工作流模块等业务接口开发模块的落地；其三为环境支撑类资源，核心是专用测试环境，为 1421 表单模块测试、1422 工作流模块测试等提供硬件部署、软件适配等基础运行支撑。

为了直观的得到每个工作包与资源之间的匹配情况，先梳理项目立项、架构设计、系统开发、系统测试、上线实施、项目验收六大工作包的资源匹配情况，并在此基础上针对可以同时进行的任务进行了深入剖析，查看在这些任务中使用的资源是否会出现瓶颈。经分析，项目存在三类核心并行任务，其一，架构设计阶段的 1220（技术选型）、1230（技术方案设计）与 1240（UI 设计）并行任务。其中，1220 与 1230 任务由 T1（技术负责人）主导推进，1240 任务由 P2（UI 设计师）独立负责，两类资源职责划分清晰，无交叉占用，不存在资源冲突问题。其二，系统开发与系统测试阶

段的并行任务。经 4.2 节关键路径优化后，该阶段形成 4 个业务模块并行推进，单个模块内开发、编写测试用例、模块测试串行执行的模式。从资源支撑能力来看，开发资源配置充足，共 5 名开发人员，可满足 4 个业务模块并行开发的需求，但测试资源存在明显瓶颈，项目仅配置 2 名测试人员，而 4 个模块的测试用例编写与模块测试任务需同步推进，单靠现有测试资源无法支撑多模块并行测试工作，成为制约该阶段进度推进的核心资源约束。其三，项目验收阶段的 1610（验收检验）与 1620（准备交付物）并行任务。其中，1610 任务由 P1（项目负责人）负责，1620 任务由 T1（技术负责人）与 P2（UI 设计师）协同完成，三类资源的任务分配无重叠，资源供给能力可完全匹配任务需求，不存在资源瓶颈。经过分析发现业务应用运行平台中每个工作包的活动最大并行数就是需要资源分配的数量，每项工作包分配项目成员以及数量以及活动最大并行数如下表 4.7 所示。

表 4.7 项目资源分配表

工作包名称	资源名称	最大并行数	资源分配数量
项目立项	P1、C1、C2	1	4
架构设计	C1、T1、P2	1	4
系统开发	P3、P4	5	8
系统测试	T2、T3	4	2
上线实施	T4	1	3
项目验收	P1、C1、T1	2	3

在发现系统测试工作包存在资源冲突后，进一步分析其任务构成可知，系统测试环节核心包含编写测试用例与模块测试两个阶段，且每个模块需完整完成这两个阶段的工作。为了降低各个测试员之间的模块交接难度，保证测试效率及质量，确定以模块为维度将测试任务划分为 4 个任务组，每组包括编写测试用例和模块测试 2 项工作，并且每个任务组均由同一人完成。

结合各模块开发完成时间与测试任务递进逻辑，明确 4 个测试任务组的具体时间窗口与工期：表单模块测试任务组执行时间为 2024/10/16-2024/10/31，共计 12 个工作日；工作流模块测试任务组，执行时间为 2024/11/07-2024/11/28，共计 16 个工作

日；数据采集模块测试任务组，执行时间为 2024/10/24-2024/11/08，共计 12 个工作日；三方接口集成测试任务组，执行时间为 2024/10/30-2024/11/18，共计 14 个工作日。

通过 Microsoft Project 2016 的资源负荷视图与甘特图叠加分析，将 4 个测试任务组的时间窗口与 2 名测试人员的资源供给能力进行匹配，发现存在严重的资源分配不合理现象。根据每组测试工作的持续期可以归纳出主要在以下两个时间段内出现资源分配不合理现象：第一时段为 2024/10/15-2024/10/20，此阶段表单模块测试任务组、数据采集模块测试任务组、三方接口集成测试任务组三项任务同步推进，按单任务组 1 名测试人员的配置需求，累计需 3 名测试人员，而项目组实际仅配置 2 名，缺少 1 名测试人员,资源冲突率达 150%；第二时段为 2024/10/21-2024/10/23，开始进入工作流模块测试任务组，至此 4 个测试任务组进入全员并行执行状态，需 4 人参与测试活动，相对于现有 2 人的资源投入，出现超载 2 倍的情况，资源冲突率上升到 200%，若按原进度计划与资源配置推进项目，将导致多个测试任务组因人力不足被迫中断或延期，从而对后继性能测试、安全性测试等任务造成影响，对整个项目的进度造成影响。

4.3.2 按照任务优先级重新排序

本小节针对业务应用运行平台项目中受资源约束的活动，结合关键路径、任务风险程度、任务时长三个实际项目维度，对任务重新进行排序。即通过对各个任务分别分配不同优先级来确定资源的分配，把资源优先投入到对于整个项目最重要的任务上，即关键路线上高风险且持续时间长的任务，优先保证高价值任务的资源分配。因此需要将关键路径、任务风险程度、任务时长三大指标，转化为可量化的优先级评估指标。具体操作上，先为三个维度分别制定定量化标准与权重分配规则，权重根据项目实际情况由项目负责人和专家组制定，再基于该标准与权重计算各任务的优先级得分，得分越高则资源分配的优先级越高。三个项目维度量化标准和权重如下：

（1）是否在关键路径：若任务在关键路径上计 2 分，否则计 1 分。权重占 40%。

（2）任务风险程度：通过活动时间估算方差来衡量风险，高方差计 2 分，低方差计 1 分。权重占 30%。

（3）任务时长：活动较长计 2 分，活动较短计 1 分。权重占 30%。

根据以上计分规则，结合表 4.2 项目活动持续时间表以及图 4.6 分别对表单模块测试任务组、工作流模块测试任务组、数据采集模块测试任务组和三方接口集成测试任务组进行打分。如表 4.8 所示。

表 4.8 受资源约束活动量化评估计分表

任务名称（权重）	关键路径（40%）	任务风险程度（30%）	任务时长（30%）	总得分
表单模块测试任务组	1	1	1	1
工作流模块测试任务组	2	2	2	2
数据采集模块测试任务组	1	1	1	1
三方接口集成测试任务组	1	2	2	1.6

由上表可知，受资源约束的四项活动中工作流模块测试任务组和三方接口集成测试任务组的优先级更高，因此这两项活动应该优先得到项目资源，表单模块测试任务组和三方接口集成测试任务组优先级低较低。

4.3.3 资源分配方案设计与对比分析

上一小节已明确任务优先级排序结果，工作流模块优先级最高，三方接口集成次之，数据采集模块用例、表单模块用例优先级最低。基于该排序，先采用纯优先级导向的资源分配逻辑设计方案，再结合项目实际情况，如任务开始时间差异、工程师经验层级等因素优化出适配实际场景的优先级落地方案，通过两种排序方式的落地效果对比，筛选最优资源分配策略，实现优先级、工期与资源效率的动态平衡。

（1）纯优先级导向的排序及资源分配方案

该资源分配方案仅以优先级为唯一依据，不考虑任务启动时间早晚、工程师能力差异等项目实际因素。测试工程师 A 完成最高优先级的工作流模块测试任务，按原始时间窗口 2024/11/07-2024/11/28（16 个工作日）执行，测试工程师 B 完成次高优先级的三方接口集成测试任务，按最早启动时间 2024/10/30-2024/11/18 执行（14 个工作日）。表单模块测试和数据采集模块优先级最低，且工期都为 12 个工作日，由测试工程师 B 承接和三方接口业务更贴合的表单模块测试任务，测试工程师 A 承接和工作流业务更紧凑的数据采集模块测试任务。具体时间安排如表 4.9 所示。

表 4.9 纯优先级导向资源分配表

任务名称	资源名称	开始时间	结束时间	时长
表单模块测试任务组	B	2024/11/19	2024/12/04	12
工作流模块测试任务组	A	2024/11/07	2024/11/28	16
数据采集模块测试任务组	A	2024/11/29	2024/12.16	12
三方接口集成测试任务组	B	2024/10/30	2024/11/18	14

（2）结合实际场景的优先级排序及资源分配方案

该任务调度算法主要根据上述的任务排序的结果为基准，并考虑任务的实际开始时间和测试人员的经验从而对任务进行资源调度，在资源调度过程中，对于开始时间早的任务给予较多的初始资源，而在测试人员安排中，给有丰富经验的人员安排关键性高的任务，用来确保关键节点的质量。具体资源分配如表 4.10 所示，测试工程师 A 负责工作流模块测试、数据采集模块测试两项任务，按照任务实际启动时间依次推进数据采集模块测试、工作流模块测试，实现资源的连续高效利用，避免资源闲置；测试工程师 B 承接表单模块测试、三方接口集成测试两项任务，依据任务开始时间先后顺序，先完成表单模块测试任务，再推进三方接口集成测试任务。

表 4.10 结合实际场景的资源分配表

任务名称	资源名称	开始时间	结束时间	时长
表单模块测试任务组	B	2024/10/16	2024/10/31	12
工作流模块测试任务组	A	2024/11/11	2024/12/02	16
数据采集模块测试任务组	A	2024/10/24	2024/11/08	12
三方接口集成测试任务组	B	2024/11/01	2024/11/20	14

（3）两种资源分配对比

综上，二者各有利弊，在考虑实际情况下的资源配置方式更适合于 2 位测试人员的配置情况，最为理想。纯优先级为参考的方式虽然满足了按优先级排序的核心要求，但是却忽略了其他因素，首先是测试人员 A 在 2024/11/07 前一直处于空闲状态，没有充分利用好资源。其次是因工作流测试开始时间较晚，导致数据采集模块测试在

2024/12/16 才完成，项目周期严重拖延，最后是没有充分考虑到工程师的经验匹配，关键性工作无法保证交付成果的质量。而结合实际场景的方案，以小幅工期延长为代价，通过优先级优先、时间窗口匹配、能力适配三维优化，解决了纯优先级方案的核心痛点，既满足任务时序与执行要求，又实现资源合理分配，最适用于本项目，根据实际场景的优先级排序后的关键路径如图 4.12 所示，项目整体进度计划甘特图如图 4.13 所示。

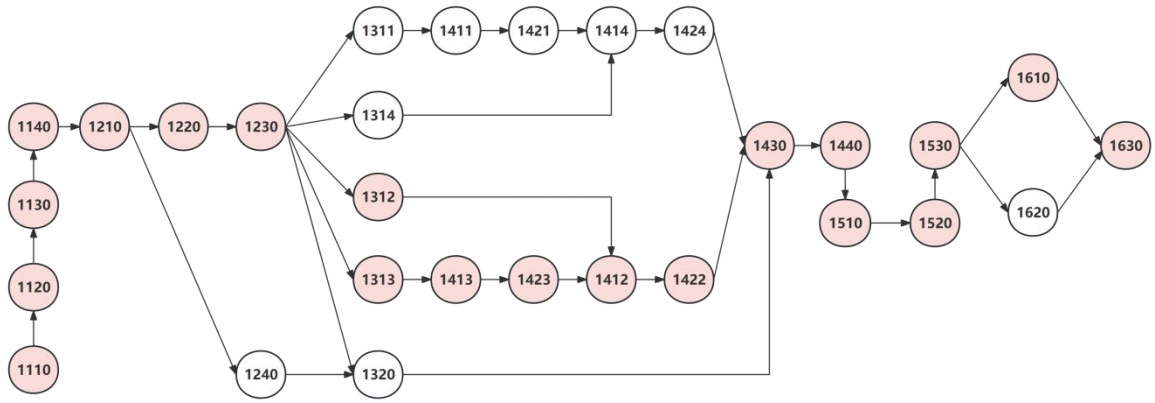
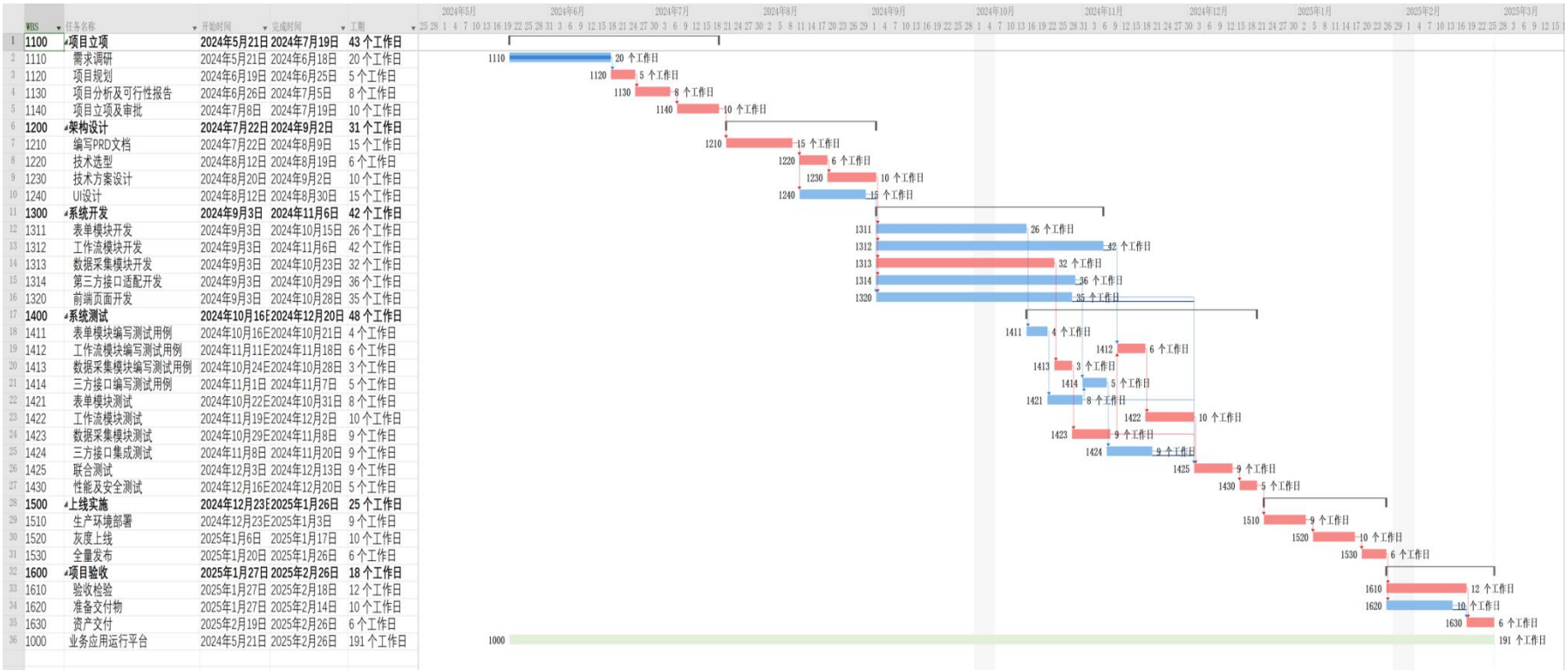


图 4.12 依据实际场景及任务优先级排序后业务应用运行平台关键路径图



	WBS	任务名称	开始时间	完成时间	工期
1	1100	**项目立项**	**2024年5月21日**	**2024年7月19日**	**43 个工作日**
2	1110	需求调研	2024年5月21日	2024年6月18日	20 个工作日
3	1120	项目规划	2024年6月19日	2024年6月25日	5 个工作日
4	1130	项目分析及可行性报告	2024年6月26日	2024年7月5日	8 个工作日
5	1140	项目立项及审批	2024年7月8日	2024年7月19日	10 个工作日
6	1200	**架构设计**	**2024年7月22日**	**2024年9月2日**	**31 个工作日**
7	1210	编写PRD文档	2024年7月22日	2024年8月9日	15 个工作日
8	1220	技术选型	2024年8月12日	2024年8月19日	6 个工作日
9	1230	技术方案设计	2024年8月20日	2024年9月2日	10 个工作日
10	1240	UI设计	2024年8月12日	2024年8月30日	15 个工作日
11	1300	**系统开发**	**2024年9月3日**	**2024年11月6日**	**42 个工作日**
12	1311	表单模块开发	2024年9月3日	2024年10月15日	26 个工作日
13	1312	工作流模块开发	2024年9月3日	2024年11月6日	42 个工作日
14	1313	数据采集模块开发	2024年9月3日	2024年10月23日	32 个工作日
15	1314	第三方接口适配开发	2024年9月3日	2024年10月29日	36 个工作日
16	1320	前端页面开发	2024年9月3日	2024年10月28日	35 个工作日
17	1400	**系统测试**	**2024年10月16日**	**2024年12月20日**	**48 个工作日**
18	1411	表单模块编写测试用例	2024年10月16日	2024年10月21日	4 个工作日
19	1412	工作流模块编写测试用例	2024年11月11日	2024年11月18日	6 个工作日
20	1413	数据采集模块编写测试用例	2024年10月24日	2024年10月28日	3 个工作日
21	1414	三方接口编写测试用例	2024年11月1日	2024年11月7日	5 个工作日
22	1421	表单模块测试	2024年10月22日	2024年10月31日	8 个工作日
23	1422	工作流模块测试	2024年11月19日	2024年12月2日	10 个工作日
24	1423	数据采集模块测试	2024年10月29日	2024年11月8日	9 个工作日
25	1424	三方接口集成测试	2024年11月8日	2024年11月20日	9 个工作日
26	1425	联合测试	2024年12月3日	2024年12月13日	9 个工作日
27	1430	性能及安全测试	2024年12月16日	2024年12月20日	5 个工作日
28	1500	**上线实施**	**2024年12月23日**	**2025年1月26日**	**25 个工作日**
29	1510	生产环境部署	2024年12月23日	2025年1月3日	9 个工作日
30	1520	灰度上线	2025年1月6日	2025年1月17日	10 个工作日
31	1530	全量发布	2025年1月20日	2025年1月26日	6 个工作日
32	1600	**项目验收**	**2025年1月27日**	**2025年2月26日**	**18 个工作日**
33	1610	验收检验	2025年1月27日	2025年2月18日	12 个工作日
34	1620	准备交付物	2025年1月27日	2025年2月14日	10 个工作日
35	1630	资产交付	2025年2月19日	2025年2月26日	6 个工作日
36	1000	业务应用运行平台	2024年5月21日	2025年2月26日	191 个工作日

图 4.13 依据实际场景及任务优先级排序后业务应用运行平台项目计划进度甘特图

4.3.4 消除安全时间

在项目的实施期间难免会出现一些意料之外的情况发生，例如需求变更、关键成员离职或者遇到技术瓶颈等问题都可能影响进度计划和交付节点。因此项目管理团队一般会在每个阶段中设置一个时间余量来避免意外事件的发生而对项目的正常进行造成影响，保证项目按时完成。这些冗余工期本来应该给项目的不确定性做最后的保险，然而分布在各个活动中的冗余工期，其利用率往往并不高。首先，冗余工期容易让团队成员产生惰性，下意识地觉得有足够多的时间可以拖延，因此不会过于关注自身工作进程，造成进度隐藏耗时。同时，各个活动单独设置的安全时间未考虑整体优化设计，并不能有效地规避风险，也无法对不同活动、不同时期的多种风险进行有效的规避。因此，应避免单个设置安全时间，对冗余的安全时间进行计算并去除，在风险分析的基础上统一设置安全时间，这也是加强项目施工管理的重要环节。

本小节结合项目 WBS 任务清单和三点估算法计算的活动时长，以此为基础核算对应工期的方差，将方差作为工期波动与不确定性的核心量化指标。方差数值与工期离散程度、任务风险等级呈正相关，可对任务风险进行分级界定：方差 0-1 为低风险任务，方差 1-3 为中风险任务，方差 3 及以上为高风险任务。方差仅能反映工期波动程度，标准差可将其转化为与工期单位一致的量化指标，但二者均无法体现项目对工期达标的风险容忍度，因此引入重要性系数 z，作为连接风险等级与工期波动的核心参数，其中 z 为关键路径属性、业务核心程度、工期波动三要素的加权平均和；鉴于本项目工作受工期、需求变更、技术难点、第三方依赖等非随机因素影响显著，本研究采用标准差并结合重要性系数 z 的方法确定各项工作安全时间，既保留方差的客观统计依据，又实现风险的量化转化，贴合软件项目实际管控需求，具体安全时间计算公式为：

$$S_i=(w_1\cdot C_i+w_2\cdot B_i+w_3\cdot F_i)\times\sigma_i \tag{4.4}$$

式中，S 表示安全时间，σ_i表示第 i 个工作活动的标准差。C_i表示关键路径属性系数，反映延误对总工期的影响程度。B_i表示业务核心程度系数，反映延误的业务损失程度。F_i为工期波动系数，客观适配工期波动幅度，方差越大、波动越剧烈，系数越高。w_1、w_2、w_3为各维度权重系数，总和为 1。

本项目基于 WBS 表格中各活动的方差、标准差等核心参数，统一采用公式（4.4）计算各工作活动的安全时间，具体实施时先由项目负责人召集专家及核心项目组成员确定关键路径属性、业务核心程度、工期波动三类系数的权重分配，设定关键路径属性权重w_1=0.4、业务核心程度权重w_2=0.4、工期波动权重w_3=0.2，该权重组合适配所有活动的计算逻辑以确保参数统一，再明确各系数的取值标准以实现安全时间的精准量化，其中关键路径属性系数C_i中关键路径活动取 1.0、非关键路径活动取 0.5，业务核心程度系数B_i中核心业务及技术支撑类活动取 1.0、辅助测试及交付类活动取 0.2，工期波动系数F_i则结合活动方差σ^2来量化工期波动，按波动幅度分级取值，高波动（$\sigma^2 \geq 3$）活动取 1.0，中波动（$1 \leqslant \sigma^2 < 3$）活动取 0.6，低波动（$\sigma^2 < 1$）活动取 0.2，确保安全时间与工期波动幅度精准匹配。

以 1210 编写 PRD 文档为例，通过表 4.6 可知，该活动标准差为 2.50，由于该活动方差为 6.25 属于高波动活动，故工期波动系数为 1.0，且该活动为关键路径上的活动，故关键路径属性系数为 1.0。在业务核心度方面，该任务为 1220 技术选型和 1240UI 设计的紧前任务，属于核心任务，故业务核心程度系数为 1.0；将上述参数及系数代入公式（4.4），计算结果为 2.5。按照向上取整的计算方法，该活动取整后安全时间为 3 天。

其他所有活动均按上述逻辑，提取标准差、方差，结合活动特性及统一取值规则确定关键路径属性系数、业务核心程度系数、工期波动系数，代入公式（4.4）完成计算，安全时间按要求四舍五入向上取整，具体详见表 4.11。

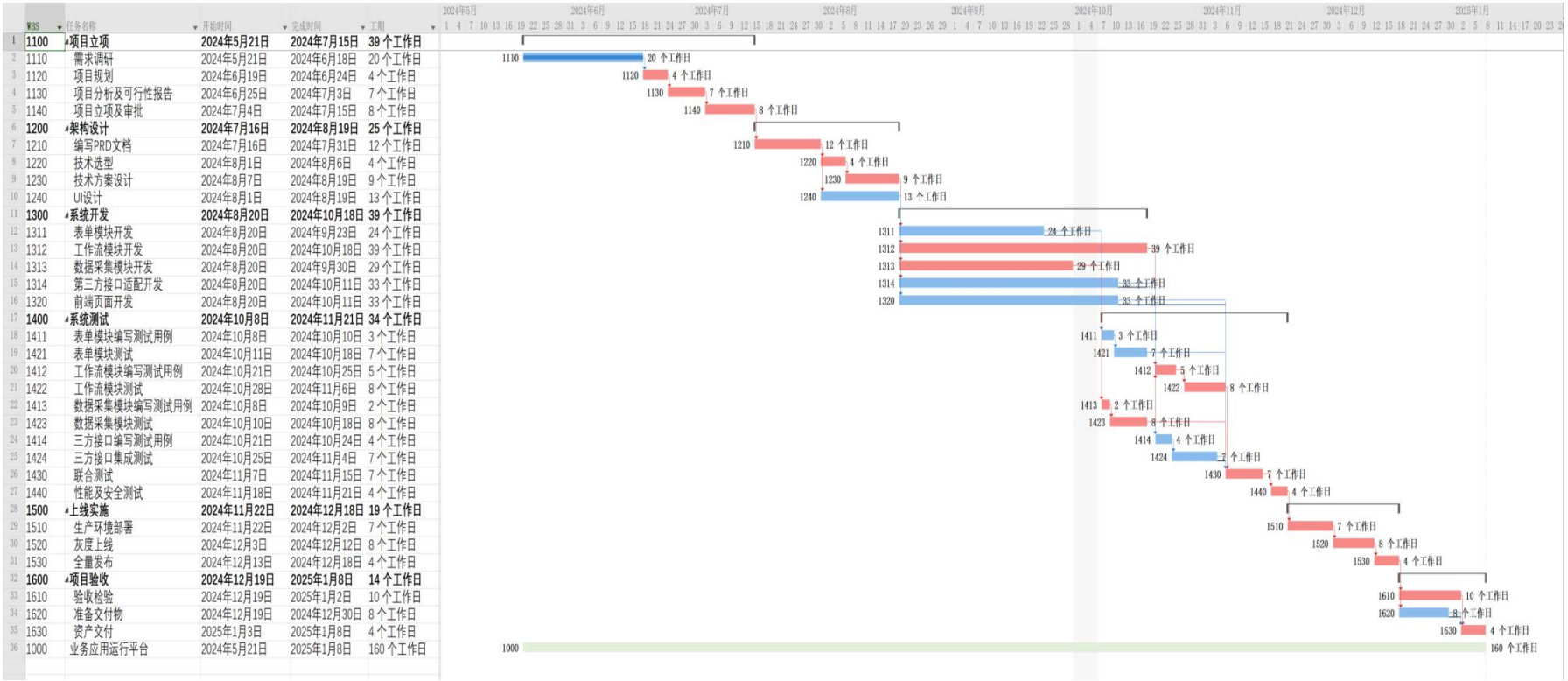
4.11 业务应用运行平台 WBS 各工作安全时间

WBS 编码	任务名称	调整前活动时间	σ^2	σ	C	B	F	S	调整后活动时间
1110	需求调研	20	-	-	-	-	-	-	-
1120	项目规划	5	0.69	0.83	1	1	0.2	1	4
1130	项目分析及可行性报告	8	0.69	0.83	1	1	0.2	1	7
1140	项目立项及审批	10	2.78	1.67	1	1	0.6	2	8
1210	编写 PRD 文档	15	6.25	2.50	1	1	1	3	12
1220	技术选型	6	1.36	1.17	1	1	0.6	2	4
1230	技术方案设计	10	0.44	0.67	1	1	0.2	1	9
1240	UI 设计	15	4.00	2.00	0.5	0.5	1	2	13
1311	表单模块开发	26	5.44	2.33	0.5	1	1	2	24
1312	工作流模块开发	42	8.03	2.83	1	1	1	3	39
1313	数据采集模块开发	32	6.25	2.50	1	1	1	3	29
1314	第三方接口适配开发	36	8.03	2.83	0.5	1	1	3	33
1320	前端页面开发	35	4.69	2.17	0.5	1	1	2	33
1411	表单模块编写测试用例	4	0.11	0.33	0.5	0.5	0.2	1	3
1412	工作流模块编写测试用例	6	0.44	0.67	1	0.5	0.2	1	5
1413	数据采集模块编写测试用例	3	0.25	0.50	1	0.5	0.2	1	2
1414	三方接口编写测试用例	5	0.44	0.67	0.5	0.5	0.2	1	4

4.11（续） 业务应用运行平台 WBS 各工作安全时间

WBS编码	任务名称	调整前活动时间	σ^2	σ	C	B	F	S	调整后活动时间
1421	表单模块测试	8	1.36	1.17	0.5	0.5	0.2	1	7
1422	工作流模块测试	10	2.25	1.50	1	0.5	0.6	2	8
1423	数据采集模块测试	9	1.36	1.17	1	0.5	0.6	1	8
1424	三方接口集成测试	9	3.36	1.83	0.5	0.5	1	2	7
1430	联合测试	9	1.78	1.33	1	1	0.6	2	7
1440	性能及安全测试	5	0.69	0.83	1	1	0.2	1	4
1510	生产环境部署	9	1.78	1.33	1	1	0.6	2	7
1520	灰度上线	10	2.78	1.67	1	1	0.6	2	8
1530	全量发布	6	1.36	1.17	1	1	0.6	2	4
1610	验收检验	12	2.78	1.67	1	1	0.6	2	10
1620	准备交付物	10	2.78	1.67	0.5	1	0.6	2	8
1630	资产交付	6	1.36	1.17	1	1	0.6	2	4

根据表 4.5 以及表 4.12，利用 Microsoft Project 软件重新制作进度计划图，如图 4.14 所示。



WBS	任务名称	开始时间	完成时间	工期
1100	**项目立项**	**2024年5月21日**	**2024年7月15日**	**39 个工作日**
1110	需求调研	2024年5月21日	2024年6月18日	20 个工作日
1120	项目规划	2024年6月19日	2024年6月24日	4 个工作日
1130	项目分析及可行性报告	2024年6月25日	2024年7月3日	7 个工作日
1140	项目立项及审批	2024年7月4日	2024年7月15日	8 个工作日
1200	**架构设计**	**2024年7月16日**	**2024年8月19日**	**25 个工作日**
1210	编写PRD文档	2024年7月16日	2024年7月31日	12 个工作日
1220	技术选型	2024年8月1日	2024年8月6日	4 个工作日
1230	技术方案设计	2024年8月7日	2024年8月19日	9 个工作日
1240	UI设计	2024年8月1日	2024年8月19日	13 个工作日
1300	**系统开发**	**2024年8月20日**	**2024年10月18日**	**39 个工作日**
1311	表单模块开发	2024年8月20日	2024年9月23日	24 个工作日
1312	工作流模块开发	2024年8月20日	2024年10月18日	39 个工作日
1313	数据采集模块开发	2024年8月20日	2024年9月30日	29 个工作日
1314	第三方接口适配开发	2024年8月20日	2024年10月11日	33 个工作日
1320	前端页面开发	2024年8月20日	2024年10月11日	33 个工作日
1400	**系统测试**	**2024年10月8日**	**2024年11月21日**	**34 个工作日**
1411	表单模块编写测试用例	2024年10月8日	2024年10月10日	3 个工作日
1421	表单模块测试	2024年10月11日	2024年10月18日	7 个工作日
1412	工作流模块编写测试用例	2024年10月21日	2024年10月25日	5 个工作日
1422	工作流模块测试	2024年10月28日	2024年11月6日	8 个工作日
1413	数据采集模块编写测试用例	2024年10月8日	2024年10月9日	2 个工作日
1423	数据采集模块测试	2024年10月10日	2024年10月18日	8 个工作日
1414	三方接口编写测试用例	2024年10月21日	2024年10月24日	4 个工作日
1424	三方接口集成测试	2024年10月25日	2024年11月4日	7 个工作日
1430	联合测试	2024年11月7日	2024年11月15日	7 个工作日
1440	性能及安全测试	2024年11月18日	2024年11月21日	4 个工作日
1500	**上线实施**	**2024年11月22日**	**2024年12月18日**	**19 个工作日**
1510	生产环境部署	2024年11月22日	2024年12月2日	7 个工作日
1520	灰度上线	2024年12月3日	2024年12月12日	8 个工作日
1530	全量发布	2024年12月13日	2024年12月18日	4 个工作日
1600	**项目验收**	**2024年12月19日**	**2025年1月8日**	**14 个工作日**
1610	验收检验	2024年12月19日	2025年1月2日	10 个工作日
1620	准备交付物	2024年12月19日	2024年12月30日	8 个工作日
1630	资产交付	2025年1月3日	2025年1月8日	4 个工作日
1000	业务应用运行平台	2024年5月21日	2025年1月8日	160 个工作日

图 4.14 消除安全时间后业务应用运行平台项目计划进度甘特图

4.3.5 设置缓冲区

缓冲区设置是关键链项目管理的核心环节，其作用是通过集中配置缓冲抵消项目工期的不确定性，降低风险对项目总工期的冲击，同时消除各活动分散预留的安全时间，将冗余时间整合为持续可用的缓冲资源。关键链技术的缓冲区主要分为接驳缓冲（FB）、项目缓冲（PB）与资源缓冲（RB）三类。接驳缓冲（FB）设置在非关键链汇入关键链的节点前，主要作用是阻断非关键链活动的延误向关键链传导，避免非关键路径的工期波动冲击核心进度[45]。项目缓冲（PB）直接设置在关键链末端，核心作用是抵御关键链上各活动的工期延误，直接兜底项目总工期，是关键链技术中最核心的缓冲类型，确保项目整体交付节点不受关键活动波动影响[46]。资源缓冲（RB）为关键链活动所需的核心资源预留的虚拟缓冲，不占用实际工期，仅作为资源调配的预警与保障机制，确保关键链活动启动时所需资源可及时到位，规避资源冲突导致的进度延误[47]。结合业务应用运行平台项目的资源约束与进度管控实际需求，仅配置接驳缓冲（FB）与项目缓冲（PB）两类，暂不设置资源缓冲（RB）。

缓冲区计算和设置是关键链法核心内容之一。常见的计算缓冲区大小主要有三种方法，首先是剪切粘贴法，核心是将关键链和接驳链路各工序消除的安全时间累加，取累加值的 50%-70%作为对应缓冲区[48]；其次是根方差法，又称标准差法，是一种基于统计原理的计算方法。它通过将项目中各个活动的安全时间进行量化，并计算这些安全时间的标准差，从而确定项目缓冲区的大小[49]；最后是基于内外部影响因素的缓冲计算方法，核心是量化工序内外部影响因素，通过构建综合影响系数模型，结合工序基准时长核算缓冲区，匹配工序实际风险特征。

对于业务应用运行平台这类软件项目，剪切粘贴法过于机械粗放，未考虑项目风险动态分布，将高风险和低风险活动安全时间同等处理，导致缓冲区设置失衡。C 公司项目软件开发活动风险等级差异显著，该方法无法动态响应风险变化。根方差法仅适用于极少数标准化、数据充足、活动独立性高的简单软件项目，对于业务应用运行平台项目这种定制化软件项目不合适。而基于内外部影响因素的缓冲计算方法，不仅能贴合软件定制项目的核心风险，适配软件迭代、关键链管理等主流实践模式，还能有效应对项目的高不确定、强依赖等特性，是更适配的缓冲计算方案。因此，本文针对业务应用运行平台选择用基于内外部影响因素的缓冲计算方法计算缓冲区大小。

（1）设置缓冲区位置

项目缓冲（PB）设置在关键链末尾。根据图 4.12 可知项目关键路径有两条，且在 1412（工作流模块编写测试用例）处汇入到一条关键链中，因此项目缓冲（PB）设置在关键链后即可。

接驳缓冲（FB）设置在非关键链汇入关键链的节点前。根据图 4.12 可知本项目有 3 处需要设置接驳缓冲。分别是 1320（前端页面开发）、1424（三方接口集成测试）和 1620（准备交付物）。具体缓冲位置如图 4.15 所示。

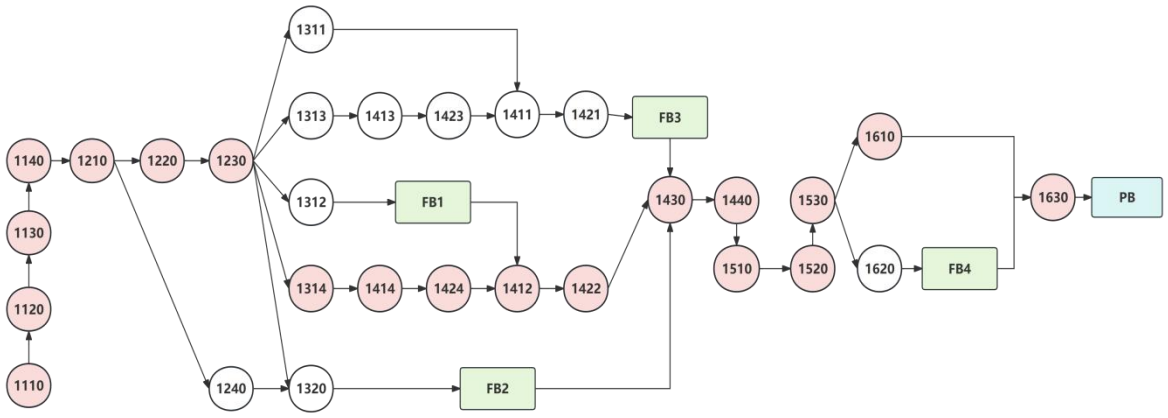


图 4.15 加入缓冲区后的业务应用运行平台单代号网络图

（2）缓冲区参数及计算

影响项目的外部因素包括工序复杂度，而内部因素涉及工序完工弹性系数和工序重要性系数的计算[50]。

① 工序复杂度（S_i）

工序复杂度是核心内部影响因素。紧前工序越多、距线路开端越久，工序受其他作业影响越强，周期波动与延迟风险越大，所需基础缓冲时间也越长。因此在项目缓冲区设置中引入了工序复杂度 S，公式如下：

$$S_i=\frac{n_p}{n_q}+\frac{t_p}{t_q} \tag{4.5}$$

式中，n_p表示第 i 道工序的紧前工序数量。紧前工序越多，说明该工序对前置作业的依赖度越高，受其他作业波动的影响越大，延迟风险也越高。n_q表示为工序 i 所在线路上的工序总数。t_p表示第 i 道工序距项目线路开端的累计时间。这个时间越长，说明该工序在项目路径上的位置越靠后，前面工序的波动累积影响越大，延迟的可能

性越高。t_q表示工序 i 所在线路上的总持续时间。整理项目中每项工作对应的四个参数，计算工序复杂度如下表所示。

表 4.12 业务应用运行平台各项目工序复杂度

WBS 编码	任务名称	n_p	n_q	t_p	t_q	S
1110	需求调研	–	–	–	–	–
1120	项目规划	1	19	20	160	0.18
1130	项目分析及可行性报告	1	19	24	160	0.20
1140	项目立项及审批	1	19	31	160	0.25
1210	编写 PRD 文档	1	19	39	160	0.30
1220	技术选型	1	19	51	160	0.37
1230	技术方案设计	1	19	55	160	0.40
1240	UI 设计	1	14	51	141	0.43
1311	表单模块开发	1	19	64	153	0.47
1312	工作流模块开发	1	17	64	160	0.46
1313	数据采集模块开发	1	19	64	160	0.45
1314	第三方接口适配开发	1	17	64	153	0.48
1320	前端页面开发	2	14	64	141	0.60
1411	表单模块编写测试用例	1	19	88	153	0.63
1412	工作流模块编写测试用例	2	19	103	160	0.75
1413	数据采集模块编写测试用例	1	19	93	160	0.63
1414	三方接口编写测试用例	2	19	98	153	0.75
1421	表单模块测试	1	19	91	153	0.65
1422	工作流模块测试	1	19	108	160	0.73
1423	数据采集模块测试	1	19	95	160	0.65
1424	三方接口集成测试	1	19	102	153	0.72
1430	联合测试	3	19	116	160	0.88
1440	性能及安全测试	1	19	123	160	0.82
1510	生产环境部署	1	19	127	160	0.85
1520	灰度上线	1	19	134	160	0.89
1530	全量发布	1	19	142	160	0.94
1610	验收检验	1	19	146	160	0.97
1620	准备交付物	1	19	146	158	0.98
1630	资产交付	2	19	156	160	1.08

② 弹性系数（K_i）

弹性系数法是基于计划评审技术时间估算法，对各工序按时完工的可能性进行度量，进而估算该工序所需的缓冲区大小。这里的弹性系数反映了工序时间的不确定性或可变性，以及这种不确定性对缓冲区大小的影响。其计算公式如下：

$$K_i=\frac{T_m-T_o}{T_p-T_o} \tag{4.6}$$

通过计算活动最可能完工时间（T_m）与活动最乐观估算时间（T_o）的差值以及活动最悲观估算时间（T_p）与活动最乐观估算时间（T_o）的差值，然后将这两个差值进行比较，得出风险弹性系数。通过对这个系数 K 进行全面测算，可以有效吸收项目中的不确定性。项目专家组根据消除安全时间后的项目工期重新评估最乐观估算时间以及最悲观估算时间，风险弹性系数如表 4.13 所示。

表 4.13 业务应用运行平台弹性系数表

WBS 编码	任务名称	T_o	T_m	T_p	K
1110	需求调研	-	-	-	-
1120	项目规划	2	4	8	0.36
1130	项目分析及可行性报告	3	7	9	0.64
1140	项目立项及审批	4	8	12	0.50
1210	编写 PRD 文档	9	12	16	0.43
1220	技术选型	2	4	8	0.36
1230	技术方案设计	4	9	12	0.63
1240	UI 设计	7	13	15	0.71
1311	表单模块开发	17	24	31	0.50
1312	工作流模块开发	30	39	45	0.59
1313	数据采集模块开发	19	29	36	0.59
1314	第三方接口适配开发	25	33	37	0.68
1320	前端页面开发	26	33	39	0.53
1411	表单模块编写测试用例	1	3	5	0.50
1412	工作流模块编写测试用例	2	5	7	0.60
1413	数据采集模块编写测试用例	1	2	3	0.50
1414	三方接口编写测试用例	2	4	5	0.67

表 4.13（续） 业务应用运行平台弹性系数表

WBS 编码	任务名称	T_o	T_m	T_p	K
1421	表单模块测试	3	7	9	0.64
1422	工作流模块测试	5	8	11	0.53
1423	数据采集模块测试	5	8	10	0.57
1424	三方接口集成测试	4	7	10	0.42
1430	联合测试	3	7	9	0.61
1440	性能及安全测试	2	4	7	0.43
1510	生产环境部署	4	7	10	0.44
1520	灰度上线	4	8	10	0.67
1530	全量发布	2	4	7	0.30
1610	验收检验	7	10	12	0.60
1620	准备交付物	6	8	12	0.33
1630	资产交付	2	4	6	0.38

③ 活动重要性系数（Z_i）

活动重要性系数是量化工序对项目总工期、关键路径或核心交付目标影响程度的权重指标，用于决定缓冲时间的分配优先级。活动重要性越高，其延误对项目整体的影响越大，需分配的缓冲时间也越多。公式如下：

$$Z_i=w_1\cdot C_i+w_2\cdot B_i+w_3\cdot F_i \tag{4.7}$$

式中各参数的具体定义、取值规则及量化标准已在 4.3.4 小节详细阐述，此处不再赘述。

根据以上三个参数，可以计算出缓冲区大小，公式如下：

$$PB=\sum_{m\in B}(s_m+k_m)\cdot z_m\cdot\frac{t_m-a_m}{2} \quad (4.8)$$

$$FB=\sum_{n\in C}(s_n+k_n)\cdot z_n\cdot\frac{t_n-a_n}{2} \quad (4.9)$$

式中，PB 为项目缓冲，放在关键链的末端。FB 为接驳缓冲，放在非关键链与关键链的交汇点；B 和 C 分别代表关键链和非关键链的活动；S 表示工序复杂度；K 表示活动的弹性系数；Z 表示活动重要性系数。t 为通过三点估算法得出的期望时间，a 为消除安全时间后的工期。根据前文计算出的参数，整理所有活动的缓冲区，在按照是否在关键链上区分项目缓冲和接驳缓冲，具体参数见下表 4.14。

表 4.14 缓冲区计算参数表

WBS 编码	任务名称	s	k	*z*	*t*	*a*	缓冲区大小
1110	需求调研	-	-	-	-	-	-
1120	项目规划	0.18	0.36	0.84	5	4	0.23
1130	项目分析及可行性报告	0.20	0.64	0.84	8	7	0.35
1140	项目立项及审批	0.25	0.50	0.92	10	8	0.69
1210	编写 PRD 文档	0.30	0.43	1	15	12	1.09
1220	技术选型	0.37	0.36	0.92	6	4	0.67
1230	技术方案设计	0.40	0.63	0.84	10	9	0.43
1240	UI 设计	0.43	0.71	0.6	15	13	0.68
1311	表单模块开发	0.47	0.50	0.8	26	24	0.78
1312	工作流模块开发	0.46	0.59	1	42	39	1.57
1313	数据采集模块开发	0.45	0.59	0.8	32	30	1.18
1314	第三方接口适配开发	0.48	0.68	1	36	33	1.74
1320	前端页面开发	0.60	0.53	0.8	35	33	0.90

表 4.14（续） 缓冲区计算参数表

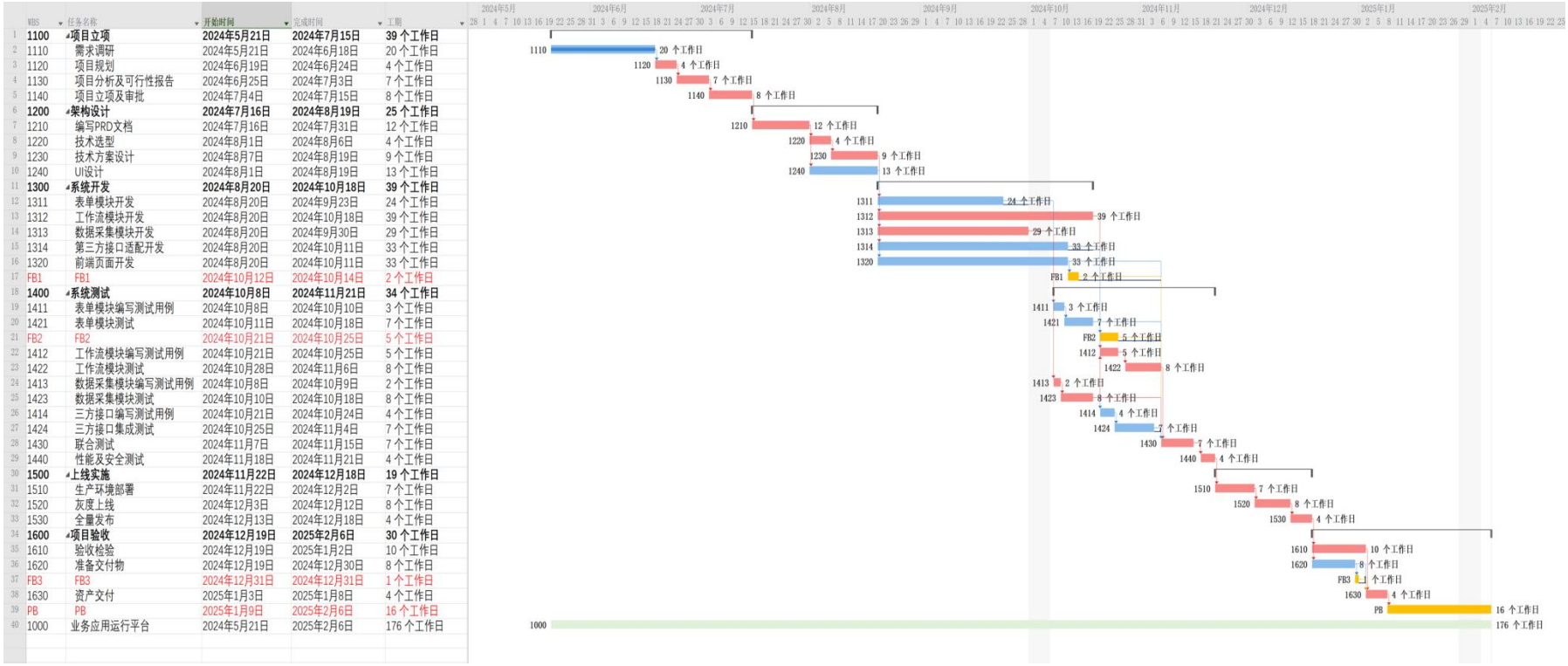
WBS 编码	任务名称	s	k	*z*	*t*	*a*	缓冲区大小
1411	表单模块编写测试用例	0.63	0.50	0.44	4	3	0.25
1412	工作流模块编写测试用例	0.75	0.60	0.64	6	5	0.43
1413	数据采集模块编写测试用例	0.63	0.50	0.44	3	2	0.25
1414	三方接口编写测试用例	0.75	0.67	0.64	5	4	0.45
1421	表单模块测试	0.65	0.64	0.44	8	7	0.28
1422	工作流模块测试	0.73	0.53	0.72	10	8	0.90
1423	数据采集模块测试	0.65	0.57	0.52	9	8	0.32
1424	三方接口集成测试	0.72	0.42	0.8	9	7	0.91
1430	联合测试	0.88	0.61	0.92	9	7	1.37
1440	性能及安全测试	0.82	0.43	0.84	5	4	0.53
1510	生产环境部署	0.85	0.44	0.92	9	7	1.19
1520	灰度上线	0.89	0.67	0.92	10	8	1.43
1530	全量发布	0.94	0.30	0.92	6	4	1.14
1610	验收检验	0.97	0.60	0.92	12	10	1.44
1620	准备交付物	0.98	0.33	0.72	10	8	0.94
1630	资产交付	1.08	0.38	0.92	6	4	1.34

根据上表的结果，计算关键链路上活动缓冲区大小。项目缓冲（PB）计算结果为 16.55，向上取整为 17 个工作日。再根据图 4.13，计算每条非关键链路上缓冲区大小总额，整理不同接驳缓冲对应的非关键节点，求和后把结果向上取整得到最终 FB，结果如下表所示。

表 4.15 接驳缓冲统计表

缓冲编号	前置节点	包含的非关键节点	FB	最终 FB
FB1	1320	1240、1320	1.58	2
FB2	1421	1311、1313、1413、1423、1411、1421	4.40	5
FB3	1620	1620	0.94	1

将项目缓冲、接驳缓冲配置至 Microsoft Project 软件对应节点后，业务应用运行平台项目的关键链路未发生变化，项目最终进度计划图如图 4.16 所示。由图可知，项目开始时间为 2024 年 5 月 21 日，结束时间为 2025 年 2 月 6 日，共 176 个工作日。不仅满足甲方工期要求，还通过项目缓冲与接驳缓冲的分层配置，隔离非关键节点波动干扰，降低工期延误风险。关键链法结合工期管控与资源优化，提升了进度计划的实操性和抗风险能力，提高了工期管控精准度与资源利用效率。



	WBS	任务名称	开始时间	完成时间	工期
1	1100	项目立项	2024年5月21日	2024年7月15日	39 个工作日
2	1110	需求调研	2024年5月21日	2024年6月18日	20 个工作日
3	1120	项目规划	2024年6月19日	2024年6月24日	4 个工作日
4	1130	项目分析及可行性报告	2024年6月25日	2024年7月3日	7 个工作日
5	1140	项目立项及审批	2024年7月4日	2024年7月15日	8 个工作日
6	1200	架构设计	2024年7月16日	2024年8月19日	25 个工作日
7	1210	编写PRD文档	2024年7月16日	2024年7月31日	12 个工作日
8	1220	技术选型	2024年8月1日	2024年8月6日	4 个工作日
9	1230	技术方案设计	2024年8月7日	2024年8月19日	9 个工作日
10	1240	UI设计	2024年8月1日	2024年8月19日	13 个工作日
11	1300	系统开发	2024年8月20日	2024年10月18日	39 个工作日
12	1311	表单模块开发	2024年8月20日	2024年9月23日	24 个工作日
13	1312	工作流模块开发	2024年8月20日	2024年10月18日	39 个工作日
14	1313	数据采集模块开发	2024年8月20日	2024年9月30日	29 个工作日
15	1314	第三方接口适配开发	2024年8月20日	2024年10月11日	33 个工作日
16	1320	前端页面开发	2024年8月20日	2024年10月11日	33 个工作日
17	FB1	FB1	2024年10月12日	2024年10月14日	2 个工作日
18	1400	系统测试	2024年10月8日	2024年11月21日	34 个工作日
19	1411	表单模块编写测试用例	2024年10月8日	2024年10月10日	3 个工作日
20	1421	表单模块测试	2024年10月11日	2024年10月18日	7 个工作日
21	FB2	FB2	2024年10月21日	2024年10月25日	5 个工作日
22	1412	工作流模块编写测试用例	2024年10月21日	2024年10月25日	5 个工作日
23	1422	工作流模块测试	2024年10月28日	2024年11月6日	8 个工作日
24	1413	数据采集模块编写测试用例	2024年10月8日	2024年10月9日	2 个工作日
25	1423	数据采集模块测试	2024年10月10日	2024年10月18日	8 个工作日
26	1414	三方接口编写测试用例	2024年10月21日	2024年10月24日	4 个工作日
27	1424	三方接口集成测试	2024年10月25日	2024年11月4日	7 个工作日
28	1430	联合测试	2024年11月7日	2024年11月15日	7 个工作日
29	1440	性能及安全测试	2024年11月18日	2024年11月21日	4 个工作日
30	1500	上线实施	2024年11月22日	2024年12月18日	19 个工作日
31	1510	生产环境部署	2024年11月22日	2024年12月2日	7 个工作日
32	1520	灰度上线	2024年12月3日	2024年12月12日	8 个工作日
33	1530	全量发布	2024年12月13日	2024年12月18日	4 个工作日
34	1600	项目验收	2024年12月19日	2025年2月6日	30 个工作日
35	1610	验收检验	2024年12月19日	2025年1月2日	10 个工作日
36	1620	准备交付物	2024年12月19日	2024年12月30日	8 个工作日
37	FB3	FB3	2024年12月31日	2024年12月31日	1 个工作日
38	1630	资产交付	2025年1月3日	2025年1月8日	4 个工作日
39	PB	PB	2025年1月9日	2025年2月6日	16 个工作日
40	1000	业务应用运行平台	2024年5月21日	2025年2月6日	176 个工作日

图 4.16 接入缓冲区的业务应用运行平台项目进度计划甘特图

4.4 本章小结

本章针对 C 公司业务应用运行平台项目进度控制存在的问题，采用关键路径法（CPM）和关键链法（CCM）依次对项目进度进行优化和资源优化，在一定程度上克服了项目初期进度拖延、资源冲突、过程繁琐及安全时间消耗等问题，得出了适用于 C 公司业务应用运行平台项目的进度控制方法。

首先，根据项目 WBS 分解结果，梳理关键活动、确定逻辑关系，各活动的负责人、紧前紧后逻辑关系，形成活动排序表及绘制单代号网络图；接着采用 PERT 三点估算法进行活动工期估计，识别初步的关键路径以及总工期，发现该工期大于甲方要求的交付时间，并且还有任务划分粗略、资源不均衡等问题，为下一步改进提供了目标。

其次，基于关键路径法 CPM 开展进度优化。一是进度逻辑优化，针对测试环节串行执行导致的等待问题，将编写测试用例按开发模块拆分为 4 个子任务，调整为开发完成一个模块、启动一个模块的并行模式；二是核心流程优化，针对 PRD 文档编写、验收检验两个关键活动的审批链路冗长问题，合并内部评审环节、前置甲方协同参与，精简评审次数与流程节点。优化后，项目关键路径略有调整，总工期压缩，有效缓解了工期压力。

再次，引入关键链法弥补关键路径法的不足，开展资源优化。先完成项目资源分配与约束分析，明确项目核心人力资源的配置情况，识别出测试资源无法支撑 4 个模块并行测试的核心资源瓶颈。通过量化关键路径属性、任务风险程度、任务时长三个维度，对受资源约束的测试类活动进行优先级排序，设计纯优先级导向与结合实际场景的两种资源分配方案，经对比筛选出最优方案，实现测试资源的高效利用，避免资源闲置与过载。

最后，完成安全时间优化与缓冲区设置，构建项目风险抵御体系。针对各活动分散预留安全时间导致的工期浪费问题，采用量化公式计算各活动的合理安全时间，消除无效冗余，调整后各活动工期更贴合实际执行需求；基于内外部影响因素，计算并设置项目缓冲与 3 处接驳缓冲，将分散的安全时间整合为集中缓冲，分别用于抵御关键链活动风险、阻断非关键链延误向关键链传导，最终绘制接入缓冲区后的项目进度计划甘特图，确定项目最终工期为 176 个工作日，完全满足甲方交付要求。

5 项目进度保障、监控措施与效果评价

虽然结合第四章的方法制定并优化了项目计划，但是在项目实际开展过程中或多或少会出现难以预见的情况影响项目进度。所以还需要制定合理有效的保障机制来支撑进度计划，以应对突发情况及风险，及时解决影响项目进度的问题。本章通过为项目增加保障措施，优化项目制度的方法来对项目进行优化，同时用三色图的方法来监控缓冲区消耗，进而控制项目进度。最后通过对比优化方案实施前后项目进度管理的区别，对 C 公司业务应用运行平台项目进度管理优化方案进行效果分析与评价。

5.1 项目进度保障措施

5.1.1 建立需求变更机制

需求变更机制通过标准化流程、明确分工及信息同步，实现需求变更可追溯、可管控、可优化，专门解决前文提及的沟通不健全、跨团队信息断层问题。它是保障业务应用运行平台项目进度、资源、质量稳定的核心，可避免无序变更引发的返工、工期延误，兼顾迭代灵活性并最大限度降低变更负面影响。本项目在前期进度优化中，已通过关键链法整合了集中化缓冲区，并形成了基于关键路径与资源配置的稳定进度框架。为适配不同程度的需求波动、避免随意变更打乱整体优化进度，本文将项目需求变更划分为一般需求变更与重大需求变更两类实行分级管控。

一般需求变更多指不涉及核心业务架构、不改变业务主线流程的局部功能调整，不会影响项目关键路径与整体资源排布。此类变更只做局部任务工期微调，依托项目预设的集中缓冲区自然消化变更带来的工期波动，无需对前文已形成的整体进度优化方案、关键路径及资源配置框架进行改动。

重大需求变更主要指涉及核心业务重构、模块架构调整、关键任务新增或删减的变更类型，会直接影响原有关键路径走向、资源负荷分配及项目总工期目标。对此类变更需在可行性论证阶段，重点评估其对关键路径、资源配置及总工期的扰动程度；若确认已冲击原有关键路径，则基于变更后的新增活动与调整逻辑，只针对受影响的关键活动及其后续依赖活动，重新开展关键链识别与资源平衡优化，仅做局部工期与资源的定向调整，不推翻、不重新全盘优化整个项目进度体系，最大程度保留前期进度优化成果，降低变更带来的管控成本与工期风险。

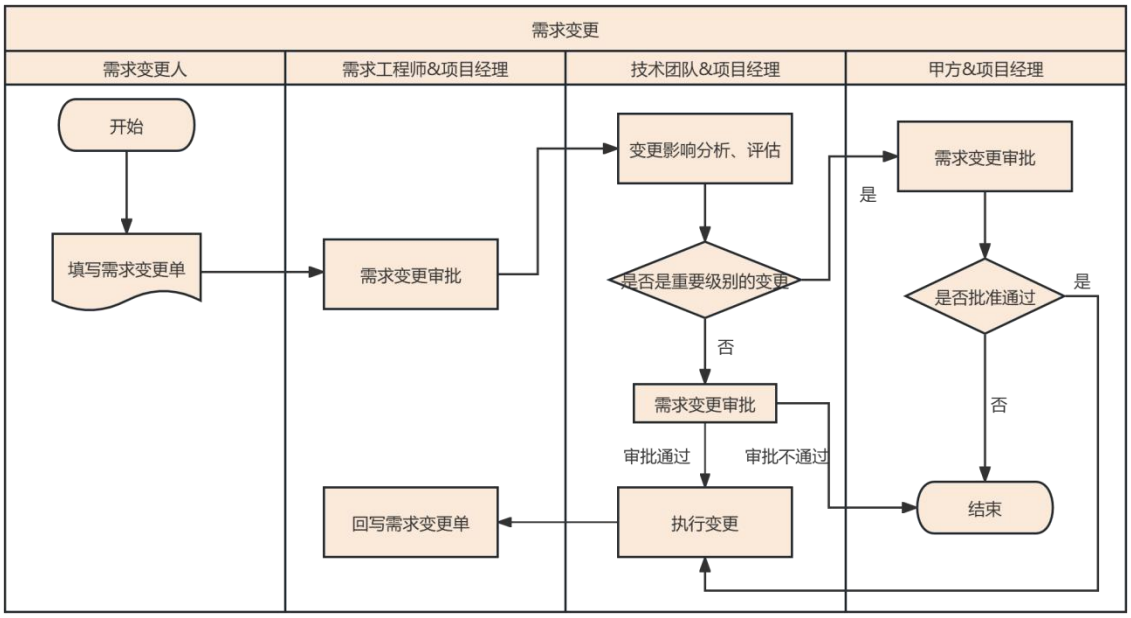


图 5.1 业务应用运行平台需求变更流程图

首先，需求提出人描述需求变更的内容，在企业微信上填写本项目的需求变更单，包括变更内容、申请原因及可能带来的后果，并将表格填写完整发送给需求工程师及项目经理，保证信息传达的全面性。接着，项目负责人接收申请后并检查信息正确性，确认无误后提交至技术团队负责人，对于信息不完整的情况，应及时向发起单位提出补充意见，以免出现信息不完整造成流转停滞的现象发生。其次，技术团队负责人需在 1 个工作日内，结合业务应用运行平台项目进度、资源配置情况，论证变更可行性及对项目质量、工期的影响，区分一般需求变更与重大需求变更实行差异化审批：若为复杂度高、对项目前后工作影响较大的重大需求变更，需同步变更信息至甲方及项目经理，由甲方主管完成二次审批；若为一般需求变更且不影响工期及现有功能，技术团队负责人直接审批确认，并将审批结果抄送至甲方备案。所有审批通过后，由技术团队完成该需求变更，变更落地后，由需求工程师在企业微信线上回写需求变更单，完成流程闭环。最后，变更完成后，由评审小组联合发起方校验执行效果，确认符合变更诉求后，将需求变更申请表、评审纪要、执行报告等相关资料统一归档，形成变更知识库，为平台后续迭代提供参考，若未达标，反馈执行团队整改后重新校验。

综上，本小结针对业务应用运行平台项目需求管控的痛点，建立分级分类的规范需求变更机制，通过一般变更靠缓冲区消化、重大变更先局部定向再优化的管控思路

提升了资源利用效率与进度计划抗风险能力，形成科学实操的管控流程，为后续工作奠定基础。

5.1.2 完善沟通交流机制

完善项目沟通交流机制的核心是让不同类型、不同紧急程度的信息都有对应的传递渠道，避免信息孤岛或沟通冗余，同时通过明确的规则和落地动作确保沟通有结果、有跟进。结合 C 公司业务应用运行平台项目的特点，为保障项目高效推进、减少沟通内耗、确保甲乙双方诉求对齐，制定以下标准化、可落地的沟通交流措施，核心以标准化正式会议为骨架，辅以落地保障机制，覆盖项目全周期沟通场景。

（1）每日早会。聚焦项目开发、测试等高密度执行阶段，快速同步当日进度、暴露即时问题，避免小问题累积影响整体进度，确保各执行岗协同一致。会议固定时间为每日 9:30-9:50，参与人固定为项目核心执行岗，具体包括前端开发人员、后端开发人员、测试人员、产品经理、项目负责人，无特殊情况不得缺席，缺席者需提前 1 小时向项目负责人报备，并委托同岗位同事代为同步工作及问题。

会上每位参会人员说明昨日完成的工作，今日准备完成的工作以及遇到的问题。项目负责人全程做好记录，针对提出的问题分类处理：可即时协调解决的当场协调对应责任人，明确解决时限；无法即时解决的，标注为待专项协调，会后 1 小时内将早会纪要同步至项目专属群，同时将问题录入项目管理工具，由项目负责人每日跟踪进度，直至问题闭环。

（2）项目周例会。用于同步本周项目整体进展、协调跨团队/跨岗位问题、确认下周工作计划，确保项目按里程碑推进，同时及时向相关方同步进度，避免信息偏差。会议固定时间为每周五下午 15 点召开，时长控制为 1 个小时。参会人基础为项目负责人、各团队负责人、产品经理，特殊场景由项目负责人邀请甲方对接人、公司相关部门负责人参与；会议前 1 天，项目负责人需制定详细会议议程，明确各环节发言时长、发言内容，同时收集各团队需协调的跨团队问题，提前同步给相关参会人，确保会上高效沟通。

会上项目负责人汇报本周整体进展，包含完成的工作和未完成的工作及理由，若存在项目进度偏差，说明已采取的补救措施；各团队负责人分别介绍本团队本周核心工作完成情况、未完成工作的推进计划，重点提出项目过程中遇到的跨团队/跨部门问题，由相关团队负责人当场回应，明确协助方案，最后按照进度要求确认下周核心

工作。如果在项目过程中出现重要变动，项目负责人同步变动内容、对项目的影响，邀请甲方确认意见，达成一致后明确执行方案。会议结束后，项目负责人整理周例会纪要并同步至项目群和公司知识库，纪要中所有待办项录入项目管理工具，项目经理每日跟踪进度。

（3）项目月例会。聚焦项目长期目标，复盘月度里程碑进度、识别项目整体风险、协调跨部门核心资源、规划下月整体工作，确保项目方向不偏离，同时向公司管理层、甲方领导同步项目整体状态。会议固定时间为每月最后一个工作日下午15:00-16:30，时长控制在 1.5 小时内；参会人员固定为项目负责人、各团队负责人、甲方领导，不邀请基层执行岗；会议前 3 天，项目负责人需整理月度项目月报同步至所有参会人员，方便参会人员提前了解项目状态。

会议汇报本月里程碑完成情况，展示进度数据。同时各团队负责人汇报本月发现的重大风险，项目负责人汇总梳理，评估风险等级，并确定风险应对方案、责任人以及管控时限，同步到甲方及公司管理层；对于在月度项目推进过程中遇到的跨部门资源卡点，项目负责人向公司管理层、相关部门负责人提出需求，落实好资源调配安排以及使用的时间，保证下月的工作能够顺利推进。结合项目总目标及本月进度偏差，项目负责人汇总各团队下月核心工作规划，明确下月里程碑目标、工作优先级、资源分配方案，邀请甲方领导、公司管理层提出意见，确认后落地执行。甲方领导同步自身诉求，项目负责人同步项目推进中的困难及需求，达成共识，避免后续协作卡点。会议结束后，项目负责人需在次月 1 个工作日内，发布月例会纪要及项目月报，纪要及月报同步至项目群、公司管理层、甲方相关人员，同时将月度风险项、跨部门资源协调事项录入项目管理工具，由项目负责人全程跟踪，直至闭环。

除了例会，针对项目推进中出现的风险和异常，项目负责人需要快速响应，避免风险扩大影响项目进度或质量。例如，核心技术出现瓶颈，导致项目关键功能无法推进、进度落后超计划 5 天以上，或者项目核心人员突发缺席影响核心工作推进等。出现这种场景后，项目负责人需在 2 小时内发起会议。参会人员根据风险类型确定，核心包括项目负责人、相关团队负责人、核心执行人员，必要时邀请公司管理层、甲方领导、第三方对接人参与，会议主要聚焦于问题原因、应对方案、确定责任人和解决期限，不做无关讨论。例如在 C 公司业务应用运行平台开发阶段，核心后端开发骨干突发离职，项目负责人立即发起风险预警专项会议，参会人员包括项目负责人、测试负责人、剩余后端开发人员、技术架构师，会上明确应对方案：由技术架构师临时接

手核心代码梳理工作，剩余后端开发人员分工衔接，同时启动紧急招聘补充人员，项目负责人同步该情况至甲方及公司管理层，甲方确认同意后，按方案推进，避免项目进度大幅滞后。

在会议举行后，还需要保证会议沟通真正落地，不可制订了不执行。需要指定项目责任人为沟通第一责任人，负责跟踪所有会议的沟通结果以及进行信息归档。并且将沟通及成果落地纳入项目组成员的日常考核，明确考核标准，占个人项目绩效的20%，倒逼全员重视沟通、落实沟通结果。

完善的项目沟通交流机制，本质是让合适的信息，在合适的时间，通过合适的渠道，传递给合适的人，并有明确的结果和跟进。针对 C 公司业务应用运行平台项目，核心以标准化正式会议为骨架，解决核心决策、进度同步、风险处置等问题，同时通过明确责任人、考核绑定、定期复盘等保障措施，确保沟通机制落地执行，避免形式化。最终实现减少沟通内耗、对齐甲乙双方诉求、管控项目风险、保障项目按里程碑推进的目标，为项目顺利交付提供有力支撑。

5.1.3 完善绩效考核、激励与晋升制度

结合 C 公司业务应用运行平台项目建设情况，采用 KPI 核心指标+360 度综合评价+OKR 的目标管理方式，把沟通交流、工作完成、结果产出、协同配合作为考核的核心，并辅以明确的奖惩及职级上升机制，人人强化落实，确保项目顺利开展。其中 KPI 占比为 60%，围绕项目进度节点达成率、任务工期偏差控制、交付质量、工作执行效率等进度相关核心指标设定，把个人工作完成时效、关键任务按期交付情况直接纳入量化考核，实现进度表现可量化、可打分、可追责；360 度综合评价占比为 20%，包括项目负责人对本人及同岗位其他成员、不同岗位合作成员的评价，侧重于沟通协调、责任心、执行力度等方面的定性评价；OKR（目标和关键结果）占比为 20%，每月制定个性化的具体目标，侧重于自我能力提高以及对项目进度瓶颈、技术难点、资源协调卡点制定专项攻坚目标，保障整体进度平稳受控。

在绩效考核方面，从月度初评，到季度总评，项目结束后再完成项目终评，最终的考核结果分为优秀、良好、合格、不合格四个等级。月度初评为每月最后一个工作日，由员工自报当月的工作总结以及 OKR 的完成情况，项目主管综合 KPI 数据、各月度计划节点完成情况以及平时表现进行打分并附上评语，表扬优点、指出缺点，并提出个性化的改进方案；季度总评为每个季度结束的时候，综合三个月的月度初评，

开展 360 度考核。综合得到季度总评结果，并推送至本人和人力资源部门；项目终评于项目交付 1 月后开展，根据项目总体情况、个人在项目全流程中的表现，给出项目终评结果，作为激励、晋升的核心依据。

在员工激励方面，可以在项目过程中设置阶段性奖励，对于月度初评或者季度总评为优秀的员工，按照个人月薪的 20%发放月度和季度绩效奖金，对于良好的员工发放 10%的绩效奖金。并在季度总结会上通报表扬。在完成里程碑工作后，全员发放里程碑奖金，可按岗位层级划分奖金，如项目负责人 4000 元、团队负责人 2000 元、基层执行岗 1000 元。对年度内 2 次及以上季度总评为优秀，或项目终评优秀的员工，发放项目奖金，比例为个人月薪的 5%，并优先获得公司内部技术培训、外部行业交流、技能认证等机会。同样除了正向奖励之外，也需要制定惩罚措施，对于月度初评或者季度总评为不合格的项目成员，扣除季度绩效 10%，并由项目负责人一对一谈话制定改进措施，次月重新考核，连续两个月不合格，则暂停项目资格并取消季度奖金。对于沟通不及时、隐瞒问题、执行力不足等问题导致项目进度滞后、出现风险、甲方投诉，扣除对应绩效分数，情节严重的取消季度奖金，通报批评。

在晋升通道层面，结合项目岗位特点，建立团队管理岗位以及专业技术岗位双晋级通道，并将项目进度管控能力、里程碑贡献度、工期保障成效作为晋升评审的核心硬性条件。管理岗通道为：基层执行岗、团队负责人、项目负责人、项目总监。专业技术岗通道为：基层专业岗、中级专业岗、高级专业岗、资深专家。对于在项目中表现突出，具备岗位所需能力且能承担核心职责的员工，可以申请晋升，或者由项目负责人推荐，提交个人绩效考核报告和项目工作总结。由人力部联合公司管理层进行业绩评审，能力面试。评审通过后，在公司内部进行公示，无异议后，正式确定晋升岗位及薪资调整方案。在晋升后设置 1-3 个月试用期，试用期内按新岗位考核标准进行考核，考核合格后，正式任职；考核不合格的，调整回原岗位，6 个月内不得再次申请晋升。

5.2 项目进度监控

5.2.1 缓冲区监控三色图

本项目采用三色监控法开展缓冲区动态监控，原理是通过绿、黄、红三色直观呈现项目执行中的缓冲剩余情况，如图 5.2 所示[51]。图中斜线为进度缓冲匹配临界线，其临界值逻辑为“项目进度推进至 X%时，缓冲区消耗的合理临界值为 X%”，并以

此划分为 3 个风险管控区域：绿色区域对应进度良好，项目进度推进速度快于缓冲区消耗速度，缓冲储备消耗少、项目风险极低；黄色区域对应持续监控状态，在临界线与红色区域之间，缓冲区消耗速度略快于项目进度，需要不断观察防止消耗增大；红色表示缓冲区消耗过大，在临界线以上，缓冲区消耗远远大于项目进展，缓冲储备即将被消耗完时应采取应急对策。以图中红色临界点为例，当业务应用运行平台项目进度推进至 60%时，缓冲区消耗达 80%，超过临界值进入红色区域，项目后续需重点关注进度影响与缓冲区消耗情况。

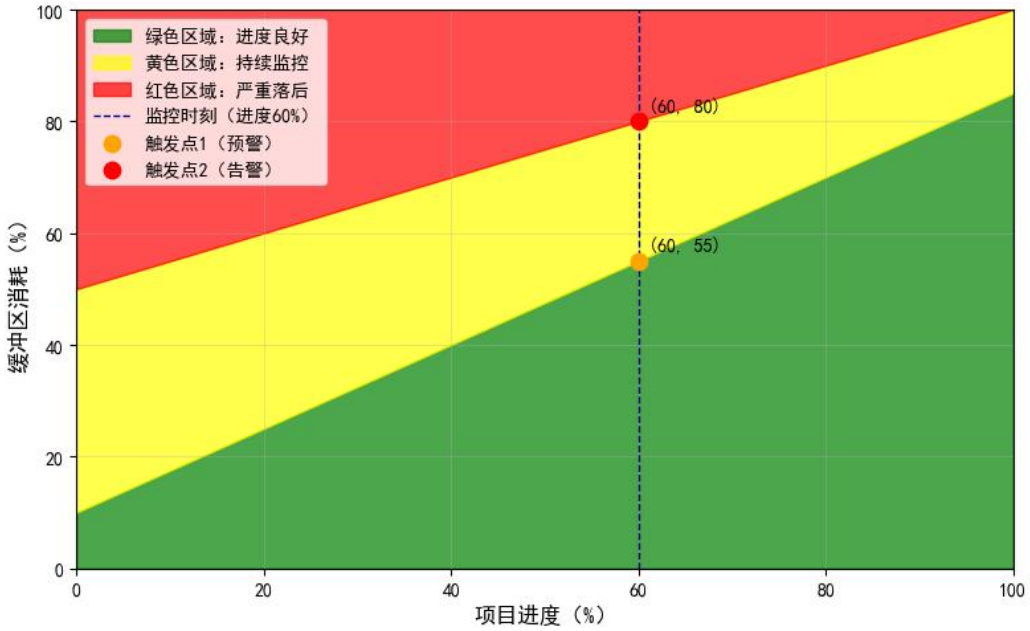


图 5.2 业务应用运行平台缓冲区监控图

5.2.2 缓冲区监控状态及分析

为弥补单一三色图颗粒度较粗、无法详细记录实时进度与纠偏情况的缺陷，本文在三色预警基础上，配套设计里程碑监控表，通过三色图和里程碑进度监控表的组合模式，实现进度监控的动态闭环。结合业务应用运行平台项目的核心里程碑事件，确定 6 个关键监控节点，分别为项目立项及审批、技术方案设计、业务接口开发、联合测试、全量发布、系统验收。在项目实施过程中，收集项目运行数据，进行监控点进度百分比、缓冲区消耗比例计算以及监控点区域判断，各监控点的缓冲区监控状态信息如下表所示。

表 5.1 缓冲区状态监控表

序号	监控点	完成时间	进度完成比例	消耗比例	绿色区域	黄色区域	红色区域
1	项目立项及审批	2024/7/15	22%	0%	√		
2	技术方案设计	2024/8/19	36%	30%	√		
3	业务接口开发	2024/10/18	58%	55%	√		
4	系统测试	2024/11/21	77%	80%		√	
5	全量发布	2024/12/18	88%	88%		√	
6	系统验收	2025/2/6	100%	88%		√	

在技术方案设计阶段的技术选型中，原定计划采用的开源工作流组件与甲方现有业务系统存在兼容性冲突，需要增加相关技术人员对同类开源组件进行调研分析，并确认是否能与甲方已有系统兼容，同时还要对整体方案进行相应的调整，因此延误了本环节进度，消耗了少量缓冲时间。

在业务接口开发阶段，负责核心表单模块接口开发的工程师突发急性肠胃炎，请假 3 个工作日，且该模块接口开发需连续推进无法临时交接，导致开发进度滞后，为保障后续测试环节按计划启动，动用缓冲弥补工期缺口。

在系统测试阶段，测试过程中发现第三方接口适配模块存在隐藏逻辑漏洞，该漏洞需联动第三方技术支持团队定位问题根源，且修复后需重新开展全量接口联调测试，额外耗时导致缓冲进一步消耗。

5.3 项目效果评价

本次业务应用运行平台项目通过第四章进度优化方案与第五章保障监控机制的协同落地，从任务管理、工期评估、资源配置、沟通监控四大核心维度实现系统性改进，解决了 C 公司历史项目中存在的进度管理痛点，还保证了成本控制与质量保障，同时验证了企业内部管理基础、部门协作效率、客户配合程度对优化效果的关键支撑作用，具体效果如下：

（1）任务分解精细化

C 公司以往的历史项目任务分解都是比较粗放式的，只分解到关键点，并没有按照业务应用平台的模块来进一步细分，导致其下级的任务界限不清晰，也没有分配到

具体的人上，容易出现任务漏做或者重复做的情况。根据 C 公司以往的经验，在项目中会有 10%左右的工作是重复进行的，平均下来会遗漏 2-3 项系统间的接口任务。这类问题不仅直接延误项目进度，更造成人力资源浪费与返工成本增加。

本业务应用运行平台项目采用 WBS 分层拆解模式，结合平台核心功能，将总任务拆解至三级子任务，明确各子任务的交付物、完成标准及责任人，例如将开发阶段拆解为核心接口开发、前端页面搭建、模块联调等细分任务，避免以往项目中存在的工作任务管理不严的问题，提高了任务开展的条理性。经优化后，项目任务颗粒度清晰度显著提升，任务重复执行率由 10%降至 2%以内，跨系统接口任务实现 100%覆盖、零遗漏，所有细分任务责任人落实率、交付物明确率均明显提高。这一改进不仅提升了任务开展的条理性，更直接减少了无效返工成本，经核算，因任务冗余与遗漏导致的成本损耗降低约 60%，同时为后续成本预算的精准管控奠定了基础。

（2）任务评估体系健全完善

C 公司历史项目的任务时长评估多依赖经验判断，任务工时预估偏差率平均达 35%，其中新增需求的工时预估偏差尤为严重。这种偏差要么造成资源冗余浪费，要么因时长不足频繁动用缓冲，引发赶工压力，赶工过程中易出现代码质量下降、测试不充分等问题，后期质量整改成本平均增加 20%。

本项目在工期时长评估阶段，结合 C 公司历史同类项目的时长数据，针对平台各模块的技术难度、风险点开展多轮论证，同时根据科学计算预留合理缓冲时长，并建立时长动态调整机制。优化后，项目任务工期预估偏差率由 35%降至 10%以内，缓冲资源无效消耗减少约 45%，需求变更引发的工期调整响应效率提升 40%以上，精准的工期评估不仅避免了资源闲置浪费，更杜绝了因赶工导致的质量隐患，实现了 “工期精准、成本可控、质量达标” 的三重目标。

（3）资源分配机制科学合理

C 公司历史项目的资源分配采用平均分配模式，未结合项目的任务优先级以及是否适合当前项目而分配人员、硬件资源，导致核心模块与次要模块占用同等资源，这种分配方式不仅降低了整体进度推进效率，更造成核心模块因资源不足进度滞后，资源投入产出比偏低。

本项目结合业务应用运行平台的任务重要性进行资源分配，将核心技术人员配置到核心模块开发、接口联调等任务中，运维以及辅助人员配置到部署、验收等任务中，并且按照任务进展程度与任务难易程度对任务执行过程中的资源比例进行调配，在核

心接口联调的重要时间节点上，合理安排关键人员工作时间，杜绝空闲；降低对非关键部署过程中的人力资源需求，把多余人力资源分配给其他如接口对接等工作上，做到人尽其用，提高人力资源利用率。优化后，核心技术人员工时利用率由不足 60%提升至 85%，次要任务资源占用比例降低 30%，项目整体人力资源利用率提升约 20%；关键节点资源保障率达到 100%，显著增强资源对项目进度的支撑能力。

（4）项目团队沟通机制健全顺畅，监控体系完善落地

C 公司以往项目没有严格的沟通流程，以周报和月报讨论为主，并偶尔进行消息更新，在不同团队之间不能有效沟通，容易造成信息孤岛的现象；同时缺乏有效的监督制度，只能手动检查进度是否落后于计划，因此不能在缓冲被耗尽以及工作落后之前采取行动来解决这些问题，往往是在风险发生之后再采取措施。

本项目针对性地进行了沟通及监控方面的改善。在沟通方面，基于企业微信搭建沟通平台，制定早会、周会以及月会制度。标准化需求变更、进度汇报等沟通机制，实现甲方、开发方、测试方等多方位的信息共享，改变了过往项目中出现的信息孤岛现象；在监控方面，采用三色监控方法将项目进度与缓冲消耗联系起来，借助缓冲区监控表来直观地反映项目的整体状态。建立多个监控点，随时监控任务进展和缓冲消耗的情况，并提前预警。优化后，在此项目中没有出现进度滞后的情况，缓冲预警提前量可达到 2-3 天，较之以前的历史项目大大提高了项目管控准确性和风险控制水平。

（4）管理理层面支撑保障

与 C 公司过往项目相比，本次业务应用运行平台项目在管理层面有三项重点支撑保障。首先，业务管理制度升级，以往项目常出现职责交叉、流程模糊的问题，而本项目构建起权责分明的管理体系，且通过数字化管理平台实现项目进度实时监控、资源动态调配与风险智能预警，流程审批效率大幅提升；其次，协同机制全面升级：过去部门间存在信息壁垒与资源争夺现象，此次采用跨部门联合办公机制，通过动态资源池实现人、财、物的智能调配，使跨部门协作响应时间缩短；最后，客户管理模式升级：传统项目往往被动接受客户需求变更，导致进度失控，本项目通过建立需求周评审制度与客户联合决策小组，将需求变更频率降低 65%，从源头保障了项目进度的稳定性。这些管理创新使本项目不仅突破了历史进度管理瓶颈，更实现了进度、成本、质量的协同提升。

5.4 本章小结

本章围绕业务应用运行平台项目进度管控，构建保障机制、完善监控体系并开展效果评价。建立了需求变更机制，完善沟通机制与绩效激励晋升体系，通过标准化会议打通信息壁垒，通过考核提升团队执行与协同效率。同时，采用三色监控法搭建缓冲区监控体系，跟踪项目进度与缓冲消耗匹配情况，预警风险。效果评价显示，本章措施解决了 C 公司历史项目任务分解粗糙、评估有缺陷、资源分配不合理、沟通监控缺位等问题，提升了项目管控与资源利用效率，项目无工期延误、缓冲未耗尽，验证了进度优化方案的可行性与有效性，为项目顺利交付及后续同类项目管控提供了实践支撑。

结论

本文以C公司业务应用运行平台为研究对象，围绕传统企业在数字化转型推进过程中对项目按期交付、稳定落地要求下的软件项目进度管理问题，开展了系统的分析、优化与应用研究。通过文献研究、项目现场调研与进度偏差分析相结合的方法，深入剖析了C公司该平台项目在需求分析阶段出现工期滞后、进度管控不足等关键问题，针对性制定了进度纠偏、阶段节点管控与资源协同保障的进度优化方案，并通过实际实施与效果检验，确保项目按期交付并验证了管控措施的可行性与有效性。

首先，本文系统梳理了C公司项目进度管理现状，通过对项目人员问卷调查与数据分析，发现项目存在任务分解粗糙、工期评估偏主观、资源配置不合理、沟通与进度监控缺位等核心问题。

其次，本文以WBS任务分解为基础，梳理活动逻辑、确定负责人，采用PERT三点估算法进行工期估算，找出初始关键路径及工期缺口；再利用CPM优化进度，拆分测试任务实现开发、测试并行，缩短关键活动审批环节，缩短总工期；然后利用关键链法优化资源，展开资源需求分析并进行活动排序，规划出最优的资源计划，解决资源闲余以及超负荷的问题；然后，对分散的安全时间进行整合，得到合理的安全时间和集中缓冲，优化工时冗余，最终把项目总工期定为176个工作日，满足甲方交付要求。

最后，为保障进度优化方案落实到位，配套建立需求变更、分层沟通、绩效激励机制，引入三色监控法跟踪缓冲消耗与进度匹配情况、预警风险。实践证明，以上进度优化操作较好地解决了历史管控痛点，提升了资源利用、项目管控效率，并最终实现项目无工期延误、缓冲未耗尽。充分论证了此次进度优化方案可行性和有效性，是项目顺利完成交付的核心保障。

总体而言，业务应用运行平台项目的顺利交付，不仅为C公司补齐了大型复杂软件项目的进度管理短板，形成了可复用的项目管控方法论，也为其后续承接政府信息化、企业数字化转型类项目积累了重要实践经验，进一步提升了公司在行业内的核心竞争力。对企业数字化转型整体进程来说，该项目通过科学、规范的进度管理实现平台快速落地，为传统企业业务上云、流程线上化提供了可靠支撑，有效推动企业运营效率提升与管理模式升级，同时也为同类企业在数字化转型中开展平台建设、控制项目风险、保障交付时效提供了可借鉴的实践范例。

本文有效解决了项目原有管控痛点，实现了项目按期交付，验证了优化方案的可行性与有效性，但结合项目实践全过程及企业数字化转型长远发展需求，仍存在一些不足，据此提出针对性展望，为C公司后续项目管理升级及行业同类项目发展提供方向。

（1）项目进度管理数据未形成系统性沉淀。本次项目优化过程中，产生了大量有价值的进度管理数据，包括各任务工期数据、方差与标准差等风险量化数据、资源配置数据、缓冲消耗数据及进度偏差数据等，但这些数据仅用于本次项目的进度优化与效果验证，未建立专门的数据存储与管理体系，未实现数据的规范化整理、分类与沉淀，导致数据无法重复利用，难以为C公司后续同类项目提供数据支撑。因此，C公司需要建立一套进度管理数据体系，明确数据采集范围、标准与流程，将任务分解数据、工期数据、风险量化数据、资源利用数据、缓冲消耗数据等全部纳入数据体系。通过数据沉淀，形成C公司专属的项目管理数据库，为后续项目的进度估算、方案优化提供真实、可靠的数据支撑，实现项目管理从“经验驱动”向“数据驱动”转型。

（2）进度管理的智能化水平较低。本次项目进度管控主要依赖传统的PERT三点估算法、CPM关键路径法、关键链法等管理方法，结合人工监控与分析实现进度优化，未引入智能化技术手段。对于项目周期预估、风险点识别等工作，仍以人工结合历史经验判断为主，缺乏精准的智能化预测与预警能力，难以应对未来更复杂、规模更大的数字化转型平台项目的管控需求。为此，未来可以引入AI算法技术，提升进度管理智能化水平。在积累足够的项目管理数据后，依托大数据分析技术，引入合适的AI算法，基于历史数据训练项目周期预估模型，实现对未来项目工期的精准预测；同时，构建风险预警模型，对项目执行过程中的关键风险点（进行提前识别与告警，实现风险的主动防控。通过AI技术与项目进度管理的深度融合，减少人工干预，提升管控效率与精准度，适配复杂数字化转型项目的管理需求。

参考文献

[1] 中华人民共和国工业和信息化部. 2025 年上半年软件业运行情况[EB/OL]. https://wap.miit.gov.cn/gxsj/tjfx/rjy/art/2025/art_fcce29bc7cfb4e71b35ac1c2033264c8.html.

[2]Stellman,Andrew,Greene, et al. Applied software project management[M]: " O'reilly Media, Inc.", 2005.

[3]杜晓利. 富有生命力的文献研究法[J]. 上海教育科研, 2013, No.317(10): 1.

[4]李本乾. 描述受众特征 揭示因果关系——调查研究法简介[J]. 当代传播, 2000, (03): 38-40.

[5]王艳双. 案例分析法在企业管理课程教学中的应用[J]. 电大理工, 2018, No.274(01): 68-69+75.

[6]Project Management Institute. A Guide to the Project Management Body of Knowledge (PMBOK® Guide)–Seventh Edition and The Standard for Project Management[C]: Project Management Institute, 2021.

[7]刘婧. 项目进度管理方法研究——关键路径法[J]. 营销界, 2020, (16): 152-154.

[8]田丰春. 项目进度管理研究[J]. 中国科技信息, 2008, (14): 1-3.

[9]范俊辉. 软件开发项目进度管理[J]. 数字通信世界, 2018, (02).

[10]尹世强. 项目进度管理的有效方式[J]. 山西科技, 2006, (01): 40-41+63.

[11]吕景丽. 软件项目管理中的进度控制问题及对策分析[J]. 无线互联科技, 2019, 16(12): 45-46.

[12]谭泽涛. 基于关键路径法的项目进度管理研究[J]. 建筑经济, 2019, 40(09): 67-71.

[13]周晓丽. 项目进度管理知识研究[J]. 现代商贸工业, 2009, 21(08): 49-50.

[14]戚安邦. 项目管理学[M]: 科学出版社, 2019.

[15]Naur,Peter,Randell, et al. Software Engineering: Report of a conference sponsored by the NATO Science Committee, Garmisch, Germany, 7-11 Oct. 1968, Brussels, Scientific Affairs Division, NATO[M], 1969.

[16]Brooks. No Silver Bullet Essence and Accidents of Software Engineering[J]. Computer, 1987, 20(4): 10-19.

[17]Roggenbach,Markus,Schlingloff, et al. Formal Methods[M]: Springer International Publishing, 2022: 1-46.

[18]Royce,Winston W. Managing the development of large software systems: concepts and techniques[C]//Proceedings of the 9th International Conference on Software Engineering, 1987: 328-338.

[19]张腾, 王迎军. 迭代式创新的研究与实践发展_张腾[J]. 现代管理科学, 2016, No.283(10): 100-102.

[20]Manifesto,Agile. Manifesto for agile software development[Z], 2001.

[21]Schwaber,Ken. SCRUM Development Process[C]: Springer London, 1997: 117-134.

[22]Beck,Kent. Embracing change with extreme programming[J]. Computer, 1999, 32(10): 70-77.

[23]Ahmad,Muhammad Ovais,Markkula, et al. Kanban in software development: A systematic literature review[C]//2013 39th Euromicro Conference on Software Engineering and Advanced Applications: Ieee, 2013: 9-16.

[24]袁航, 李育芸, 陈娇娇, 等. 基于敏捷开发与 DevOps 协同的软件开发效能提升策略研究_袁航[J]. 电脑知识与技术, 2025, 21(11): 57-60+63.

[25]杨世利, 安敬勇, 宋阳, 等. 基于 DevOps 的研发运维一体化平台设计与实现_杨世利[J]. 计算机应用与软件, 2025, 42(02): 66-71.

[26]刘博涵, 张贺, 董黎明. DevOps 中国调查研究[J]. 软件学报, 2019, 30(10): 3206-3226.

[27]宋伟. 项目管理学[M]: Beijing Book Co. Inc., 2013.

[28]潘广钦. 项目进度管理研究综述[J]. 价值工程, 2014, 33(31): 86-89.

[29]虞世杰. 建筑工程进度管理中关键路径优化研究[C]//智慧建筑与智能经济建设学术研讨会论文集（三）, 2025: 1108-1112.

[30]MacCrimmon,Kenneth R,Ryavec, et al. An analytical study of the PERT assumptions[J]. Operations Research, 1964, 12(1): 16-37.

[31]张伟梁, 任璟. 项目进度管理中 CPM、PERT 和 CCPM 的比较研究[J]. 中国高新技术企业, 2011, No.175(04): 136-137.

[32]马国丰, 陈强. 项目进度管理的研究现状及其展望[J]. 上海管理科学, 2006, (04): 70-74.

[33]Goldratt,Eliyahu M. Critical chain, 1997[J]. Virine, Lev and Trumper, Michael. Schedule Network Analysis Using Event Chain. Sl: Projectdecisions. Org, 1997.

[34]马鑫. 基于关键链理论下项目进度管理的优化路径[J]. 产业创新研究, 2023, No.116(15): 166-168.

[35]李宁, 吴之明. 网络计划技术的新发展——项目关键链管理(CCPM)[J]. 公路, 2002, (10): 83-86.

[36]姜海莹. 关键链项目管理法（CCPM）研究综述与展望[J]. 建筑施工, 2019, 41(09): 1764-1769.

[37]黄振宇, 张文雅. 面向关键链的敏捷软件开发项目进度管理研究[J]. 科技创新与应用, 2016, No.182(34): 281.

[38]陈仲平. 信息系统项目中的进度管理分析[J]. 电子技术, 2023, 52(06): 81-83.

[39]张玉婷, 杨镜宇. 基于 WBS 的某决策支持系统开发项目进度管理研究[J]. 项目管理技术, 2023, 21(10): 142-148.

[40]詹伟. PERT 技术在软件项目管理中的实践探究[J]. 信息与电脑(理论版), 2021, 33(20): 1-3.

[41]周小桥. 关键路径法:确定项目的工期及活动的时差[J]. 项目管理技术, 2006, (03): 73-76.

[42]蔡晨, 万伟. 基于 PERT/CPM 的关键链管理[J]. 中国管理科学, 2003, (06).

[43]邢梓烁. 基于 PERT 和人力资源约束条件下的项目进度优化——以 J 公司香精香料研发项目为例[J]. 项目管理技术, 2024, 22(07): 59-67.

[44]李恒强. 基于关键链法的工程项目进度管理与优化策略[J]. 建设监理, 2025, No.320(11): 23-26.

[45]褚春超. 缓冲估计与关键链项目管理[J]. 计算机集成制造系统, 2008, No.121(05): 1029-1035.

[46]黄建文, 王亚珂, 王宇峰, 等. 资源柔性视角下项目进度管理关键链缓冲区设置[J]. 人民长江, 2022, 53(04): 149-154.

[47]赵岫华, 马军杰, 乞建勋. 基于 CPM 网络的关键链汇入缓冲设置研究[J]. 技术经济与管理研究, 2011, No.180(07): 7-11.

[48]徐哲, 王黎黎. 基于关键链技术的项目进度管理研究综述[J]. 北京航空航天大学学报(社会科学版), 2011, 24(02): 54-59.

[49]徐小琴, 韩文民. 关键链汇入缓冲区的设置方法[J]. 工业工程与管理, 2007, No.66(05): 51-55.

[50]查京民, 王波. 关键链法在项目进度规划中的应用研究[J]. 工程管理学报, 2014, 28(05): 73-77.

[51]张俊光, 沈正凡. 基于资源冲量的关键链项目缓冲监控方法[J]. 科技管理研究, 2023, 43(09): 191-196.

附录 A

尊敬的项目参与同事：

您好！为了解梳理 C 公司业务应用运行平台项目的进度管理现状，排查项目实施过程中的影响项目进度的因素，以便后期对项目进度管理进行改善提升，现做此问卷调查，请您在百忙中抽出宝贵的时间填写一下，所有问题仅供科研及工作参考，本次调查对您所提供的信息我们会严格保密，请根据本项目的实际状况认真回答，真诚地希望您能提供真实的答案，谢谢您的协助和支持!

一、基本信息

1. 您所在的项目团队：

□ 项目管理 □ 产品设计 □ 技术开发 □ 测试验证 □ 运维支撑

2. 您在项目中的角色：

□ 项目经理 □ 核心负责人 □ 技术骨干 □ 普通成员 □ 外部协作人员

3. 您参与项目的阶段（可多选）：

□ 项目启动 □ 需求分析 □ 系统设计 □ 系统开发 □ 系统测试 □ 数据迁移 □ 系统验收 □ 全程参与

二、项目进度管理问题

4. 项目工作分解未覆盖关键细节，存在遗漏或模糊的任务项。

□ 完全不符合 □ 较不符合 □ 一般 □ 较符合 □ 完全符合

5. 项目任务的时间预估依赖主观经验，缺乏历史数据或行业标准支撑。

□ 完全不符合 □ 较不符合 □ 一般 □ 较符合 □ 完全符合

6. 用户需求频繁变更，开发任务反复修改。

□ 完全不符合 □ 较不符合 □ 一般 □ 较符合 □ 完全符合

7. 项目人力配置与任务复杂度、工作量不匹配。

□ 完全不符合 □ 较不符合 □ 一般 □ 较符合 □ 完全符合

8. 关键技术资源集中在单一环节，导致部分阶段资源闲置、部分阶段资源短缺。

☐ 完全不符合 ☐ 较不符合 ☐ 一般 ☐ 较符合 ☐ 完全符合

9. 硬件、软件等技术资源未提前预置，出现资源缺口时延误工作推进。

☐ 完全不符合 ☐ 较不符合 ☐ 一般 ☐ 较符合 ☐ 完全符合

10. 跨团队协作时缺乏高效对接机制，需求变更、问题反馈不及时。

☐ 完全不符合 ☐ 较不符合 ☐ 一般 ☐ 较符合 ☐ 完全符合

11. 未建立明确的进度偏差预警机制，出现问题后被动应对。

☐ 完全不符合 ☐ 较不符合 ☐ 一般 ☐ 较符合 ☐ 完全符合

三、开放性建议

12. 结合您的项目经验，除上述因素外，您认为导致项目进度落后的其他关键原因是什么？

13. 针对项目任务分解、评估、资源分配及沟通监控等环节，您有哪些具体的优化建议？